

QQ群：628096131

数据结构

第一章 绪论

郭岗

计算机学院 网络技术中心

13370139202

guogang@bupt.edu.cn



2024秋 数据结构课程

群号：628096131



扫一扫二维码，加入群聊

考核方式

- 平时成绩：
 - 考勤，作业，期中
 - 课堂纪律
 - 不要无故迟到：
 - 不要无故旷课：
 - 不要玩游戏、上网聊天
- 期末成绩：考试
- 课堂上，有问题提出来讨论

教材和参考书

➤ 教材:

- 《数据结构》(C语言版), 严蔚敏, 吴伟民等, 清华大学出版社

➤ 参考书:

- 《数据结构题集 (C语言版) 》, 严蔚敏等
- 《数据结构与算法分析 (《Data Structures and Algorithm Analysis in C》) 》, Mark Allen Weiss, 机械工业出版社



❖ 做几个C/C++的编程小练习

- 1、从命令行输入5个整数，输出其中的最大值
- 2、从命令行输入5个整数，依次输出其中的最大值和最小值
- 3、从命令行输入5个整数，依次输出其中的最大值、最小值和第三大的值
- 4、从命令行输入5个整数，依次输出其中的最大值、第二大的值、第三大的值、第四大的值和最小值
- 5、从命令行输入若干个不为0的整数，以0为输入结束，将这些不为0的整数按照从大到小的顺序依次输出

	主要变量使用方式	程序主要逻辑（程序实现、算法）
1、从命令行输入5个整数，输出其中的最大值	2个整数类型变量： 1个存放当前最大值，1个存放当前输入的数	程序开始运行时，将当前最大值初始化为整数中的最小值。每次输入时将两个变量进行比较，读入的数和当前最大值找出那个大，将大的值保存到当前最大值里，输入结束后输出当前最大值
2、从命令行输入5个整数，依次输出其中的最大值和最小值		
3、从命令行输入5个整数，依次输出其中的最大值、最小值和第三大的值		
4、从命令行输入5个整数，依次输出其中的最大值、第二大的值、第三大的值、第四大的值和最小值		
5、从命令行输入若干个不为0的整数，以0为输入结束，将这些不为0的整数按照从大到小的顺序依次输出		

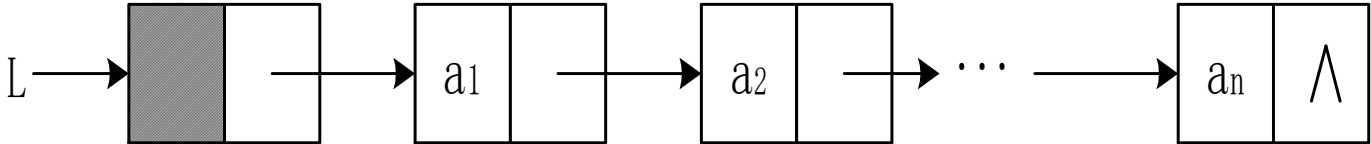
	主要变量使用方式	程序主要逻辑（程序实现、算法）
1、从命令行输入5个整数，输出其中的最大值	2个整数类型变量： 1个存放当前最大值，1个存放当前输入的数	程序开始运行时，将当前最大值初始化为整数中的最小值。每次输入时将两个变量进行比较，读入的数和当前最大值找出那个大，将大的值保存到当前最大值里，输入结束后输出当前最大值
2、从命令行输入5个整数，依次输出其中的最大值和最小值	3个整数类型变量： 1个存放当前最大值，1个存放当前最小值，1个存放当前输入的数	在1的基础上，增加一下比较并保存最小值的程序逻辑
3、从命令行输入5个整数，依次输出其中的最大值、最小值和第三大的值		
4、从命令行输入5个整数，依次输出其中的最大值、第二大的值、第三大的值、第四大的值和最小值		
5、从命令行输入若干个不为0的整数，以0为输入结束，将这些不为0的整数按照从大到小的顺序依次输出		

	主要变量使用方式	程序主要逻辑（程序实现、算法）
1、从命令行输入5个整数，输出其中的最大值	2个整数类型变量： 1个存放当前最大值，1个存放当前输入的数	程序开始运行时，将当前最大值初始化为整数中的最小值。每次输入时将两个变量进行比较，读入的数和当前最大值找出那个大，将大的值保存到当前最大值里，输入结束后输出当前最大值
2、从命令行输入5个整数，依次输出其中的最大值和最小值	3个整数类型变量： 1个存放当前最大值，1个存放当前最小值，1个存放当前输入的数	在1的基础上，增加一下比较并保存最小值的程序逻辑
3、从命令行输入5个整数，依次输出其中的最大值、最小值和第三大的值	6个整数类型变量：5个整型变量按大小保存5个整数，1个存放当前输入数 或 1个5个整数的数组变量	读入后和现有数比较大小，保存到合适的变量中 或 读入的同时做插入排序，将5个数排序后输出或读入5个数后再排序输出
4、从命令行输入5个整数，依次输出其中的最大值、第二大的值、第三大的值、第四大的值和最小值	这时候，我们开始用数组变量了；就是采用了数据结构的方法来处理数据的存储和计算（操作）了：让数据按照某种规律（方法）存入内存中，并且可以按照某种规律（方法）从内存中读取到CPU中。数组使用的规律是：通过数组的下标来读取和存入。	
5、从命令行输入若干个不为0的整数，以0为输入结束，将这些不为0的整数按照从大到小的顺序依次输出		

	主要变量使用方式	程序主要逻辑（程序实现、算法）
1、从命令行输入5个整数，输出其中的最大值	2个整数类型变量： 1个存放当前最大值，1个存放当前输入的数	程序开始运行时，将当前最大值初始化为整数中的最小值。每次输入时将两个变量进行比较，读入的数和当前最大值找出那个大，将大的值保存到当前最大值里，输入结束后输出当前最大值
2、从命令行输入5个整数，依次输出其中的最大值和最小值	3个整数类型变量： 1个存放当前最大值，1个存放当前最小值，1个存放当前输入的数	在1的基础上，增加一下比较并保存最小值的程序逻辑
3、从命令行输入5个整数，依次输出其中的最大值、最小值和第三大的值	6个整数类型变量：5个整型变量按大小保存5个整数，1个存放当前输入数 或 1个5个整数的数组变量	读入后和现有数比较大小，保存到合适的变量中 或 读入的同时做插入排序，将5个数排序后输出 或读入5个数后再排序输出
4、从命令行输入5个整数，依次输出其中的最大值、第二大的值、第三大的值、第四大的值和最小值	1个5个整数的数组变量	读入的同时做插入排序，将5个数排序后输出 或读入5个数后再排序输出
5、从命令行输入若干个不为0的整数，以0为输入结束，将这些不为0的整数按照从大到小的顺序依次输出		

	主要变量使用方式	程序主要逻辑（程序实现、算法）
1、从命令行输入5个整数，输出其中的最大值	2个整数类型变量： 1个存放当前最大值，1个存放当前输入的数	程序开始运行时，将当前最大值初始化为整数中的最小值。每次输入时将两个变量进行比较，读入的数和当前最大值找出那个大，将大的值保存到当前最大值里，输入结束后输出当前最大值
2、从命令行输入5个整数，依次输出其中的最大值和最小值	3个整数类型变量： 1个存放当前最大值，1个存放当前最小值，1个存放当前输入的数	在1的基础上，增加一下比较并保存最小值的程序逻辑
3、从命令行输入5个整数，依次输出其中的最大值、最小值和第三大的值	6个整数类型变量：5个整型变量按大小保存5个整数，1个存放当前输入数 或 1个5	读入后和现有数比较大小，保存到合适的变量中 或 后输出
4、从命令行输入5个整数，依次输出其中的最大值、第二大的值、第三大的值、第四大的值和最小值	1个5	后输出
5、从命令行输入若干个不为0的整数，以0为输入结束，将这些不为0的整数按照从大到小的顺序依次输出	1个整 变量，数组中的数据个数；1个整数数 组变量；当数组大小不够时，需要扩充 或 1个整数链表，1个整数变量记录数量	在数组里；结束后按顺序输出 读入的同时做链表的插入排序，并记录当前有多少个数在链表里；结束后按顺序输出

这时，我们用了一个新的存储数据的方式：整数链表。整数链表的使用规律是：通过一个结点的链接找到下一个结点；不断这样找，直至找到所需数据再进行读取和存入。



为什么要学习数据结构

❖ 思考一下上面五个程序里面

- 1) 我们是如何将程序外部的信息在程序内保存的?
- 2) 我们是如何处理这些保存在程序内的信息并得到结果的?

为什么要学习数据结构

❖ 思考一下上面五个程序里面

1) 我们是如何将程序外部的信息在程序内保存的?

保存在程序的变量中。

2) 我们是如何处理这些保存在程序内的信息并得到结果的?

为什么要学习数据结构

❖ 思考一下上面五个程序里面

1) 我们是如何将程序外部的信息在程序内保存的？

保存在程序的变量中。

2) 我们是如何处理这些保存在程序内的信息并得到结果的？

通过一系列的指令（高级程序设计语言的语句），按照特定的逻辑处理方法，对这些变量值进行判断和重新赋值后得到结果。

为什么要学习数据结构

❖ 思考一下上面五个程序里面

1) 我们是如何将程序外部的信息在程序内保存的？

保存在程序的变量中。

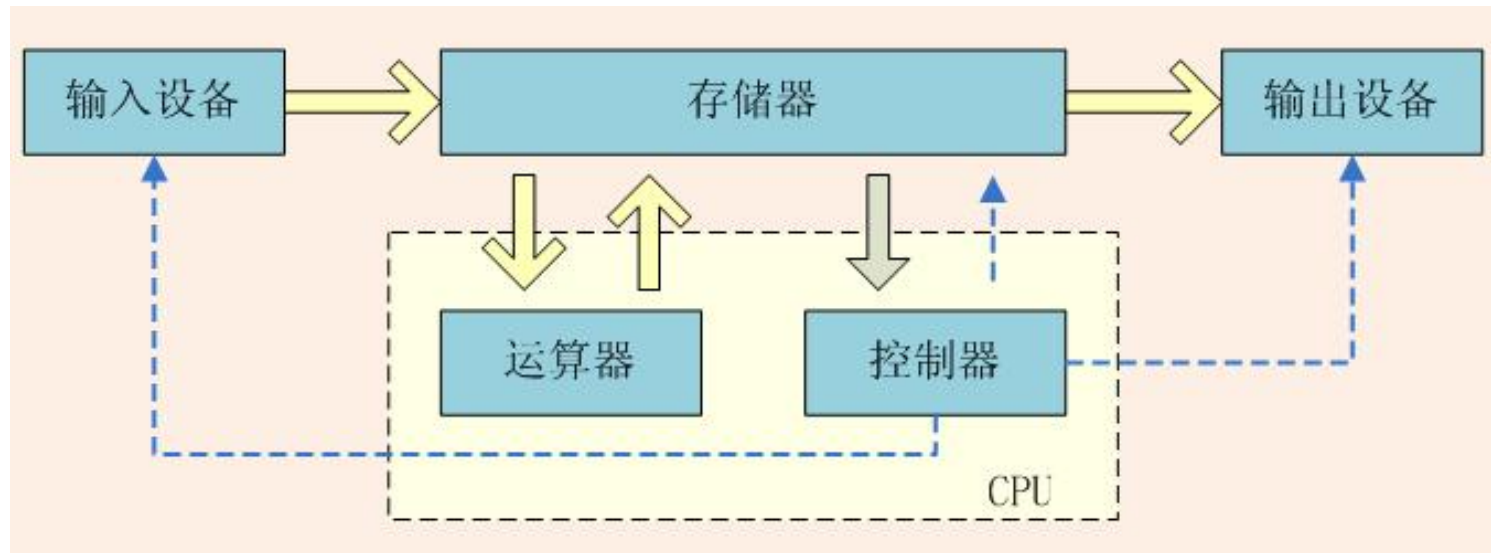
Q: 变量是什么？或者 变量代表了什么？

2) 我们是如何处理这些保存在

通过一系列的指令（高级程序设计语言的语句），按照特定的逻辑处理方法，对这些变量值进行判断和重新赋值后得到结果。

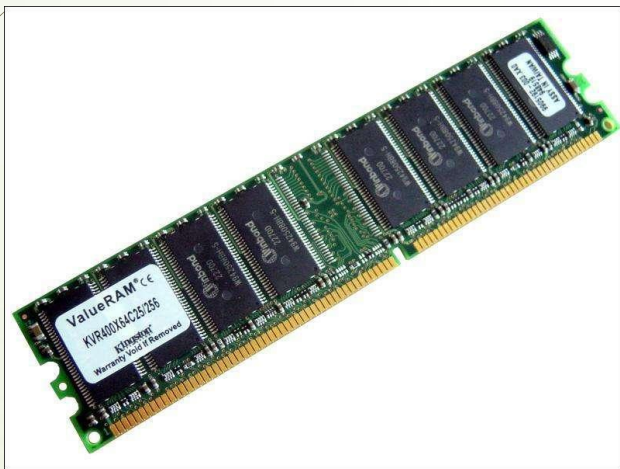
计算机中如何表示/存储和操作？

- 冯.诺依曼体系结构是现代计算机的基础，计算机由控制器、运算器、存储器、输入设备、输出设备五部分组成



计算机中如何表示/存储和操作？

- 按照与CPU的接近程度，存储器分为内存存储器与外存储器，简称内存与外存



内存

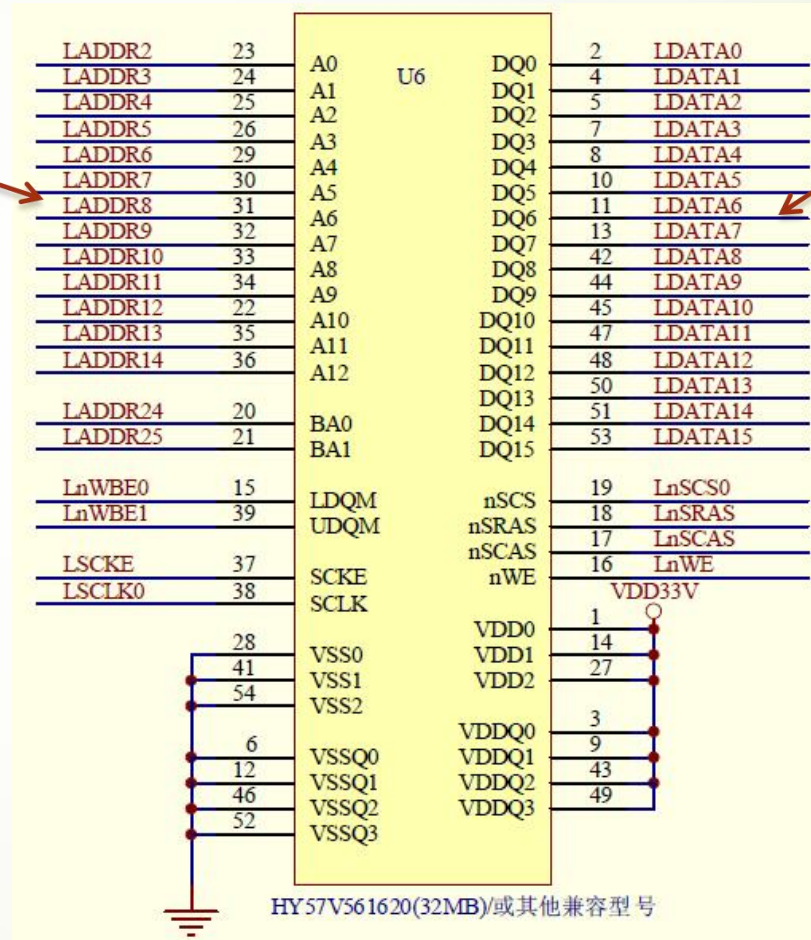


外存

计算机中如何表示/存储和操作？

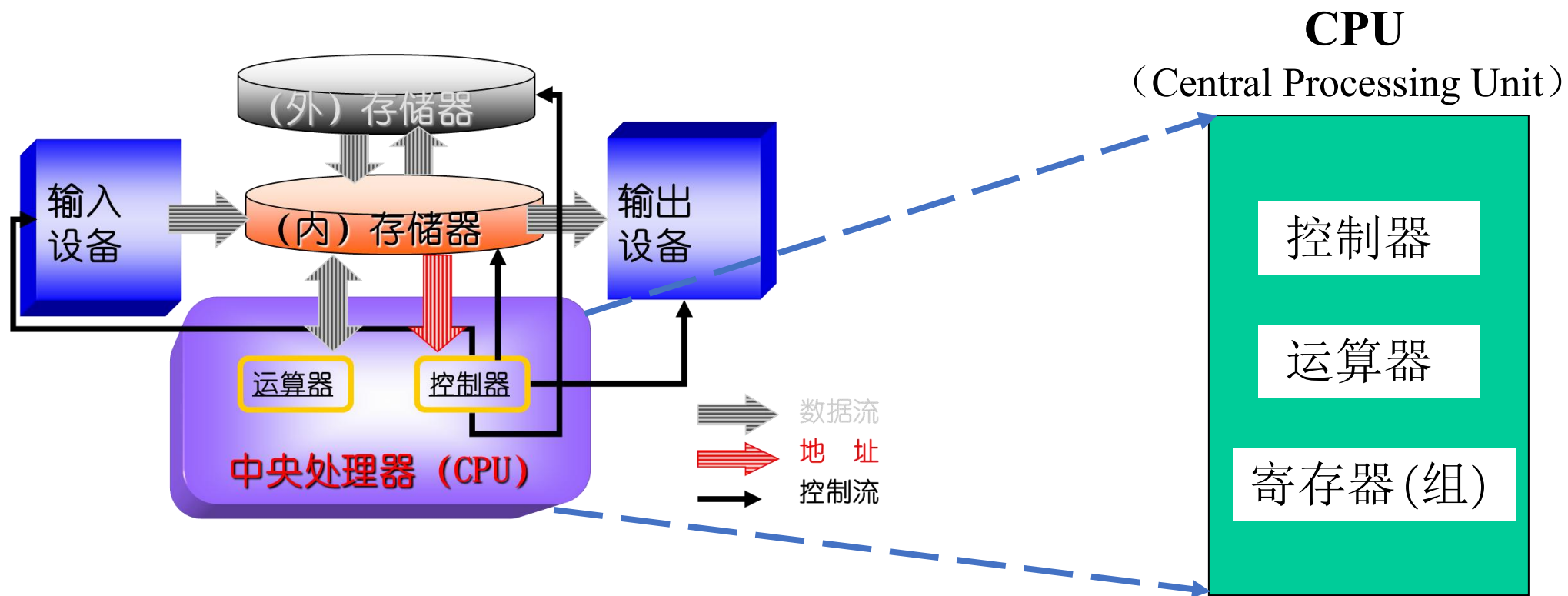


地址总线



数据总线

结合冯诺依曼式计算机的结构 理解程序和变量

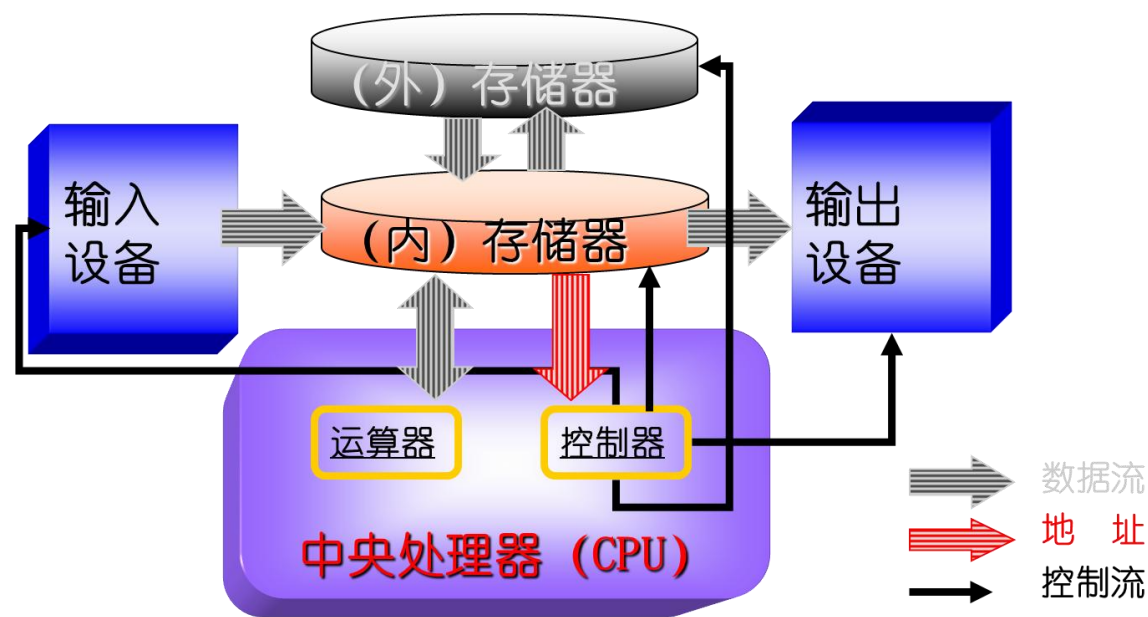


控制器：控制系统一步步从存储器中取出指令进行译码，或从内存中取出数据到寄存器

运算器：根据指令完成对寄存器中数据的算术/逻辑运算

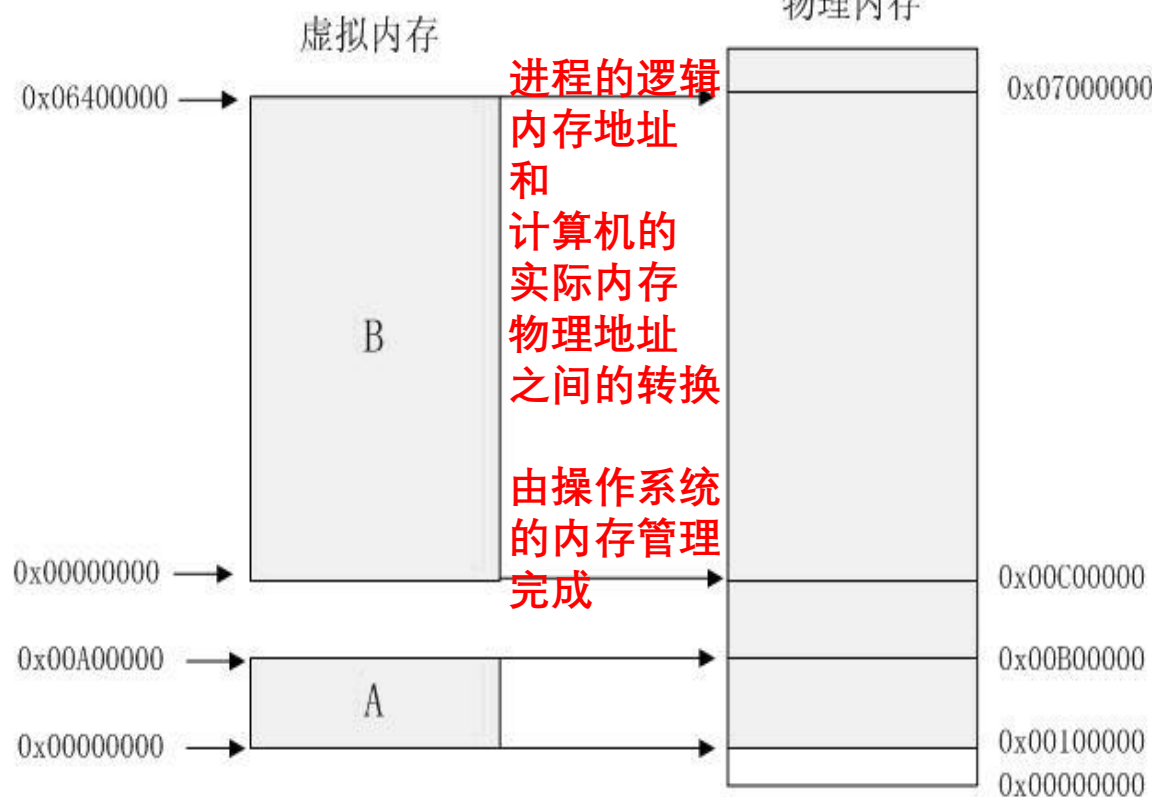
寄存器：存储需要进行运算的数据、保存程序运行状态、当前指令信息地址及下一条指令地址等

结合冯诺依曼式计算机的结构 理解程序和变量



进程的逻辑内存

计算机的实际内存
物理内存



- CPU要进行计算，需要从**内存**中将数据取到CPU中计算
- 为了完成这样一个计算，需要**先把数据放在内存**中
- 如果没有先把数据存入内存，就从内存中取到CPU进行计算，取回的值是多少，程序无法控制
- **内存**可以理解为，若干个存储单元，每个存储单元的大小为字长；为了准确取出放在内存中的某个数据到CPU里面做计算，程序需要**记住**这个数据存在那个存储单元；需要在数据存入内存时，给被存入的数据指定一个存放位置。
- 通过对存储单元进行编号，记住这个编号就可以了，**这个编号就是地址-内存地址**。
- 在OS的支持下，我们这样可以抽象的理解内存：是一个从**序号0**开始的、**连续编号**的**线性表**，表中每个存储单元都有**自己的唯一序号**（即，**这个存储单元的内存地址**）

结合冯诺依曼式计算机的结构 理解程序和变量

编程是在干嘛？

1. 让数据从外设**保存到内存**;
 2. 让CPU**从内存中将数据取出**做计算（加减乘除、整除、取模，以及**比较**，等等）
 3. 计算后在**放回到内存中**
 4. 让**内存中的数据输出**到外设;
0. 为**每个进入内存的数据分配存储单元**。（可以是在编写程序时分配（变量定义，0.A），也可以是在程序运行时分配（变量动态内存分配，0.B）。）
- 对0、1、2、3、4做各种组合和重复，把数据计算（处理）成为程序员想要的结果

一段C程序：

```
int a;  
int *b;  
sscanf("3","%d",&a);  
a=a+1;  
  
b=(int*)malloc(sizeof(int));  
  
*b=a+2;  
  
printf("%d", *b);  
free(b);
```

改造一下格式：

int a;	0.A
int *b;	0.A
sscanf("3","%d",&a);	1
a=	3
a+1;	2
b=	3
(int*)malloc(sizeof(int));	0.B
*b=	3
a+2;	2
printf("%d", *b);	4
free(b);	0.B

结合冯诺依曼结构 理解程序和变量

一段C程序： 加上一些注释后的这段C程序：

int a=3;

int a=3;

/*此时， 为a变量分配内存， a的值是3*/

int b;

int b;

/*此时， 为b变量分配内存， b的值未知， a的值是3， */

a=a+1;

a=a+1;

/*此时， a的值是4， b的值未知*/

b=a+2;

b=a+2;

/*此时， a的值是4， b的值是6*/

- 思考一下：
- 1) 4和6这两个值， 放在哪里了？ 具体地址是多少？
- 2) a和b在冯诺依曼式计算机的C语言程序中， 起了什么作用？

结合冯诺依曼结构 理解程序和变量

C程序代码	对应的汇编程序代码	汇编代码说明
int a = 3;	mov [010B], 3	将010B地址的内存的值修改为3
int b;		
a=a+1;	mov ax, [010B]	将寄存器ax的值, 修改为010B地址的内存中的值 2
	add ax, 1	将寄存器ax的值加1
	mov [010B], ax	将010B地址的内存的值, 修改为ax的值 3
b=a+2;	mov ax, [010B]	将寄存器ax的值, 修改为010B地址的内存中的值 2
	add ax, 2	将寄存器ax的值加2
	mov [010D], ax	将010D地址的内存的值, 修改为ax的值 3

- 思考一下:
- 1) 4和6这两个值, 放在哪里了? 答: 放在内存中某个存储单元了;
- 具体地址是多少? 答: 这段汇编中, 地址分别是 010B 和 010D;在C语言代码中, 分别是 &a计算出的地址, 和 &b计算出的地址。
- 2) a和b在冯诺依曼式计算机的C语言程序中, 起了什么作用? 答: 起到了名字的作用。为了编程方便, 对值的存储位置起个名字; 值直接用变量名来表示, 存储位置 (内存地址) 用对变量的操作符 & 取出。

程序设计中的变量，是什么？

中文维基百科 x +

wiki.hk.wjbk.site/baike-变量_(程序设计)

在程序设计中，变量（英语：Variable，scalar）是指一个包含部分已知或未知数值或信息（即一个值）之存储地址，以及相对应之符号名称（识别字）。通常使用变量名称引用存储值，将名称和内容分开能让被使用的名称独立于所表示的精确消息之外。电脑源代码中的识别字能在运行期间绑定一个值，且该变量的值可能在程序运行期间改变。程序设计中的变量不一定能直接对应到数学中所谓的变量之概念。在程序设计中，变量的值不一定要为方程或数学公式之一部分。程序设计中的变量可使用在一段可重复的程序：在一处赋值，然后使用于另一处，接着在一次赋值，且以相同方式再使用一次（见迭代）。程序设计中的变量通

Variables - Computer Science x +

computersciencewiki.org/index.php/Variat

Variables

The official definition: in computer programming, a variable is a storage location paired with an associated symbolic name (an identifier) which contains a value.^[4]

In other words, **a variable is a storage location for data.** Variables have names. Some computer language mandate you assign a data type to variables. When asked for the "official definition" (or best definition) please use the definition stated above.

变量的三个要素：名字、值、内存地址。

例：int a=3;

a是名字，值是'3'，值保存在 &a 表示的内存中。

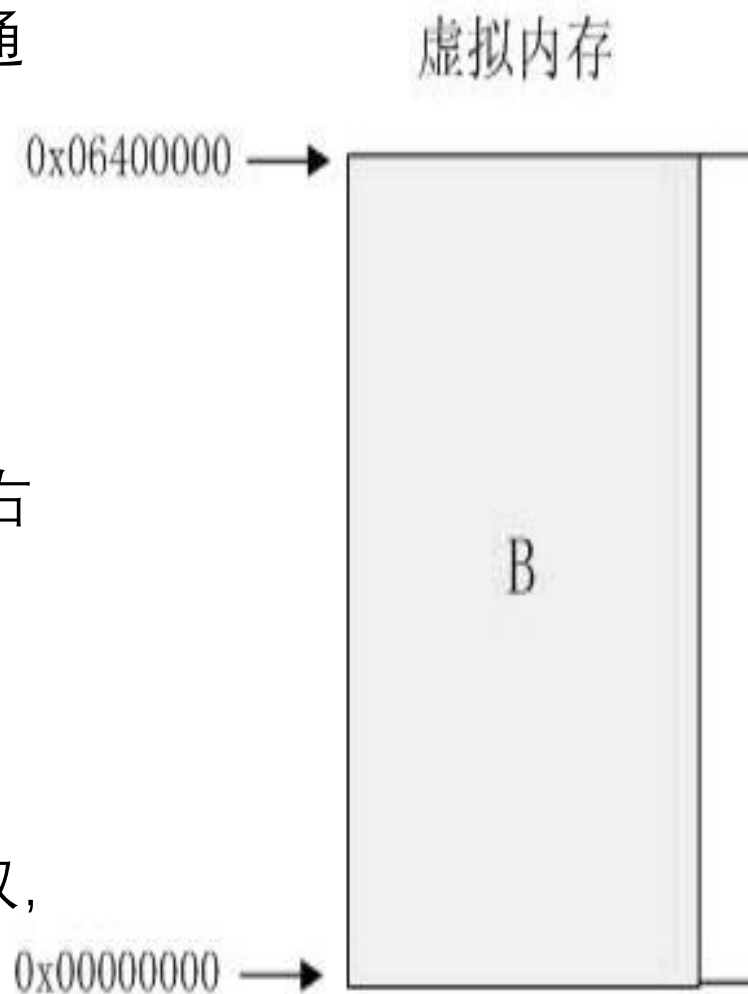
在声明（定义）变量时，编译器给变量指定一块内存来使用，但这个内存中的值是未知的；使用此变量前需要对这块内存赋予初始值。

数据结构课程和内存的关系？

研究在进程级别所使用的虚拟内存中存储数据（逻辑结构的具体实现，即C语言的类型定义和变量定义）、处理数据的通用方法和高效方法（算法）。

要求大家**在前序课程中理解和掌握**如下基础知识：

- 1、内存
- 2、变量：变量名、变量值、变量名所代表的内存地址
- 3、对变量的赋值（ '=' 号的左边）和变量值的使用（ '=' 号的右边）
- 4、存储地址值的变量（某个类型的指针类型的变量）
- 5、如何取得并存储变量的地址（&操作符， malloc、new等函数和方法）
- 6、如何通过变量的地址存（写入，赋值号的左边）取（读取，赋值号的右边）变量的值（*操作符）
- 7、如何在函数调用时将数据通过参数的方式传入函数
- 8、如何在函数中将处理后的数据返回到调用者进一步使用



为什么要学习数据结构

❖ 思考一下上面五个程序里面

1) 我们是如何将程序外部的
保存在程序的变量中。

2) 我们是如何处理这些保存
通过一系列的指令（高级程
逻辑处理方法，对这些变量

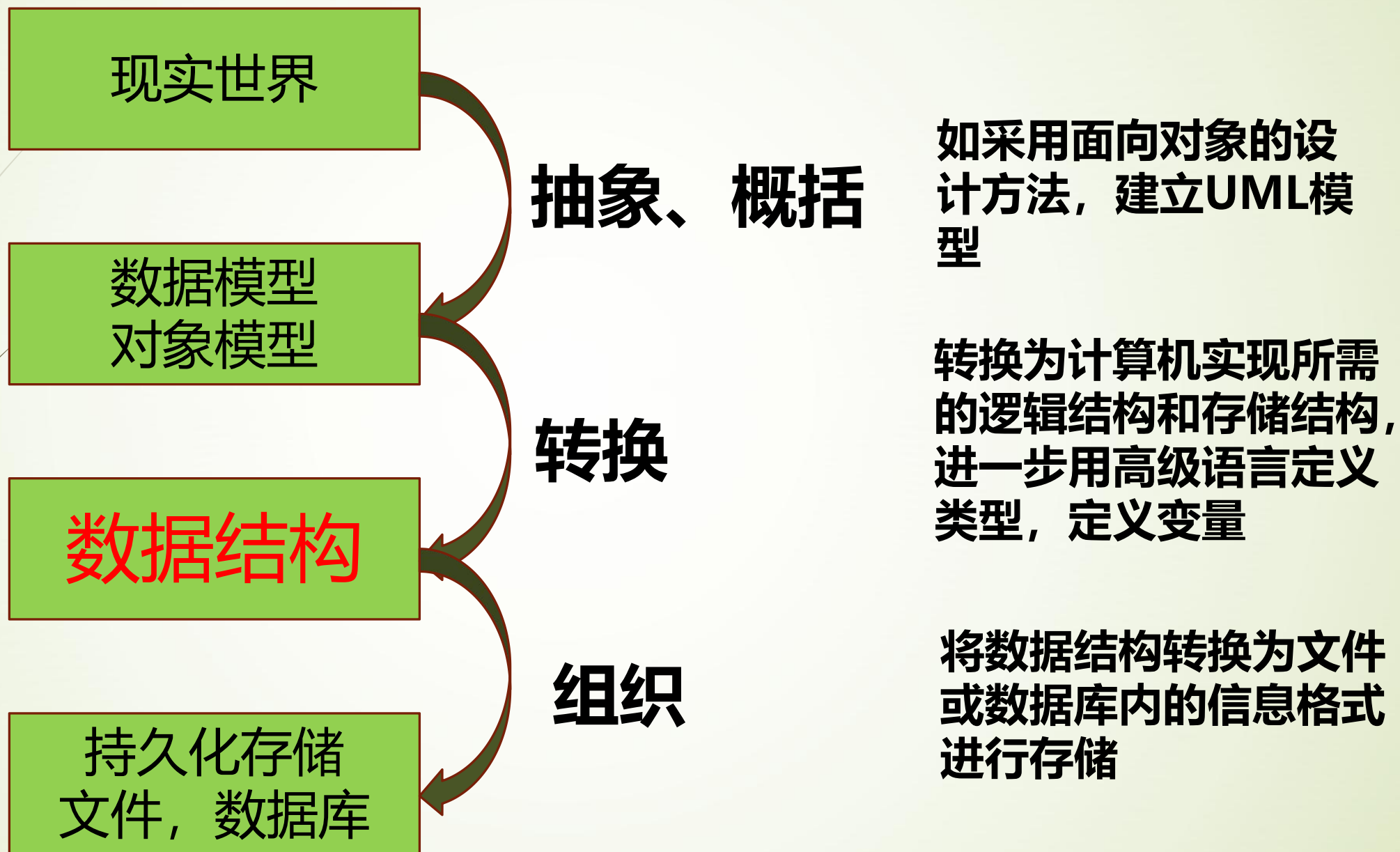
在几十年的过程中，这些存储数据的结构和相应的方法，前辈们已经研究的很透彻了，我们在数据结构课程中，就是学习和掌握其中最基础的一部分

的
果。

3) 在大规模数据情况下，如何在内存中更高效的处理信息？

能在内存中将数据保存下来（增），能快速查找到需要的数据（查），能快速对数据进行重新赋值（改和删）。

采用计算机系统解决问题的方法



为什么要学习数据结构

- ❖ 编程基础和基础编程训练

- ❖ 应用计算机解决问题时，对需要保存和处理的数据

- ❖ 分清数据的内在联系

- ❖ 互不关联，一对一，一对多，多对多等

- ❖ 合理组织数据，选择合理的方式表达外部事物

- ❖ 表、树、图

- ❖ 有效处理数据

- ❖ 选择快速、节省资源的**数据结构和算法**解决问题

为什么要学习数据结构

❖ 考察一下 n 个整数的排序问题

为什么要学习数据结构

❖ 考察一下 n 个整数的排序问题

❖ 1) 采用数组的方式存储，使用**简单选择排序**方法实现 n 个整数的排序

	65	38	27	97	76	03	11	49
第一趟选择	03	38	27	97	76	65	11	49
第二趟选择	03	11	27	97	76	65	38	49
第三趟选择	03	11	27	97	76	65	38	49
第四趟选择	03	11	27	38	76	65	97	49
第五趟选择	03	11	27	38	49	65	97	76
第六趟选择	03	11	27	38	49	65	97	76
第七趟选择	03	11	27	38	49	65	76	97

为什么要学习数据结构

❖ 考察一下 n 个整数的排序问题

❖ 1) 采用数组的方式存储，使用**简单选择排序**方法实现 n 个整数的排序

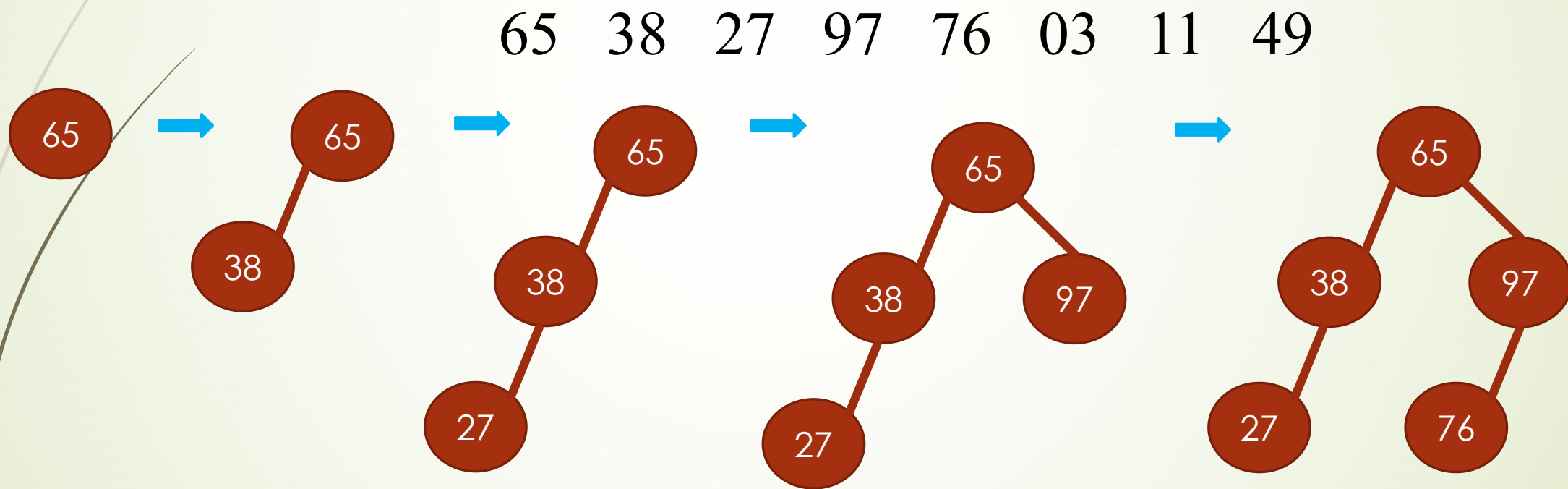
	65	38	27	97	76	03	11	49
第一趟选择	03	38	27	97	76	65	11	49
第二趟选择	03	11	27	97	76	65	38	49
第三趟选择	03	11	27	97	76	65	38	49
第四趟选择	03	11	27	38	76	65	97	49
第五趟选择	03	11	27	38	49	65	97	76
第六趟选择	03	11	27	38	49	65	97	76
第七趟选择	03	11	27	38	49	65	76	97

完成 n 个数的排序，需要 $n-1$ 趟，第 m 趟要进行 $n-m$ 次比较来找出最小值。

为什么要学习数据结构

❖ 考察一下 n 个整数的排序问题

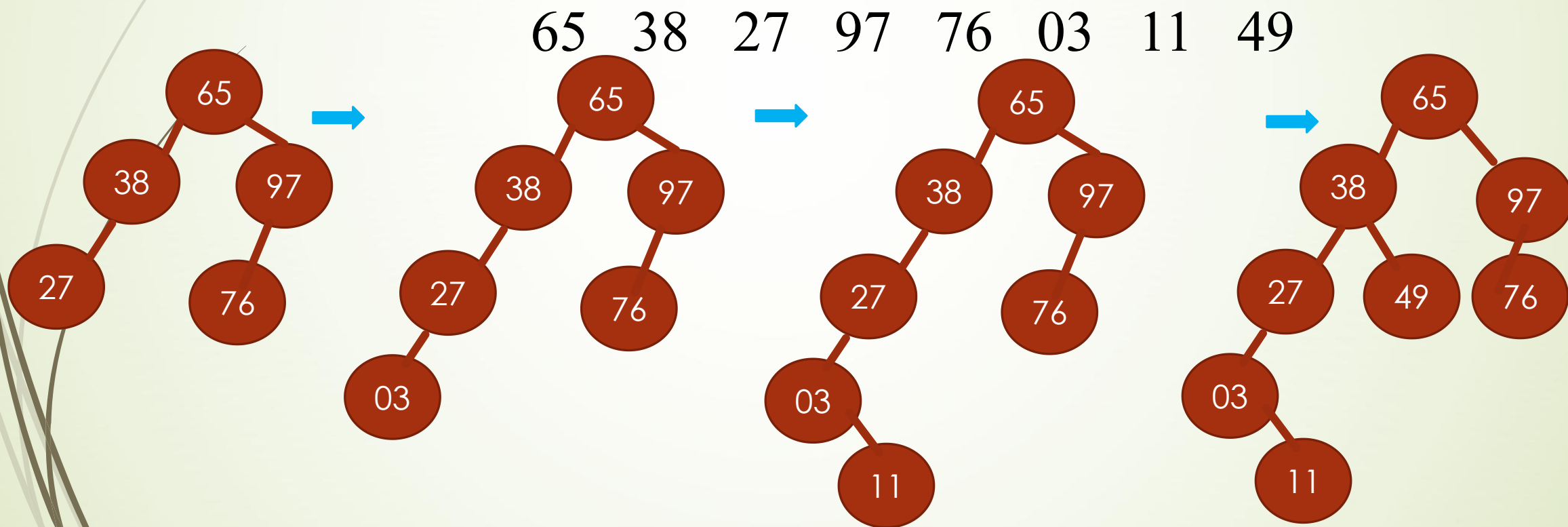
❖ 2) 换一种数据结构，二叉树来思考一下，把数据依次插入到一颗树中，插入时，比当前值大的向右边走，小的向左边走：



为什么要学习数据结构

❖ 考察一下 n 个整数的排序问题

❖ 2) 换一种数据结构，二叉树来思考一下，把数据依次插入到一颗树中，插入时，比当前值大的向右边走，小的向左边走：



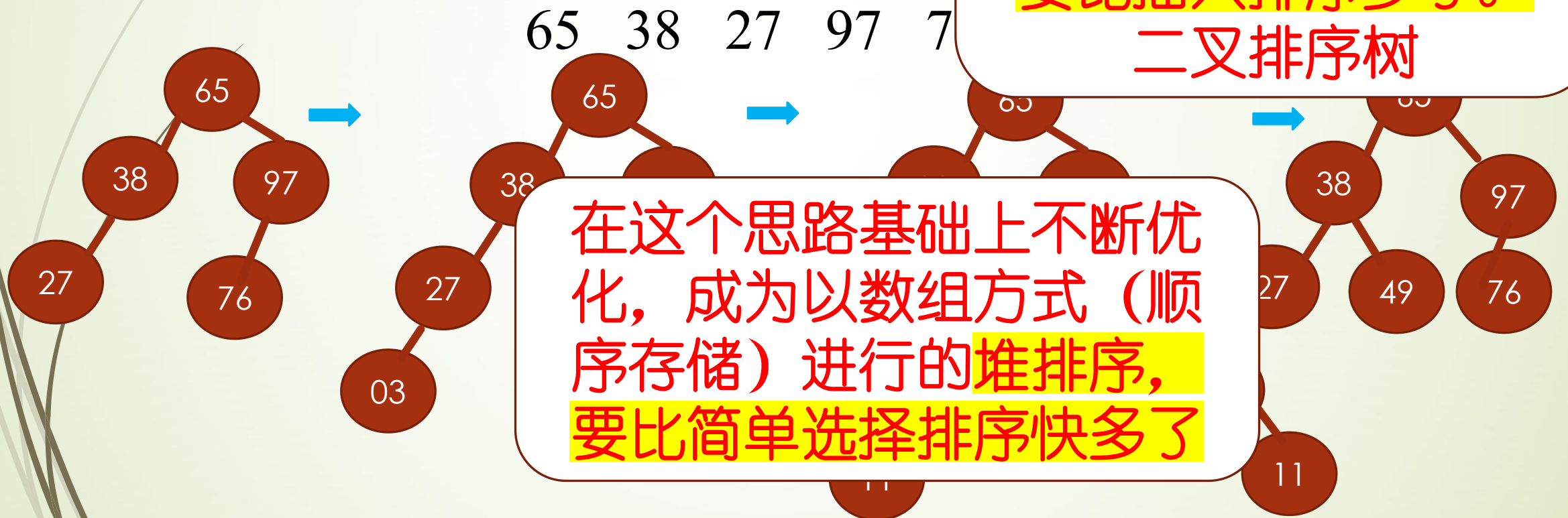
为什么要学习数据结构

❖ 考察一下 n 个整数的排序问题

❖ 2) 换一种数据结构，二叉树来思考一颗树中，插入时，比当前值大的向右

完成 n 个数的排序，依然需要 $n-1$ 趟，但是第 m 趟要进行的比较次数要比插入排序少了。

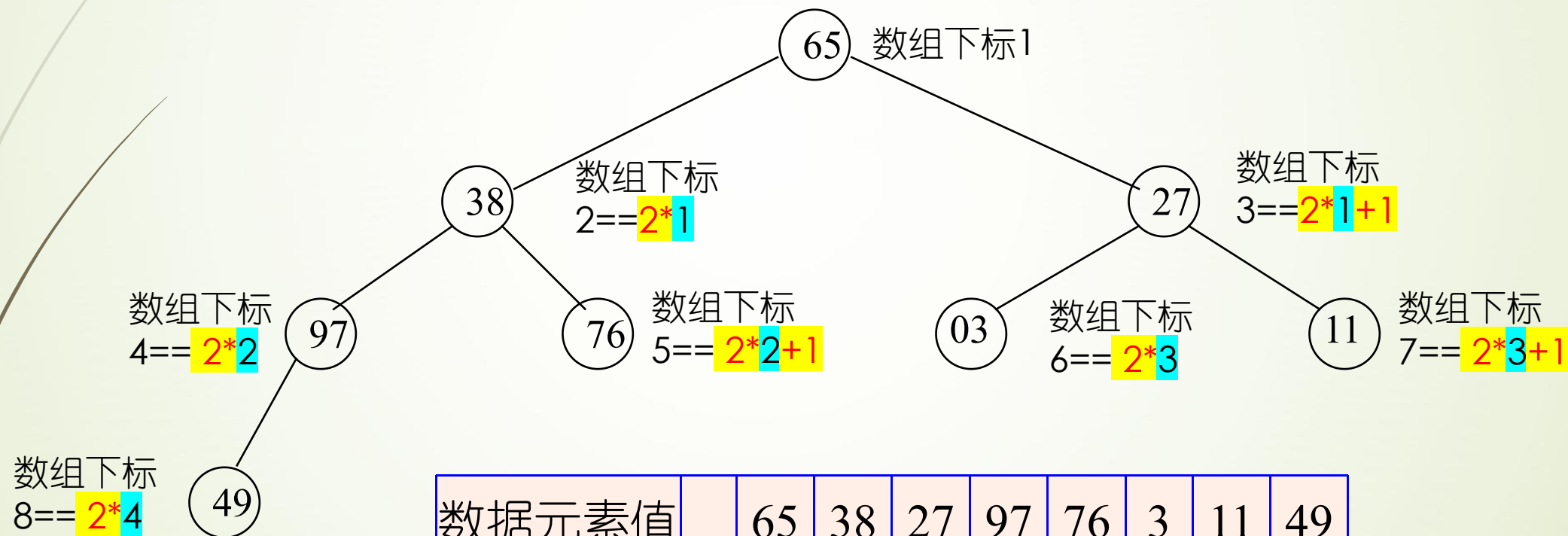
二叉排序树



为什么要学习数据结构

❖ 考察一下n个整数的排序问题

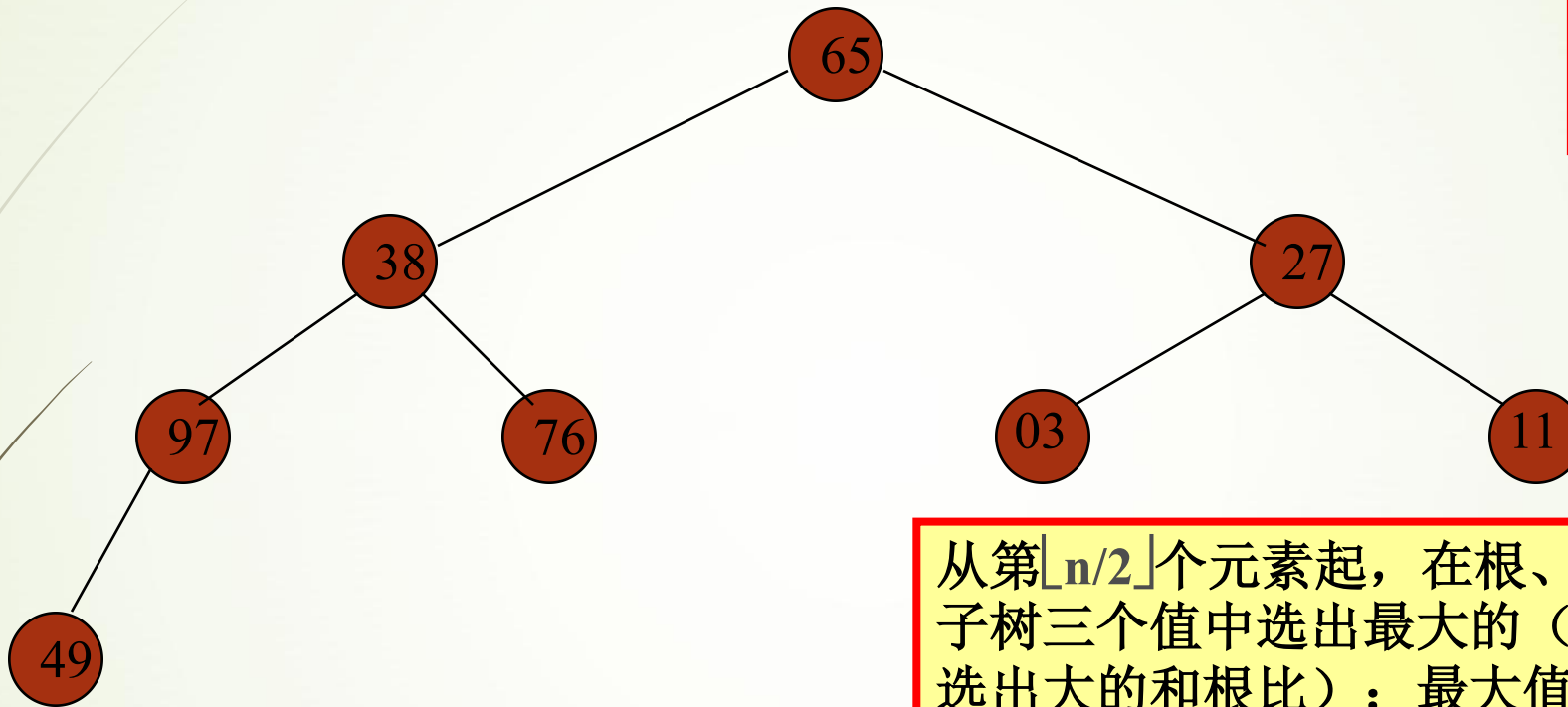
❖ 3) 堆排序：把数组表示的数据当做一棵二叉树来思考



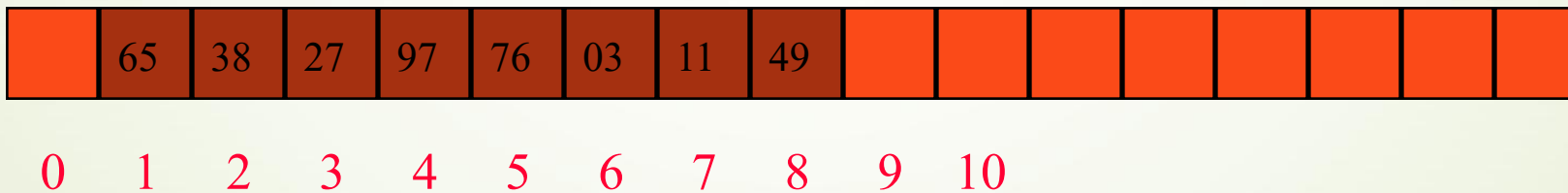
数据元素值		65	38	27	97	76	3	11	49
数组下标	0	1	2	3	4	5	6	7	8

A Heap Is Efficiently Represented As An Array

无序数据
构造为
大根堆

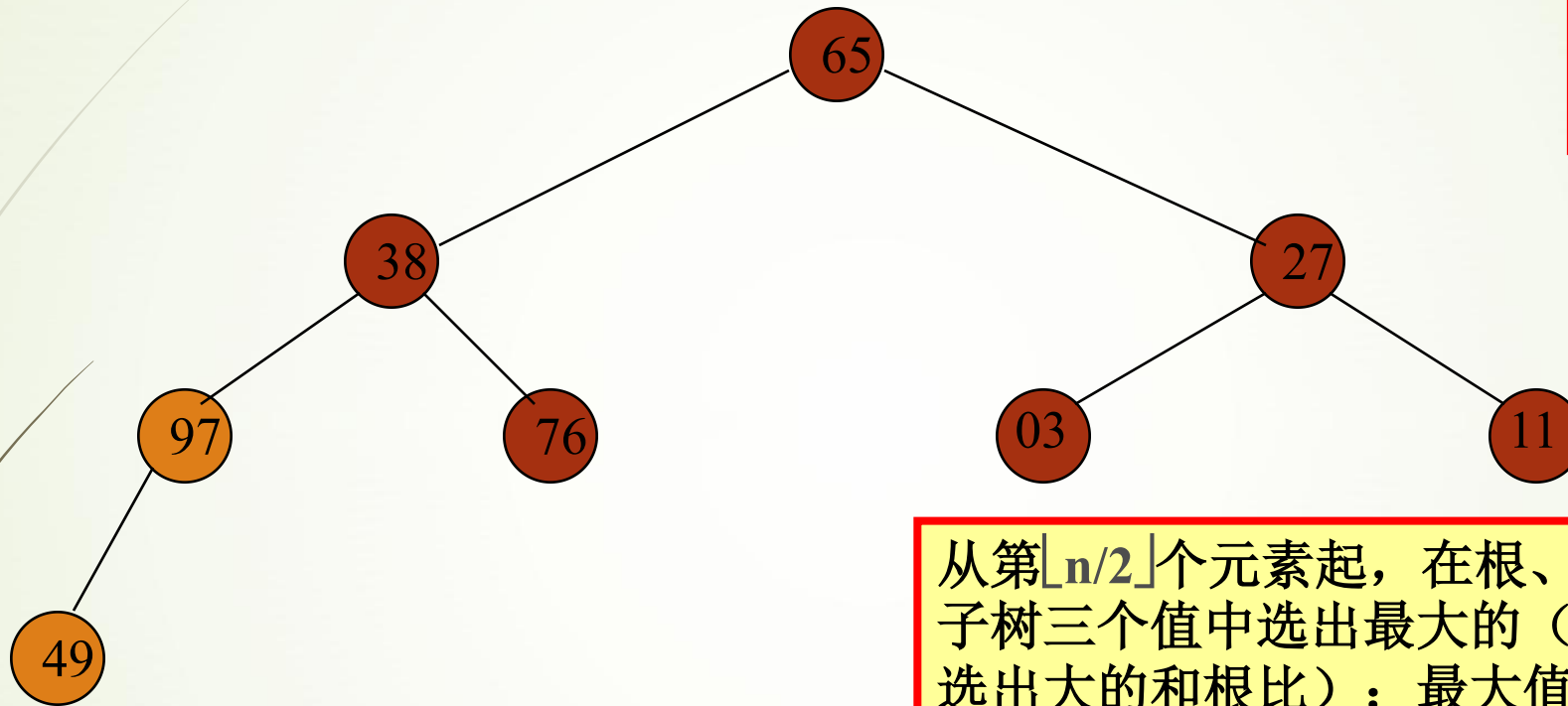


从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。

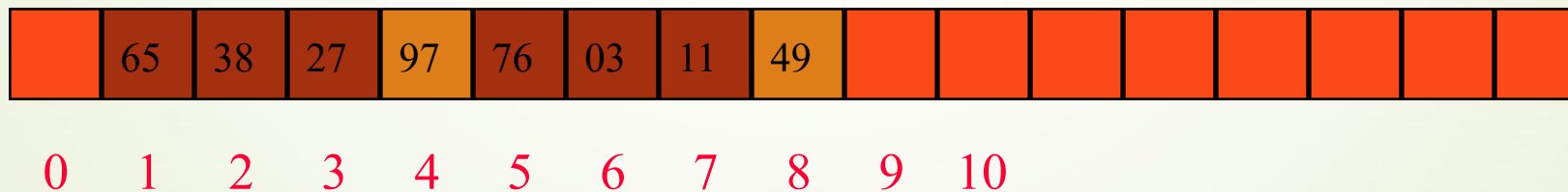


A Heap Is Efficiently Represented As An Array

无序数据
构造为
大根堆

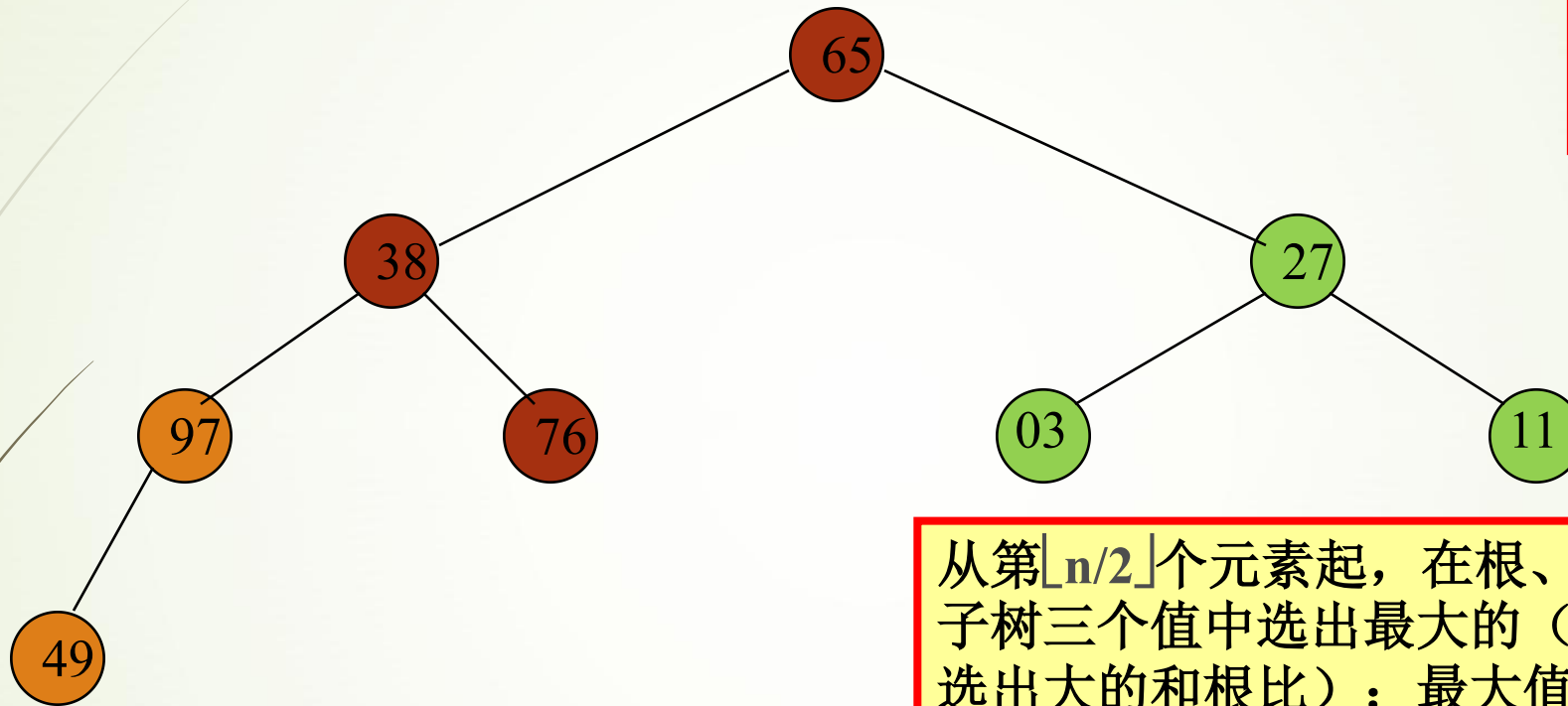


从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。

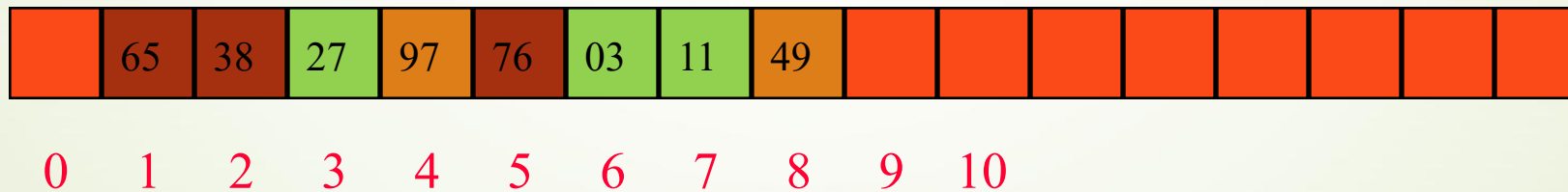


A Heap Is Efficiently Represented As An Array

无序数据
构造为
大根堆

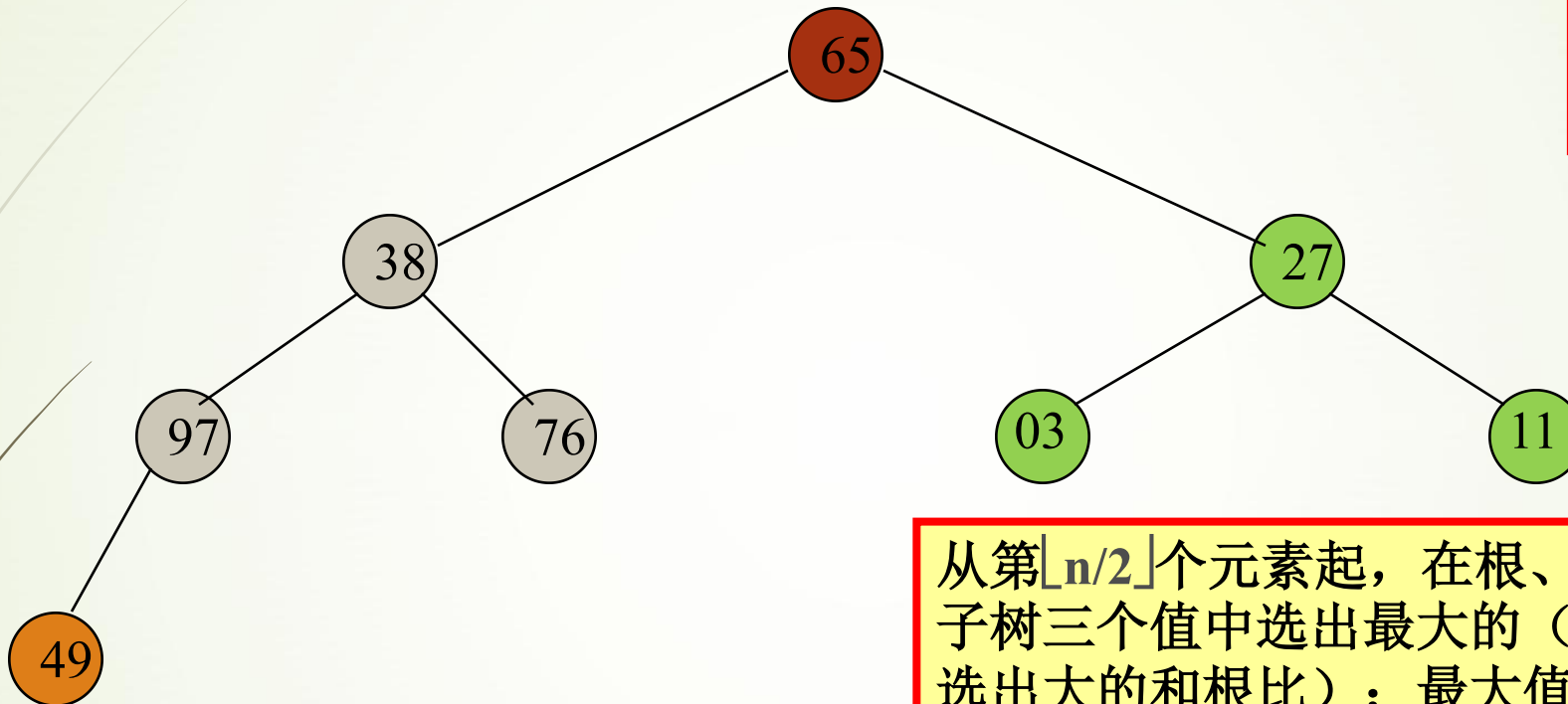


从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。

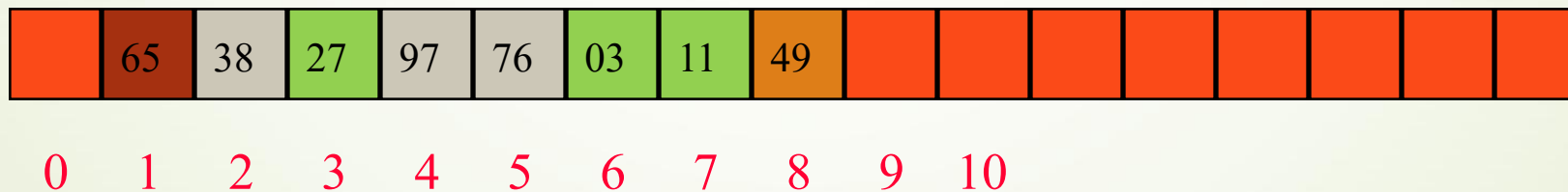


A Heap Is Efficiently Represented As An Array

无序数据
构造为
大根堆

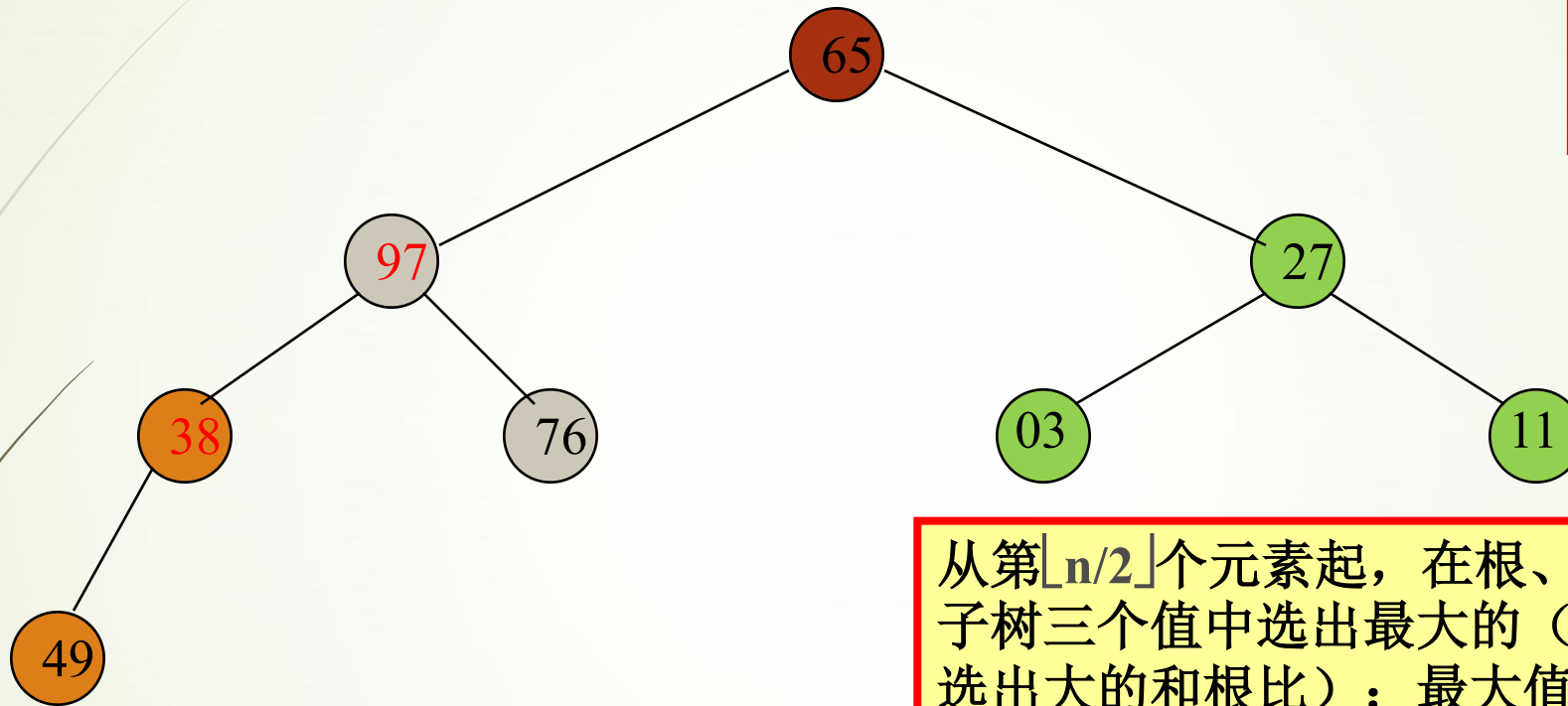


从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。

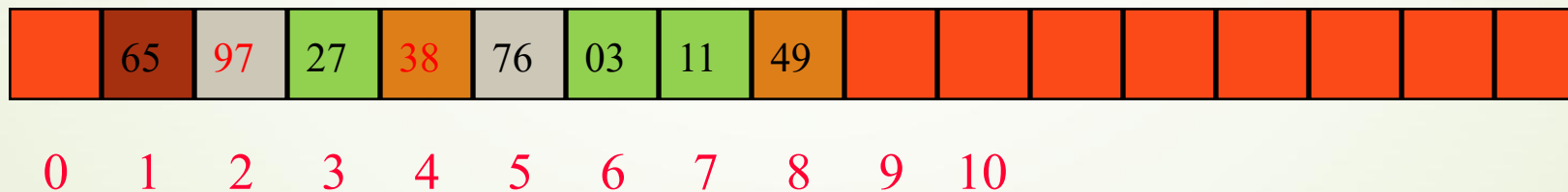


A Heap Is Efficiently Represented As An Array

无序数据
构造为
大根堆

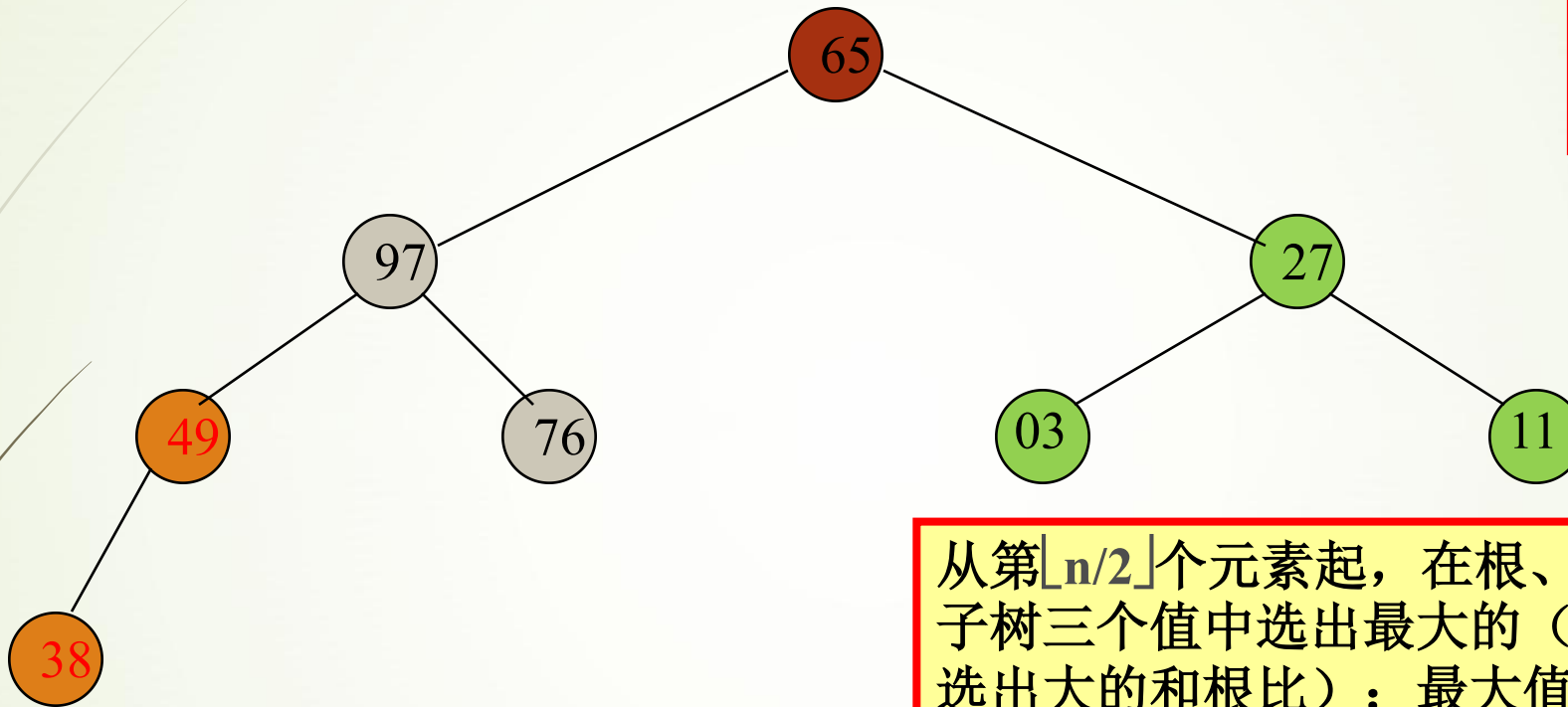


从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。

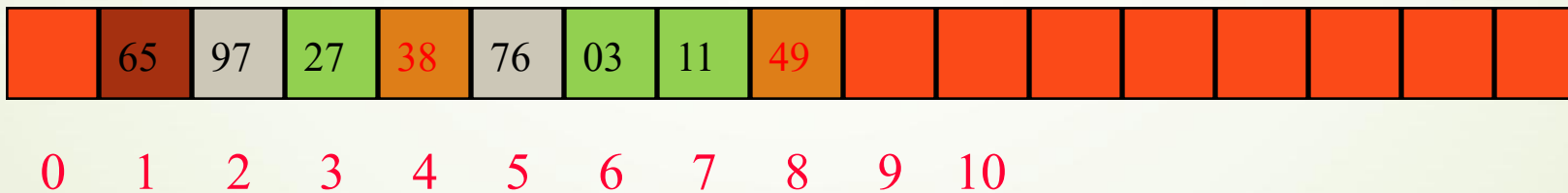


A Heap Is Efficiently Represented As An Array

无序数据
构造为
大根堆

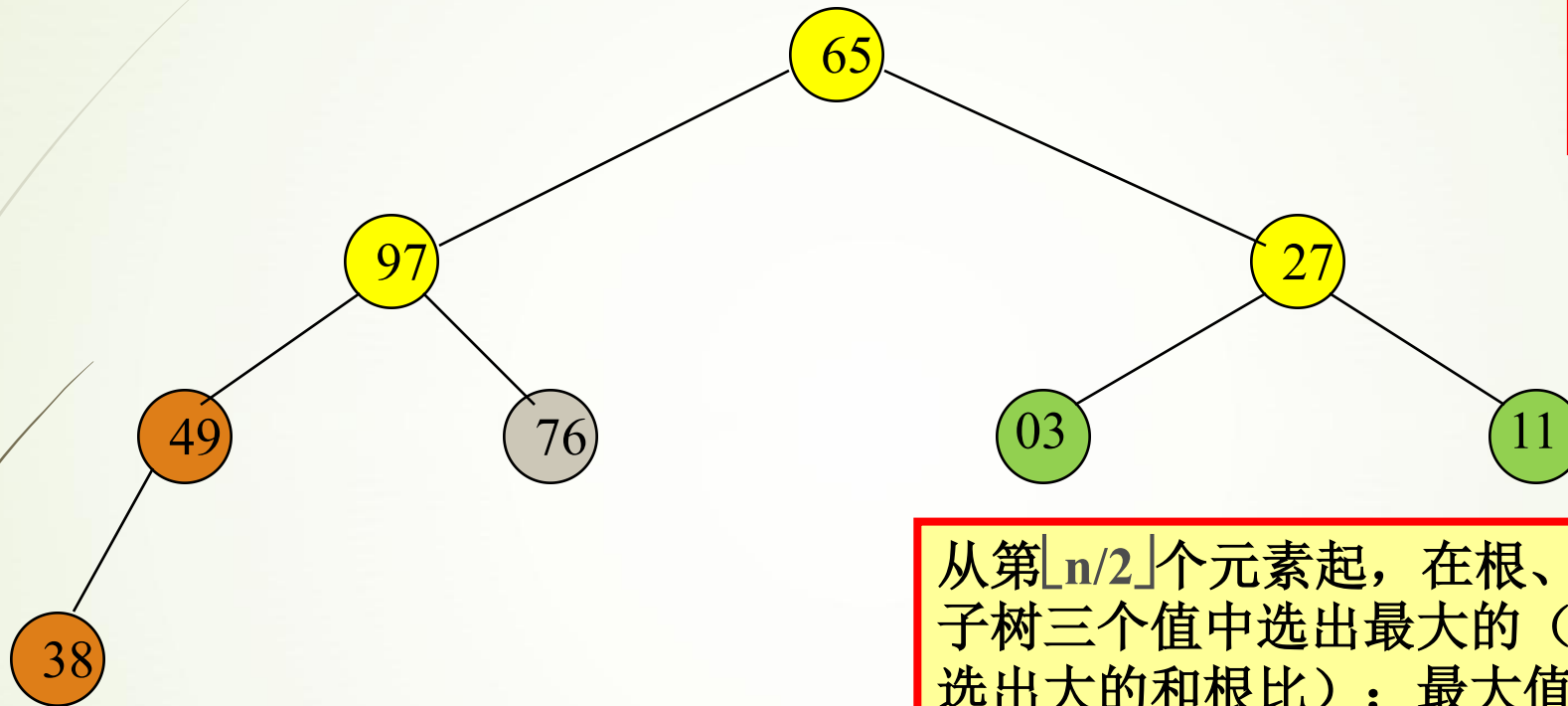


从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。

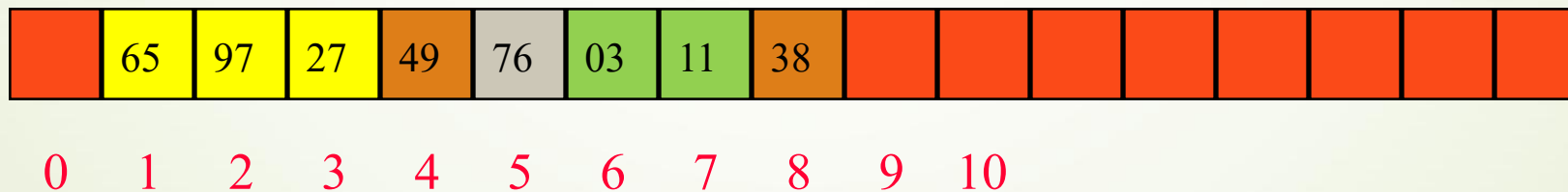


A Heap Is Efficiently Represented As An Array

无序数据
构造为
大根堆

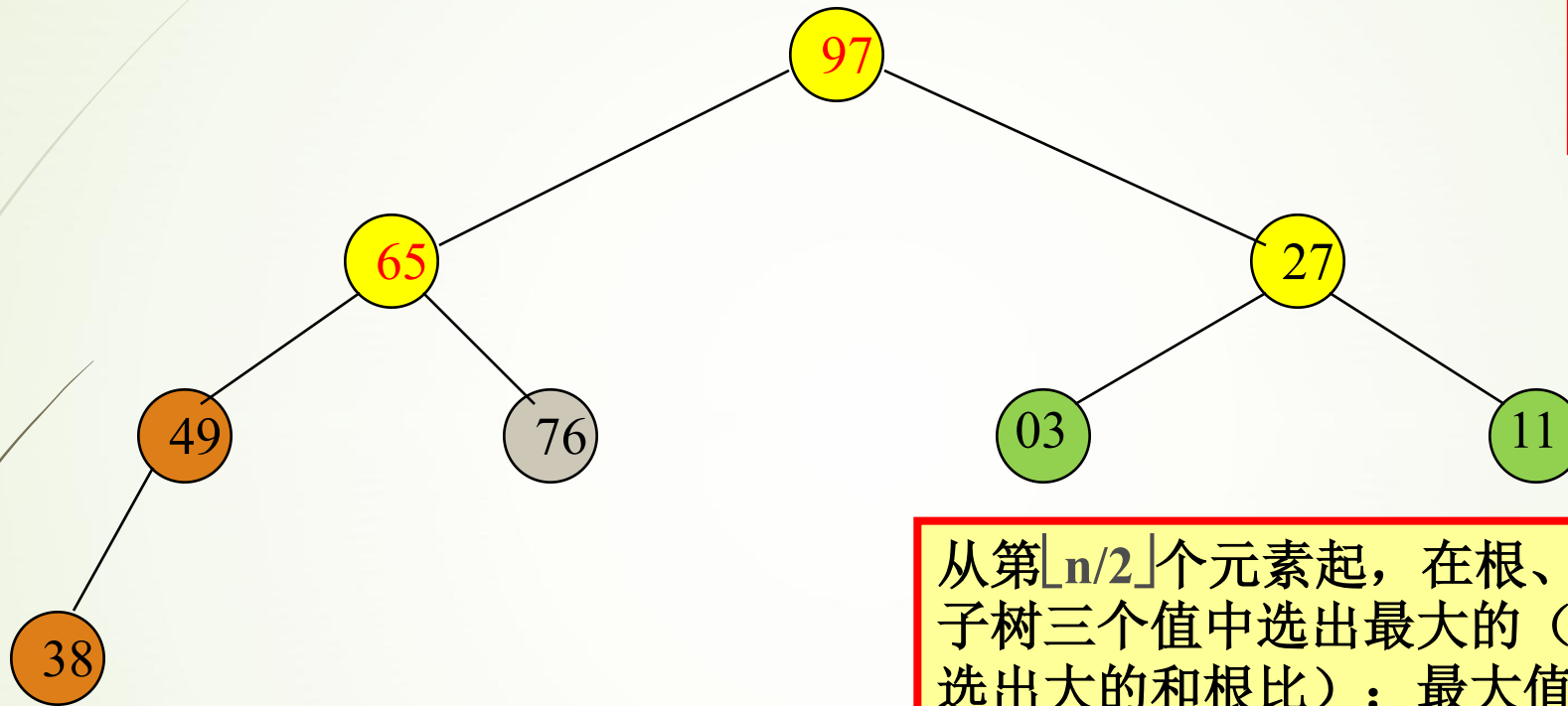


从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。

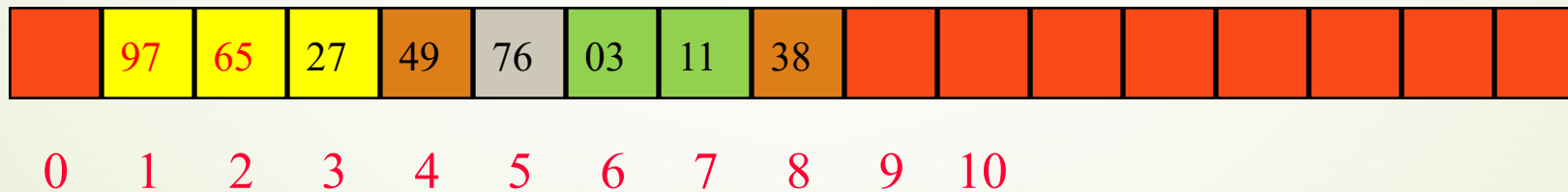


A Heap Is Efficiently Represented As An Array

无序数据
构造为
大根堆

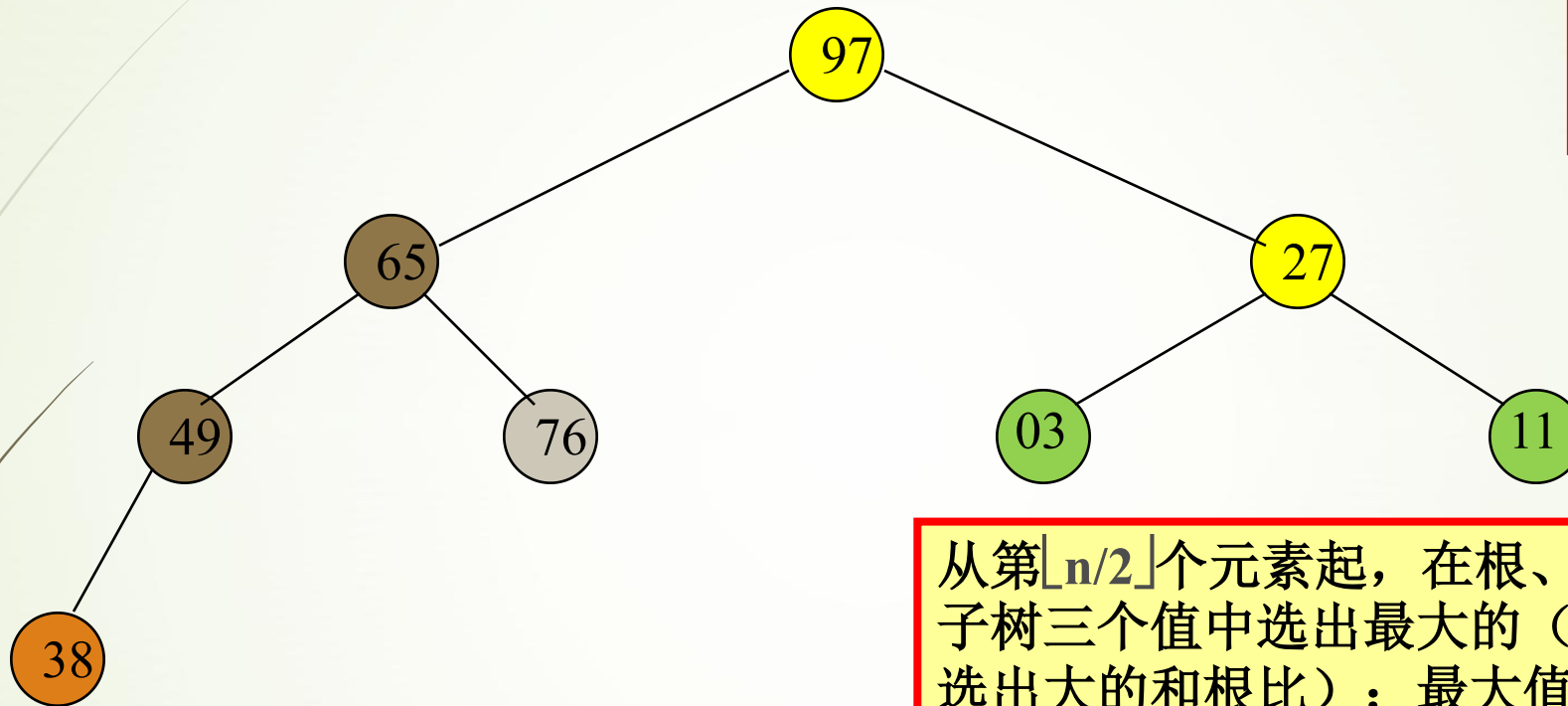


从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。

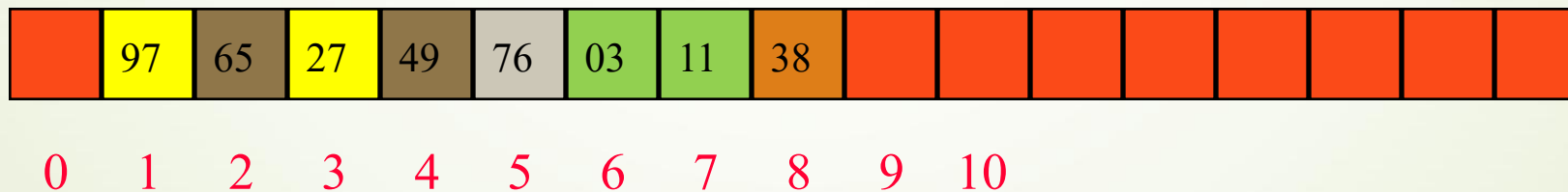


A Heap Is Efficiently Represented As An Array

无序数据
构造为
大根堆

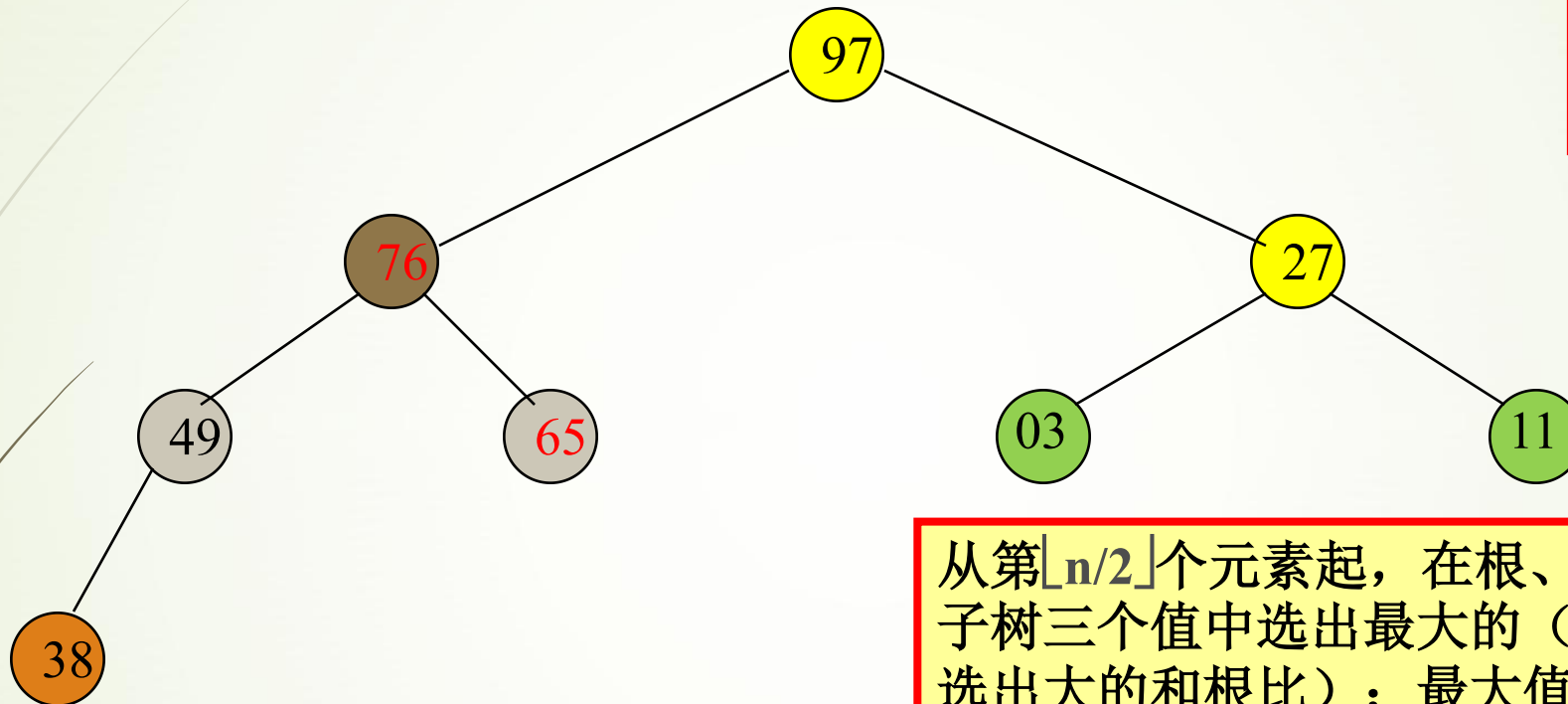


从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。

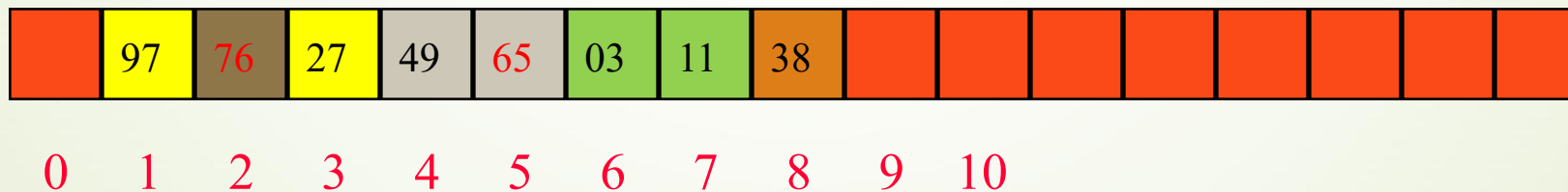


A Heap Is Efficiently Represented As An Array

无序数据
构造为
大根堆

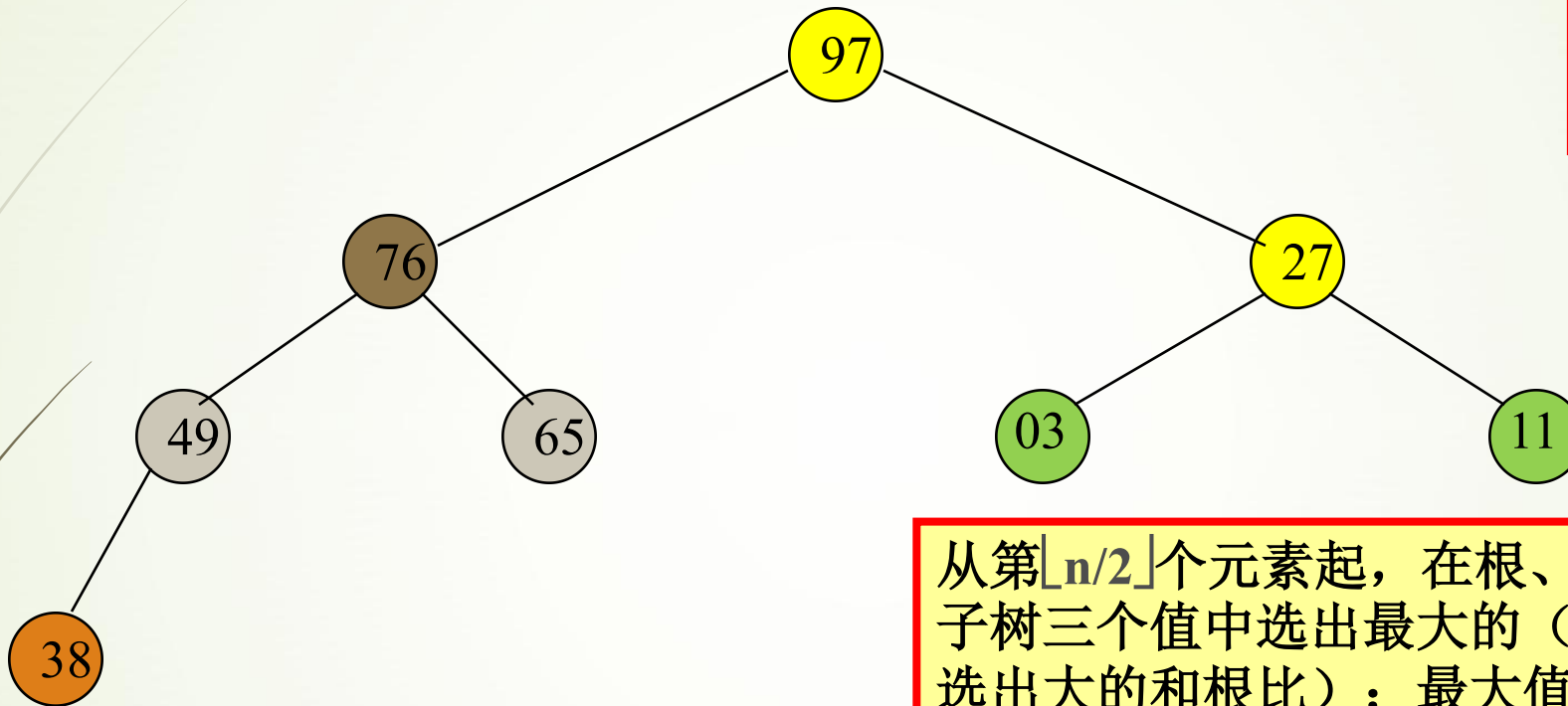


从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。

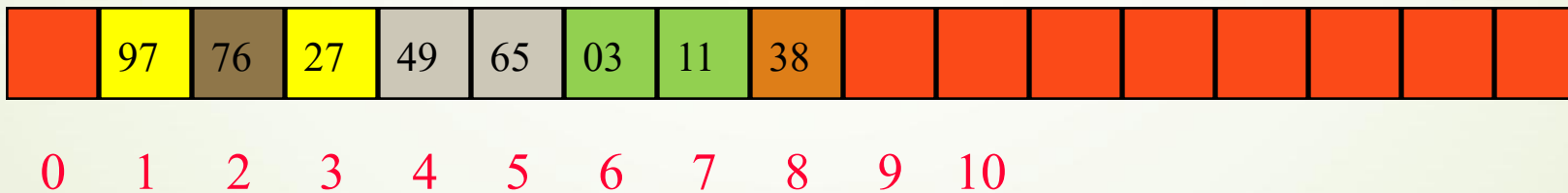


A Heap Is Efficiently Represented As An Array

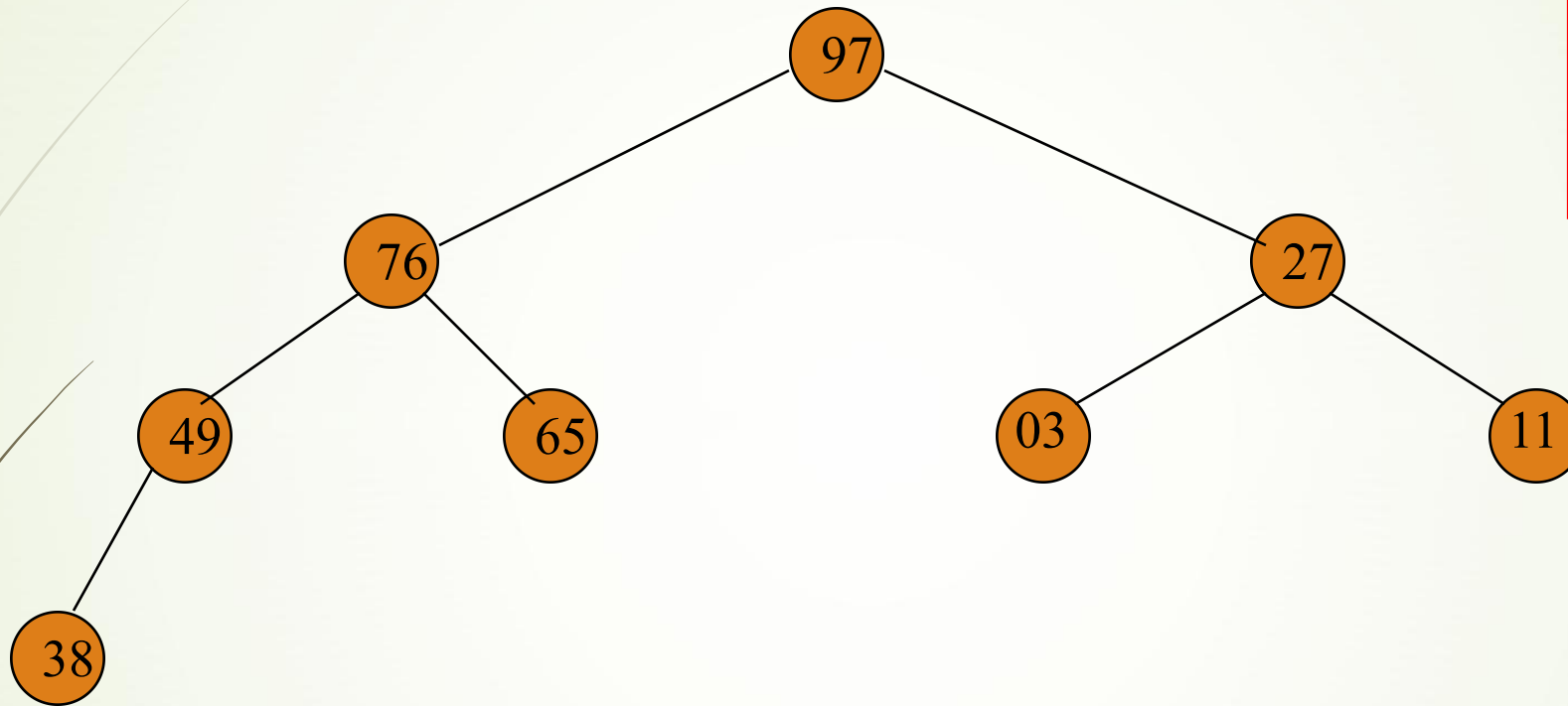
无序数据
构造为
大根堆



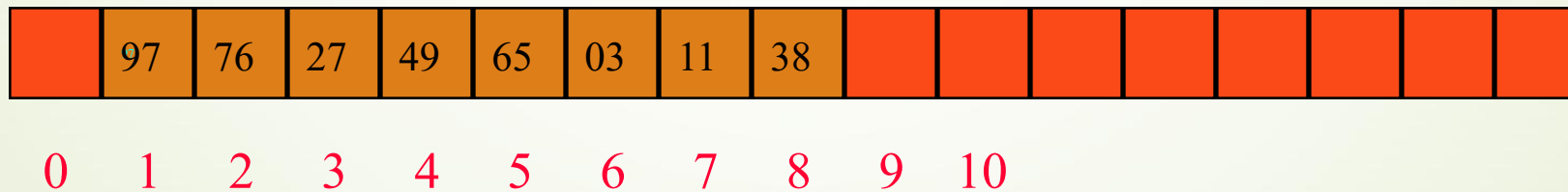
从第 $\lfloor n/2 \rfloor$ 个元素起，在根、左子树、右子树三个值中选出最大的（先左右比，选出大的和根比）：最大值如果是根则保留，否则根和最大值交换。



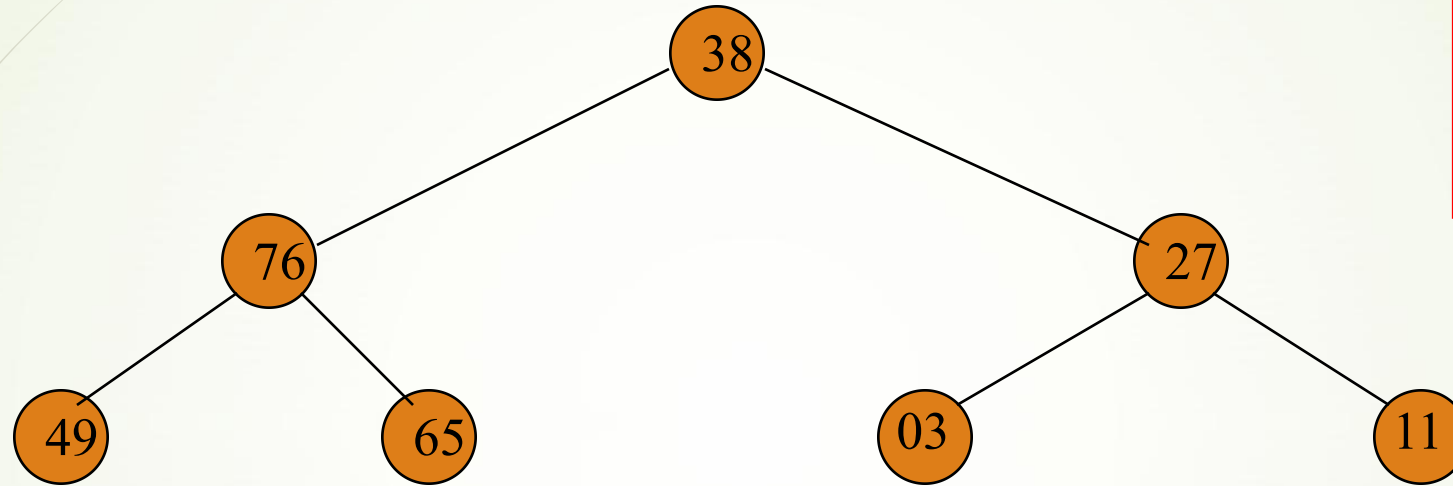
A Heap Is Efficiently Represented As An Array



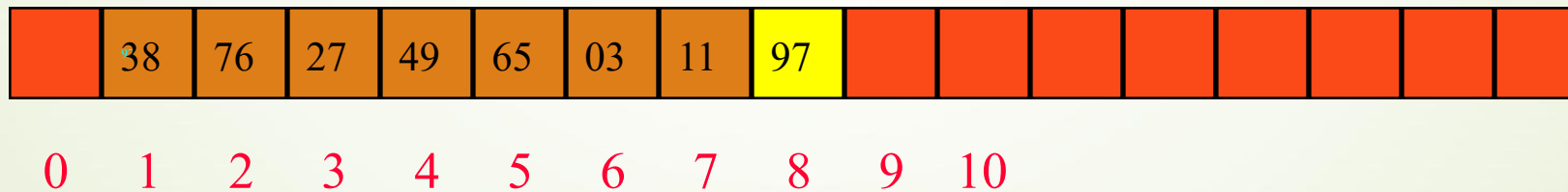
无序数据
构造为
大根堆
结束



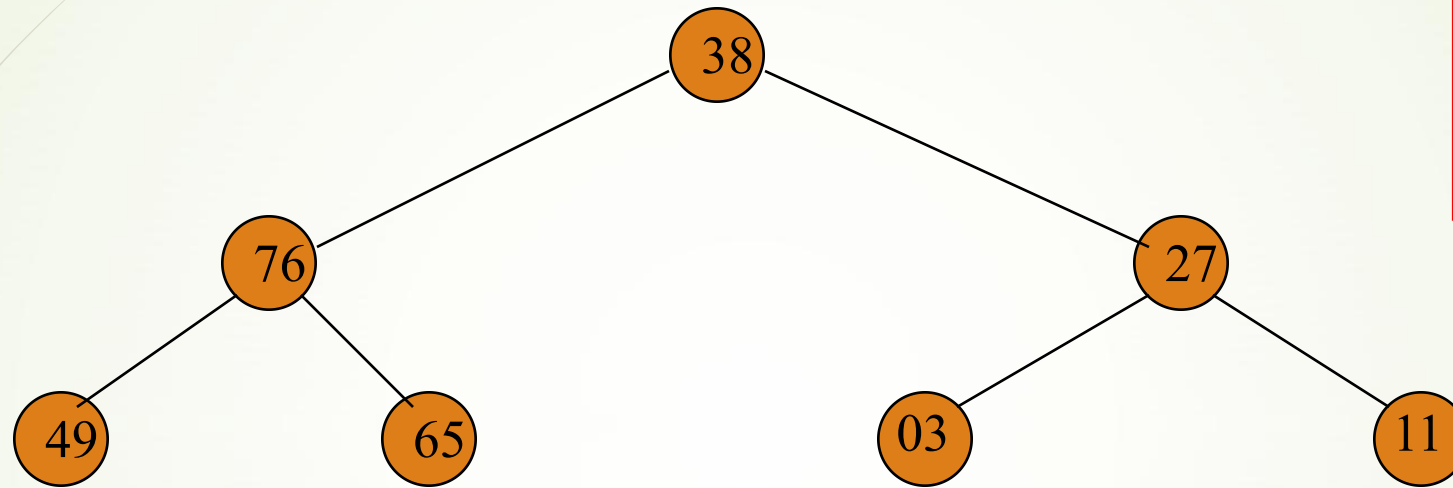
A Heap Is Efficiently Represented As An Array



大根输出后
重新将树调
整为
大根堆

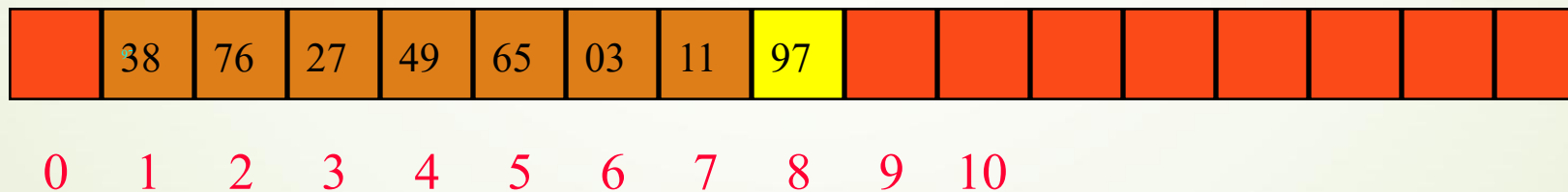


A Heap Is Efficiently Represented As An Array

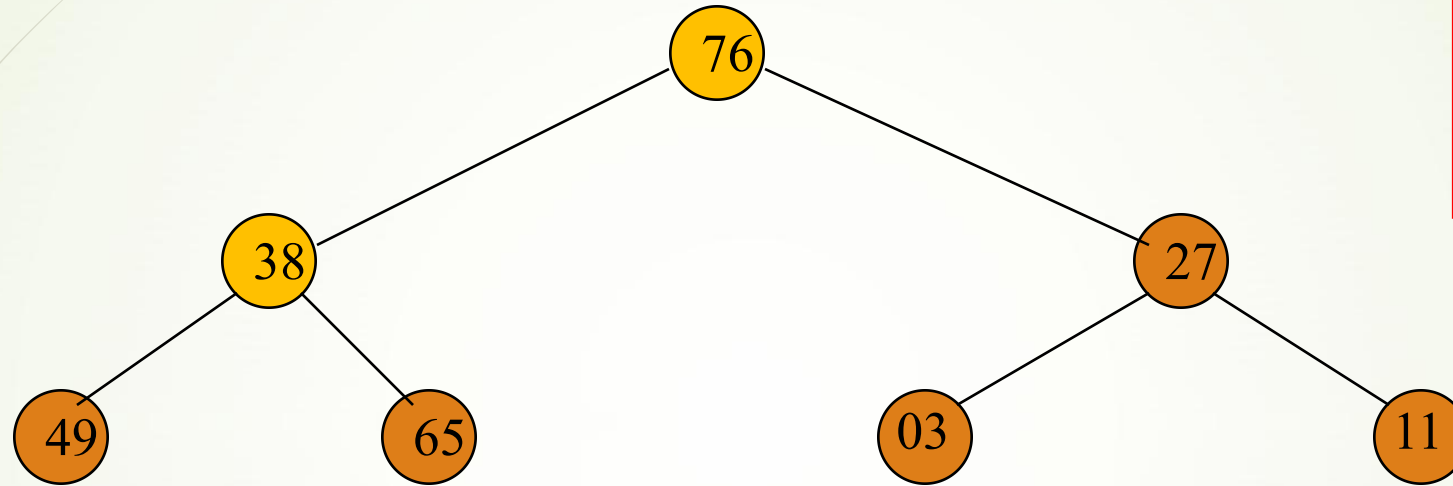


大根输出后
重新将树调整
为大根堆

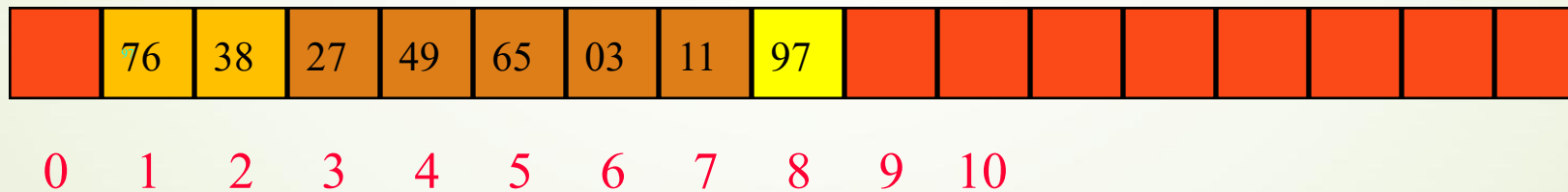
调整方法：
从1个元素起，和其左右子树比较，
和大值的子树结点交换



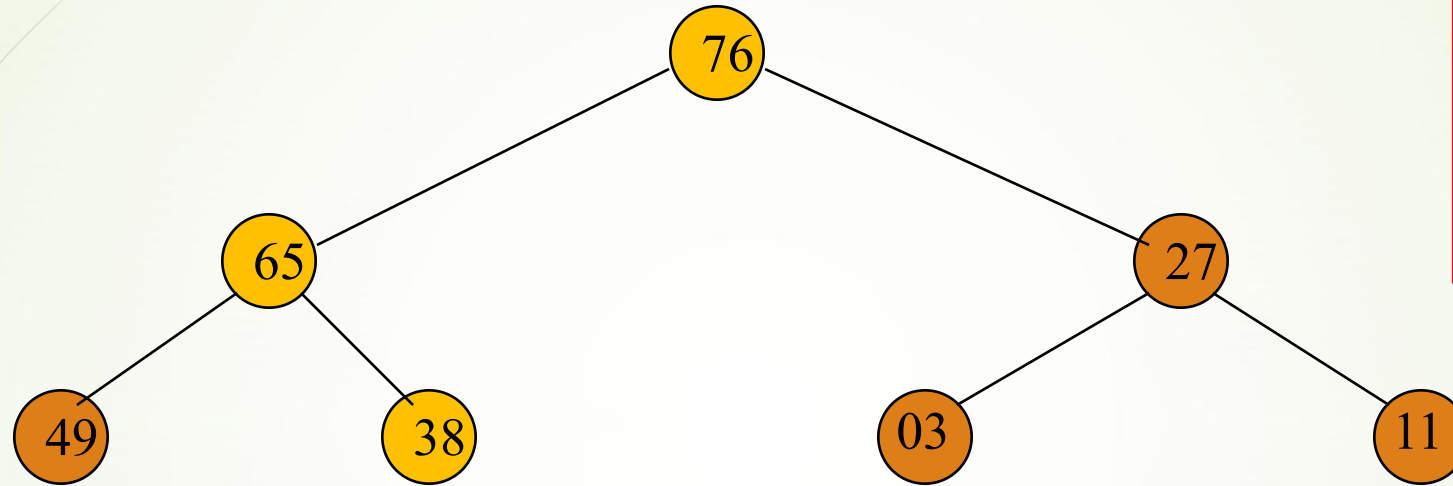
A Heap Is Efficiently Represented As An Array



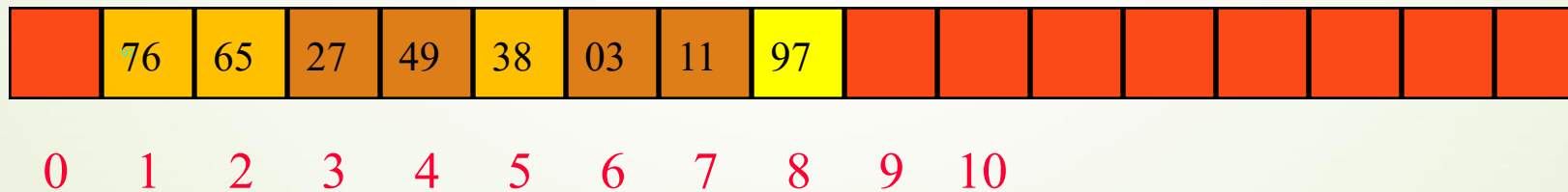
大根输出后
重新将树调
整为
大根堆



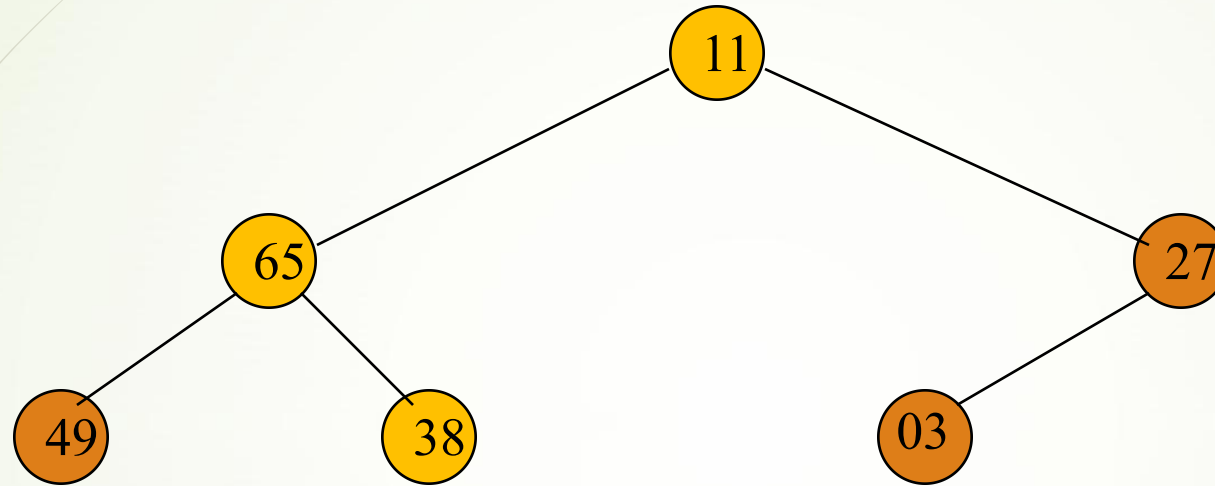
A Heap Is Efficiently Represented As An Array



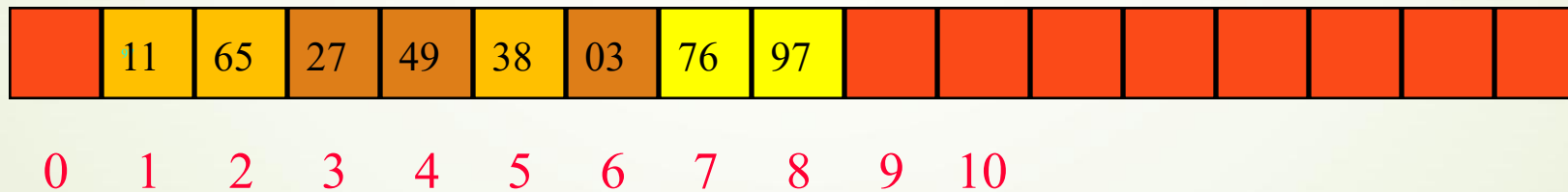
大根输出后
重新将树调
整为
大根堆
结束



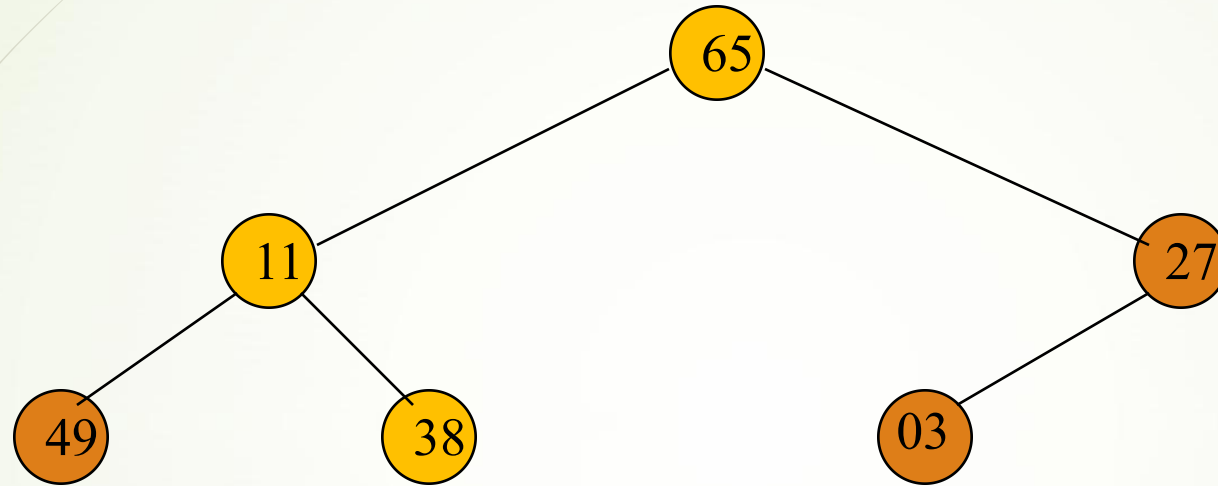
A Heap Is Efficiently Represented As An Array



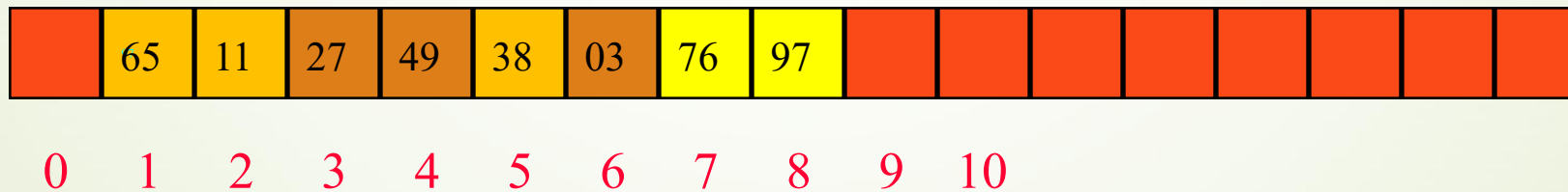
大根输出后
重新将树调
整为
大根堆



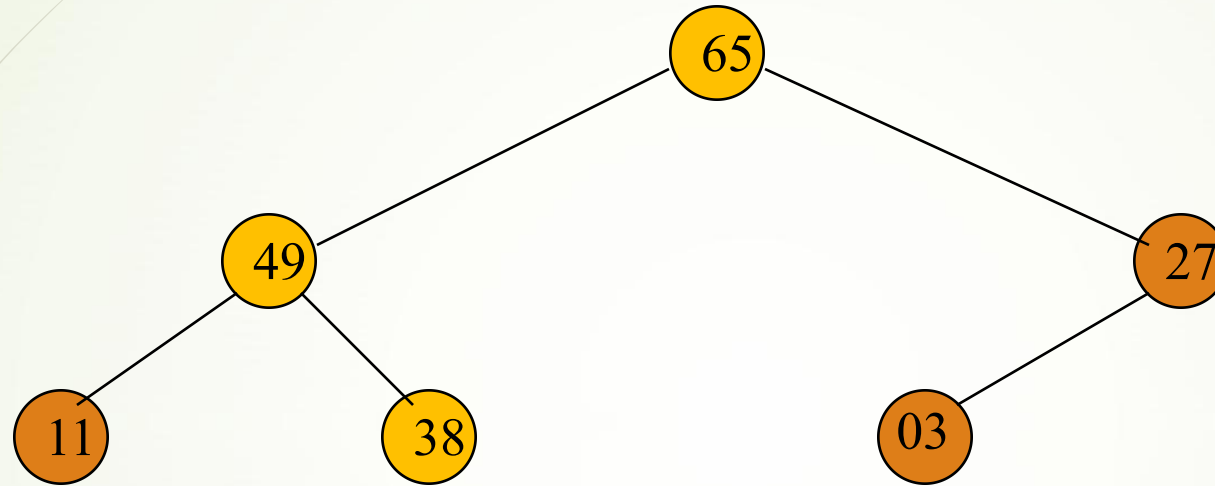
A Heap Is Efficiently Represented As An Array



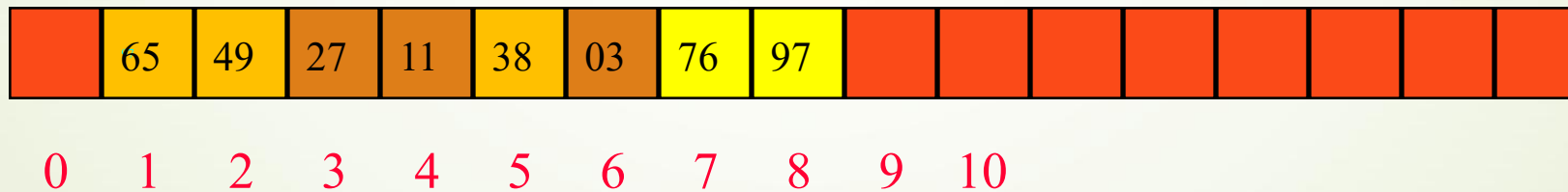
大根输出后
重新将树调
整为
大根堆



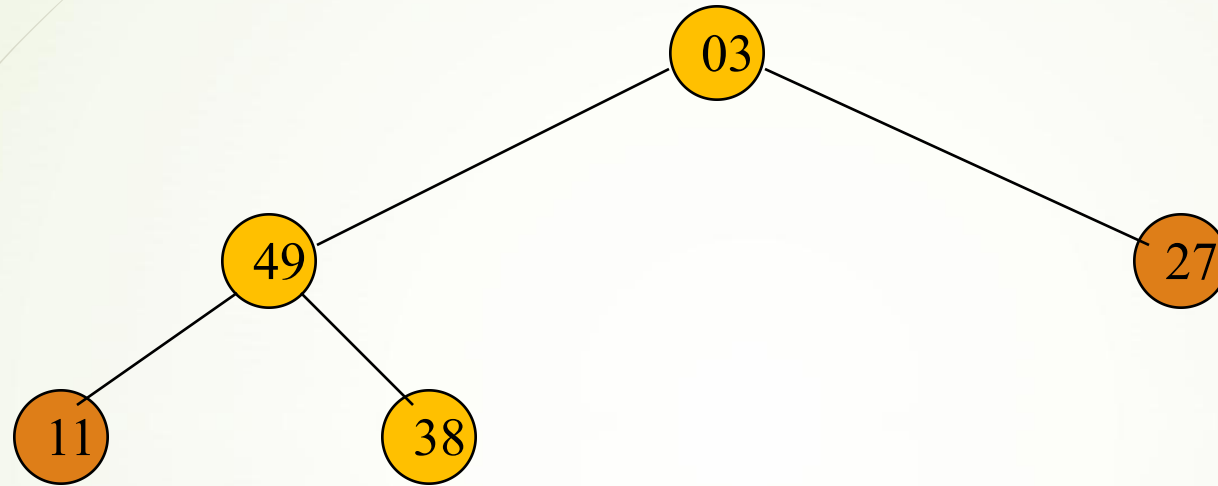
A Heap Is Efficiently Represented As An Array



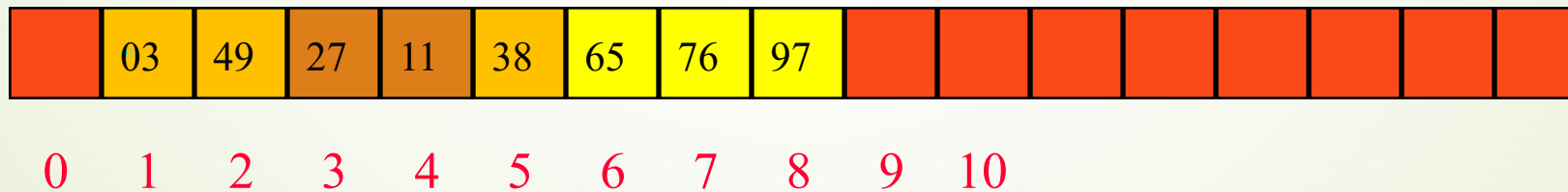
大根输出后
重新将树调
整为
大根堆
结束



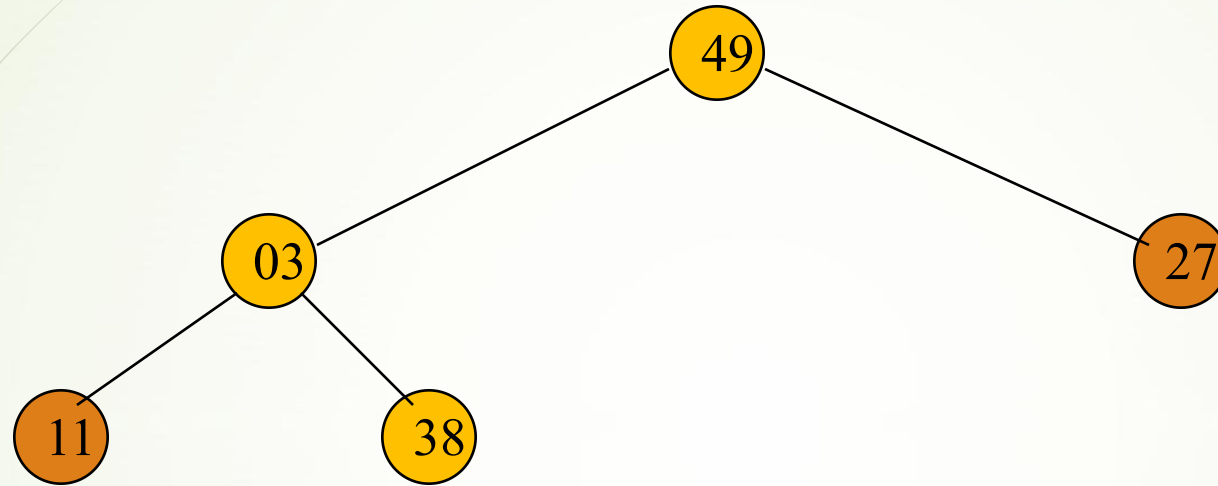
A Heap Is Efficiently Represented As An Array



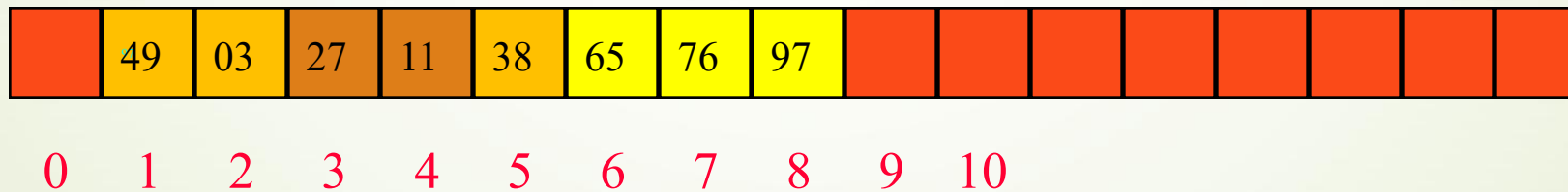
大根输出后
重新将树调
整为
大根堆



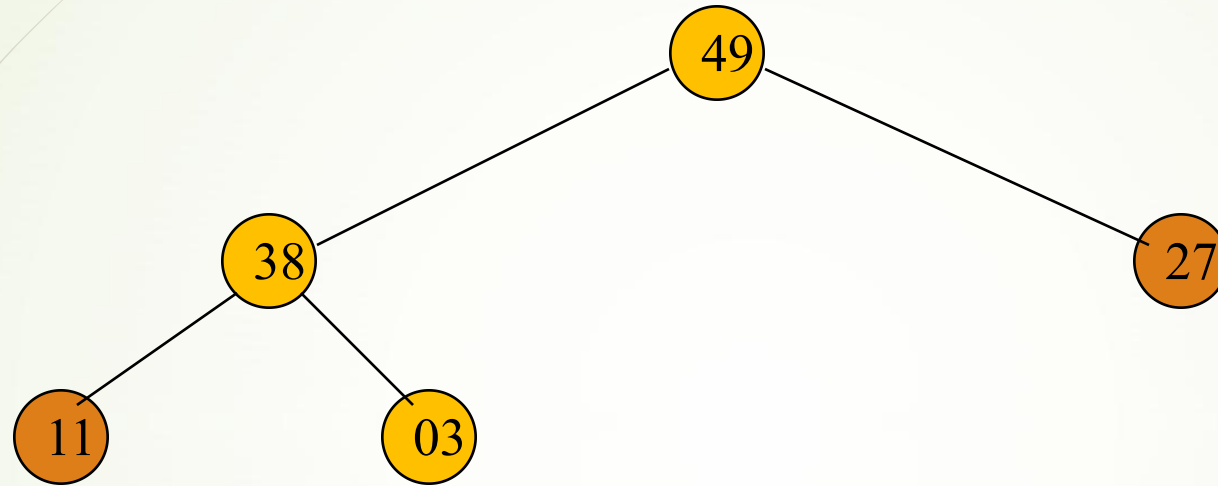
A Heap Is Efficiently Represented As An Array



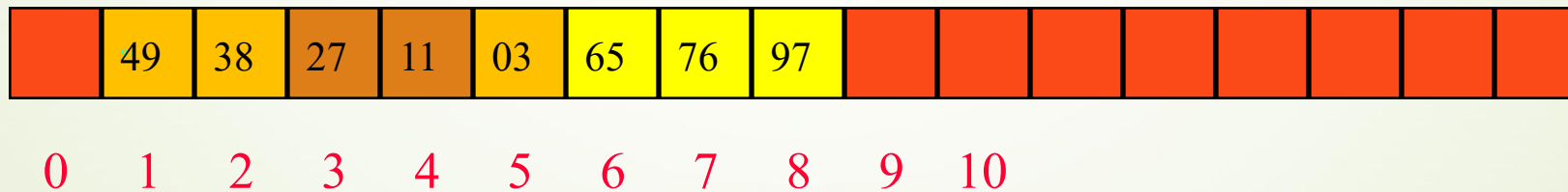
大根输出后
重新将树调
整为
大根堆



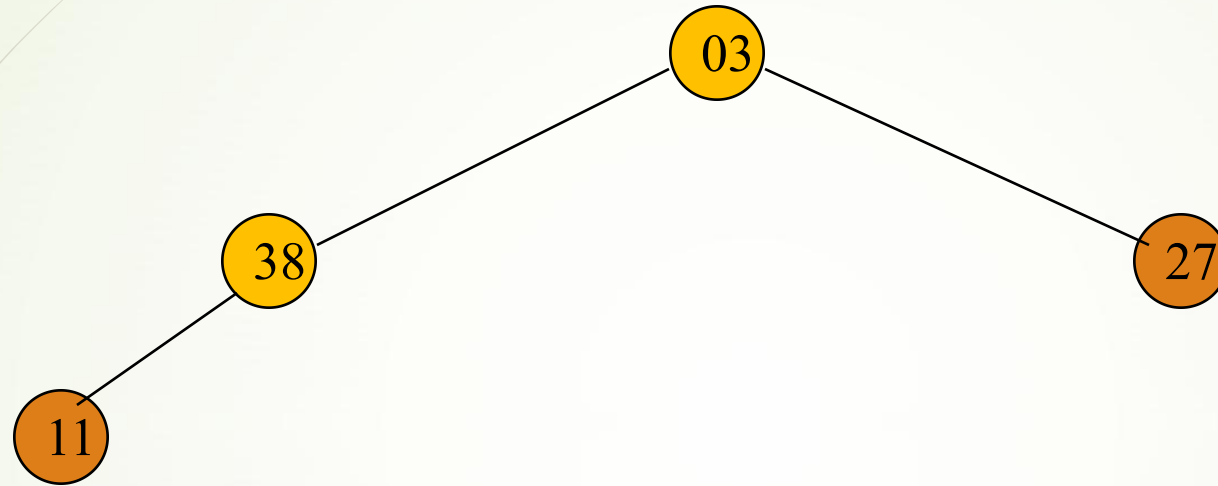
A Heap Is Efficiently Represented As An Array



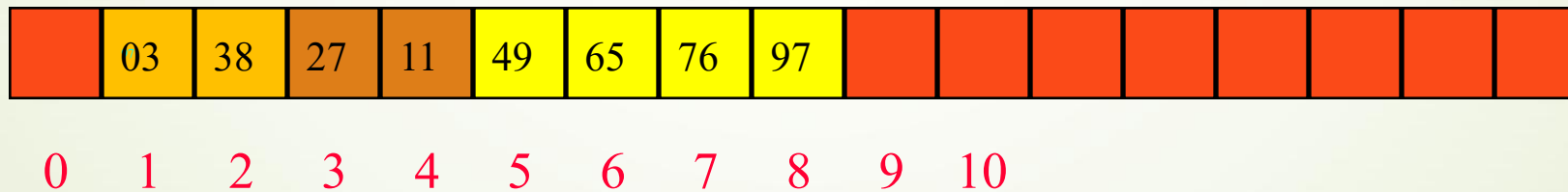
大根输出后
重新将树调
整为
大根堆
结束



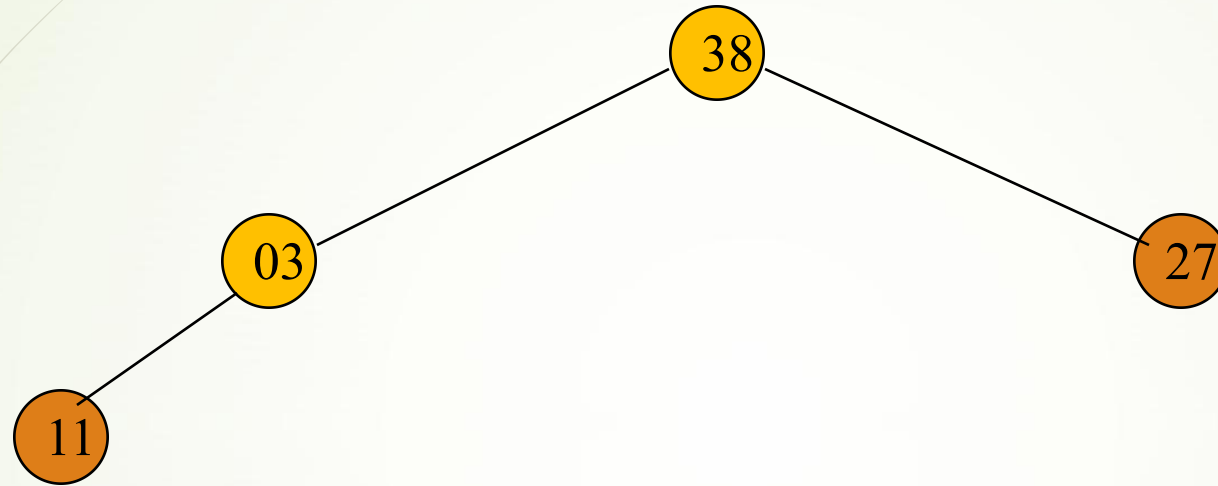
A Heap Is Efficiently Represented As An Array



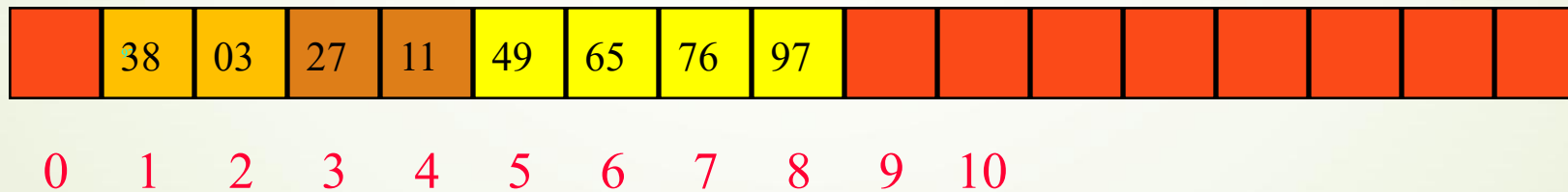
大根输出后
重新将树调
整为
大根堆



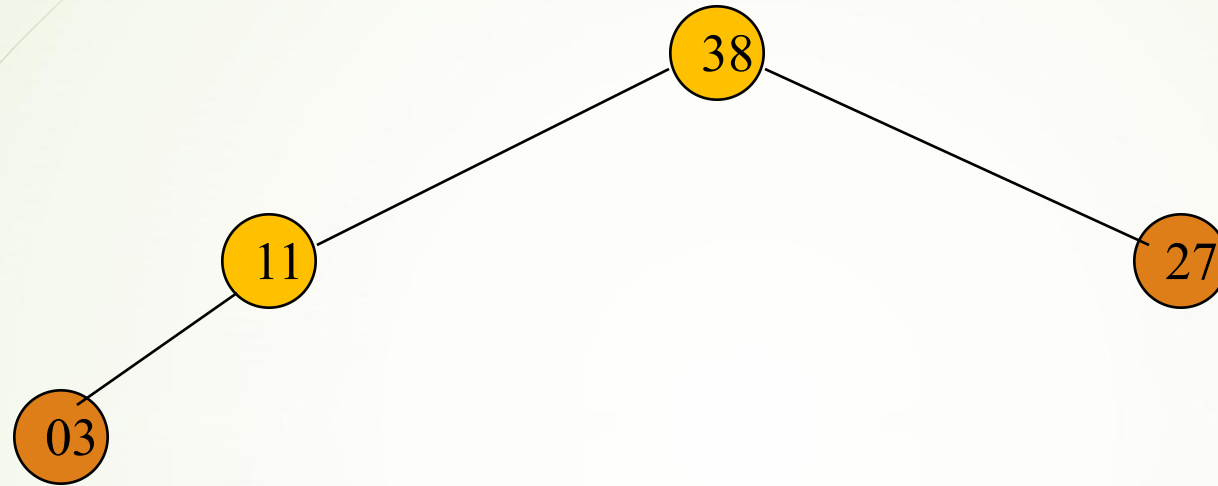
A Heap Is Efficiently Represented As An Array



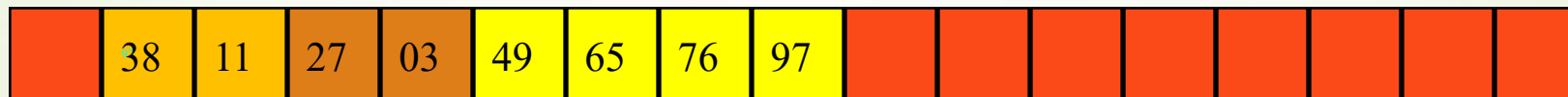
大根输出后
重新将树调
整为
大根堆



A Heap Is Efficiently Represented As An Array

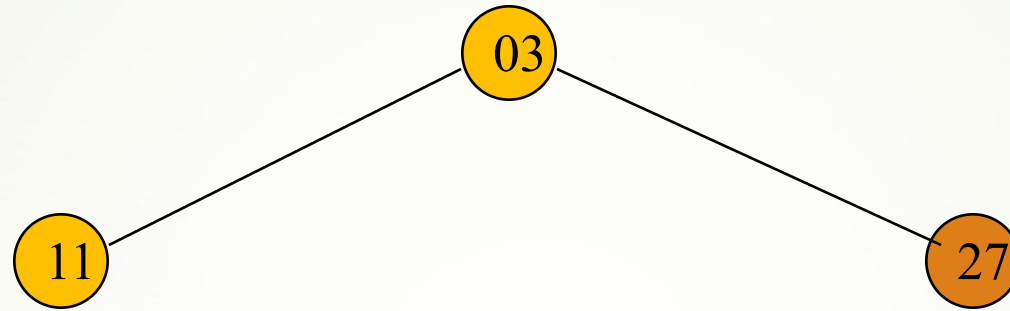


大根输出后
重新将树调
整为
大根堆
结束

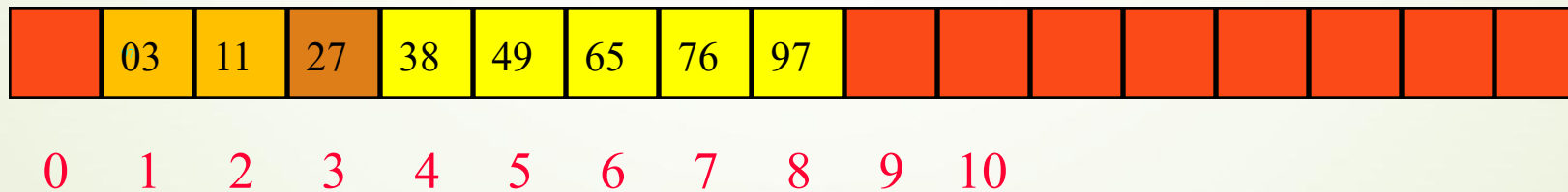


0 1 2 3 4 5 6 7 8 9 10

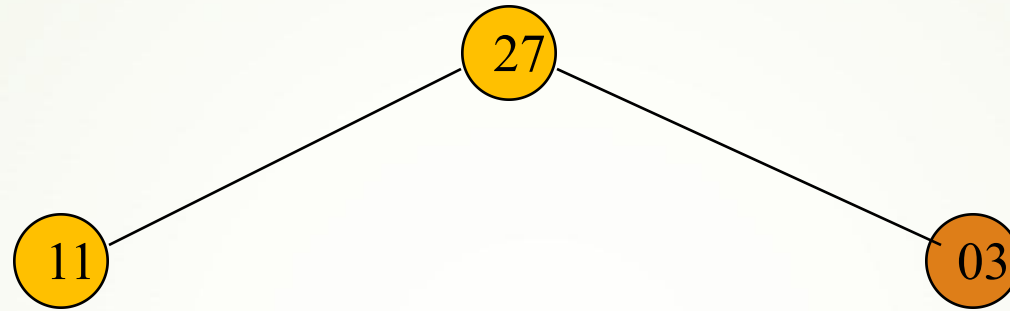
A Heap Is Efficiently Represented As An Array



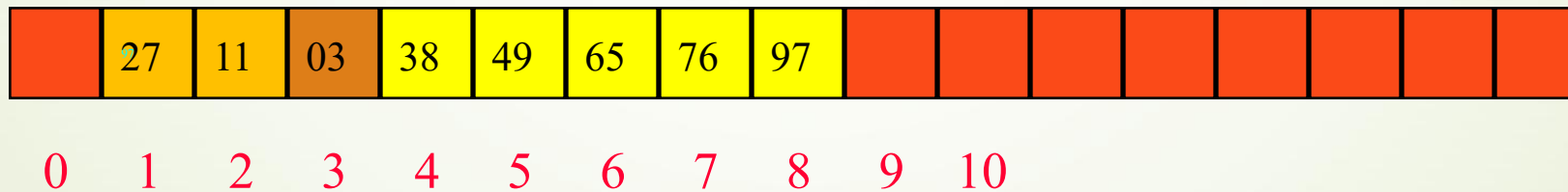
大根输出后
重新将树调
整为
大根堆



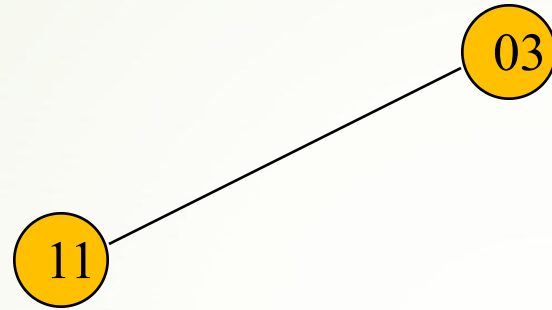
A Heap Is Efficiently Represented As An Array



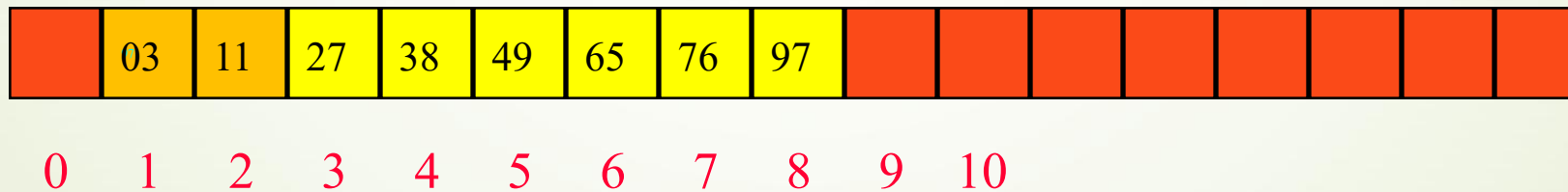
大根输出后
重新将树调
整为
大根堆
结束



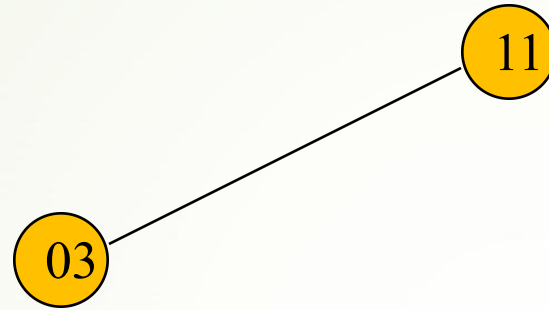
A Heap Is Efficiently Represented As An Array



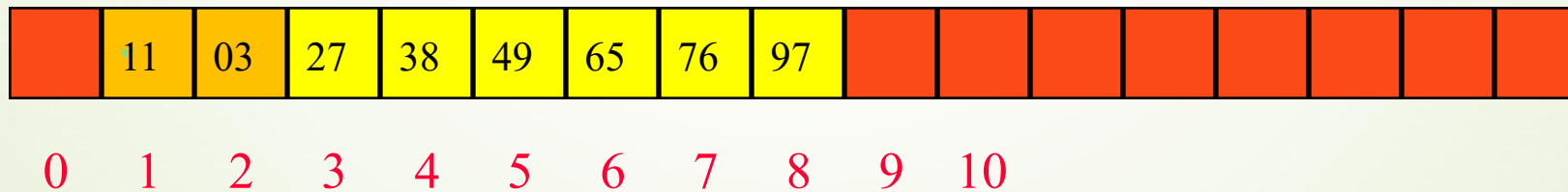
大根输出后
重新将树调
整为
大根堆



A Heap Is Efficiently Represented As An Array



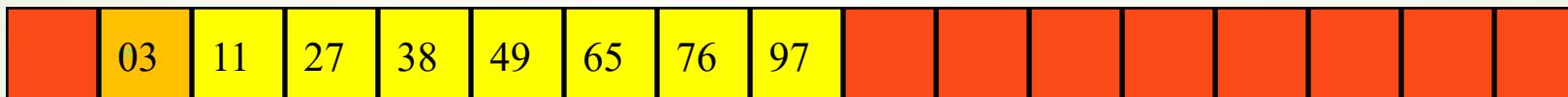
大根输出后
重新将树调
整为
大根堆
结束



A Heap Is Efficiently Represented As An Array

03

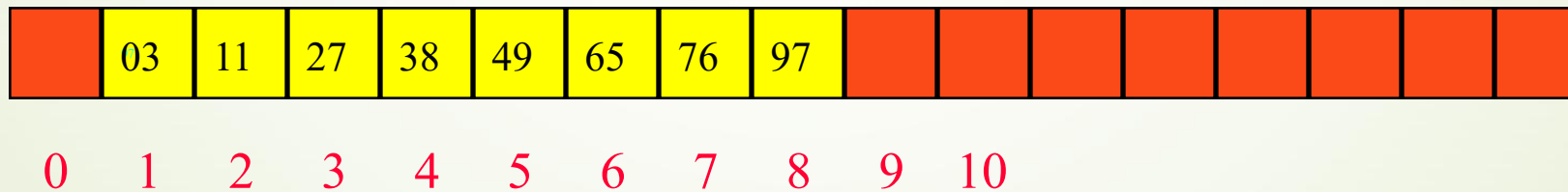
大根输出后
重新将树调
整为
大根堆
结束



0 1 2 3 4 5 6 7 8 9 10

A Heap Is Efficiently Represented As An Array

大根输出后
树空，
排序结束

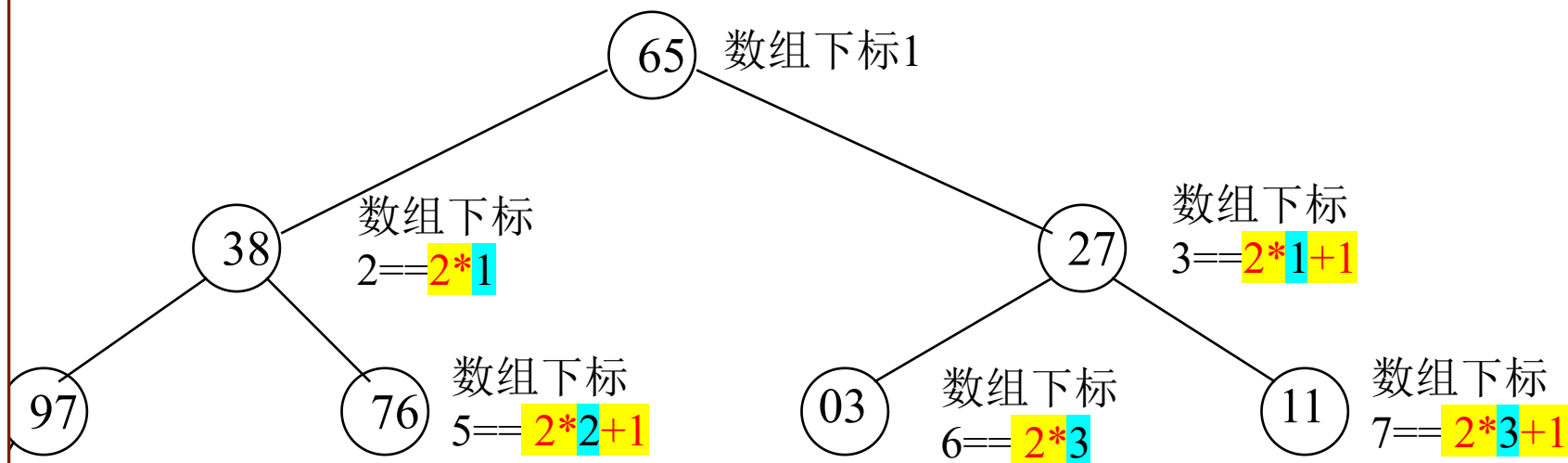


为什么要学习数据结构

❖ 考察一下n个整数的排序问题

完成n个数的排序，依然需要n-1趟，但是每趟要进行的比较次数要比选择排序少了。
总的比较次数比选择排序少了很多

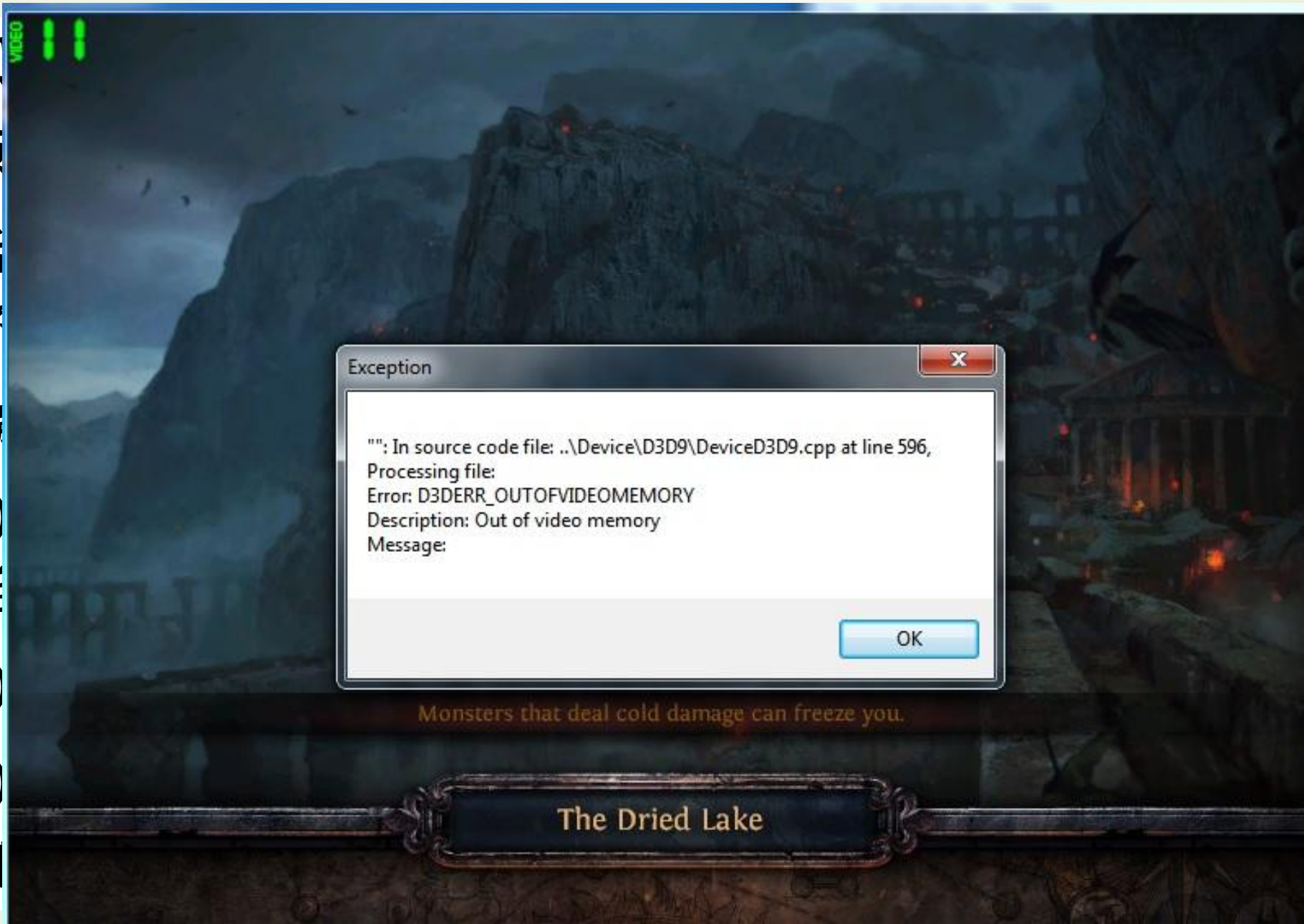
一种数据结构来思考一下，把数组表示的数一棵二叉树



数据元素值		65	38	27	97	76	3	11	49
数组下标	0	1	2	3	4	5	6	7	8

内存泄露检查

- ❖ 问
- ❖ 原
- ❖ 后
- ❖ 又不
- ❖ 编
- ❖ 1)
- ❖ 点后
- ❖ 2)
- ❖ 3)
- ❖ 一样



内存

个结

针

一个故事：内存泄露检查（取自《算法技术手册 p11-p17》）

❖ Graham为了解决公司内程序员编写的程序有内存泄露的问题，他改写了C/C++的几个库函数malloc、free和exit，改写的目的是希望知道一个程序是否有内存泄露：

❖ 改写malloc函数：

在完成原始malloc功能的同时记录内存分配的程序位置

❖ 改写free函数

在完成原始free功能时，找到相应分配记录

记录这个free调用的程序位置，表示对应的malloc释放了

❖ 改写exit函数

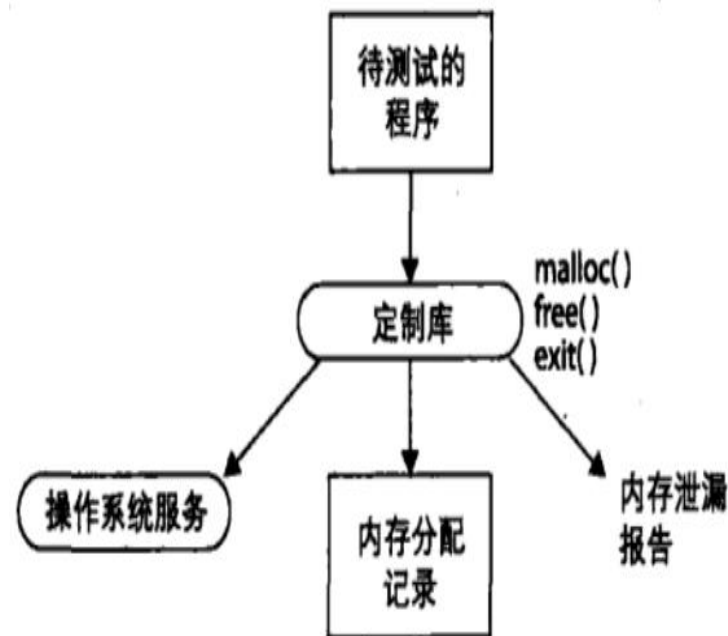
在程序退出时，输出未释放的malloc记录供debug。

❖ Graham花了几个小时就实现了上述代码，并调试通过可以使用了。

❖ 但是，使用了Graham的三个函数的程序，在运行时变得非常慢：

❖ 你可以先启动程序，然后去喝一杯咖啡，然后回来，程序还在启动中。。。

❖ 于是Graham找了Gray，希望Gray能帮助他解决问题。



一个故事：内存泄露检查（取自《算法技术手册 p11-p17》）

如果需要，尽可能用实践检验

Gary和Graham的第一个测试程序（程序A）

例1-1：程序A代码

```
int main(int argc, char **argv) {  
    int i = 0;  
    for (i = 0; i < 1000000; i++) {  
        malloc(32);  
    }  
    exit (0);  
}
```

❖ 实践检验结果：

- ❖ 程序运行时间为数分钟，不能接受；
- ❖ 程序显示有32MB内存未释放，正确；

❖ 改写代码继续检验

如果需要，尽可能用实践检验

❖ 第二个测试程序：

例1-2：程序B代码

```
int main(int argc, char **argv) {  
    int i = 0;  
    for (i = 0; i < 1000000; i++) {  
        void *x = malloc(32);  
        free(x);  
    }  
    exit (0);  
}
```

❖ 实践检验结果：

- ❖ 程序运行时间为几秒钟，非常正常；
- ❖ 程序显示内存都释放了，正确；

❖ 改写代码继续检验

一个故事：内存泄露检查（取自《算法技术手册 p11-p17》）

如果需要，尽可能用实践检验

Gary和Graham的第一个测试程序（程序A）

例1-1：程序A代码 **慢**

```
int main(int argc, char **argv) {  
    int i = 0;  
    for (i = 0; i < 1000000; i++) {  
        malloc(32);  
    }  
    exit (0);  
}
```

例1-3：程序C代码

```
int main(int argc, char **argv) {  
    int i = 0;  
    void *addrs[1000000];  
    for (i = 0; i < 1000000; i++) {  
        addrs[i] = malloc(32);  
    }  
    for (i = 0; i < 1000000; i++) {  
        free(addrs[i]);  
    }  
    exit (0);  
}
```

慢

例1-2：程序B代码

```
int main(int argc, char **argv) {  
    int i = 0;  
    for (i = 0; i < 1000000; i++) {  
        void *x = malloc(32);  
        free(x);  
    }  
    exit (0);  
}
```

正常

❖ C程序实践检验结果：

❖ 就像程序A一样缓慢运行，时间为数分钟；

❖ 程序显示内存都释放了，正确；

❖ Gray想到了问题所在，我们能想到吗？

一个故事：内存泄露检查（取自《算法技术手册 p11-p17》）

如果需要，尽可能用实践检验

Gary和Graham的第一个测试程序（程序A）

例1-1：程序A代码 **慢**

```
int main(int argc, char **argv) {  
    int i = 0;  
    for (i = 0; i < 1000000; i++) {  
        malloc(32);  
    }  
    exit (0);  
}
```

例1-3：程序C代码

```
int main(int argc, char **argv) {  
    int i = 0;  
    void *addrs[1000000];  
    for (i = 0; i < 1000000; i++) {  
        addrs[i] = malloc(32);  
    }  
    for (i = 0; i < 1000000; i++) {  
        free(addrs[i]);  
    }  
    exit (0);  
}
```

慢

例1-2：程序B代码

```
int main(int argc, char **argv) {  
    int i = 0;  
    for (i = 0; i < 1000000; i++) {  
        void *x = malloc(32);  
        free(x);  
    }  
    exit (0);  
}
```

正常

❖ malloc之后，马上free，就正常；
❖ mallo之后，继续malloc，再集中free或不free就很慢。

❖ 问题出在malloc和free上，而且是未释放的内存记录存在时会慢。

一个故事：内存泄露检查（取自《算法技术手册 p11-p17》）

如果需要，尽可能用实践检验

❖ Gray后Graham讨论了malloc的实现：

❖ 采用二叉树存储内存分配记录

❖ 每个结点的key是内存地址

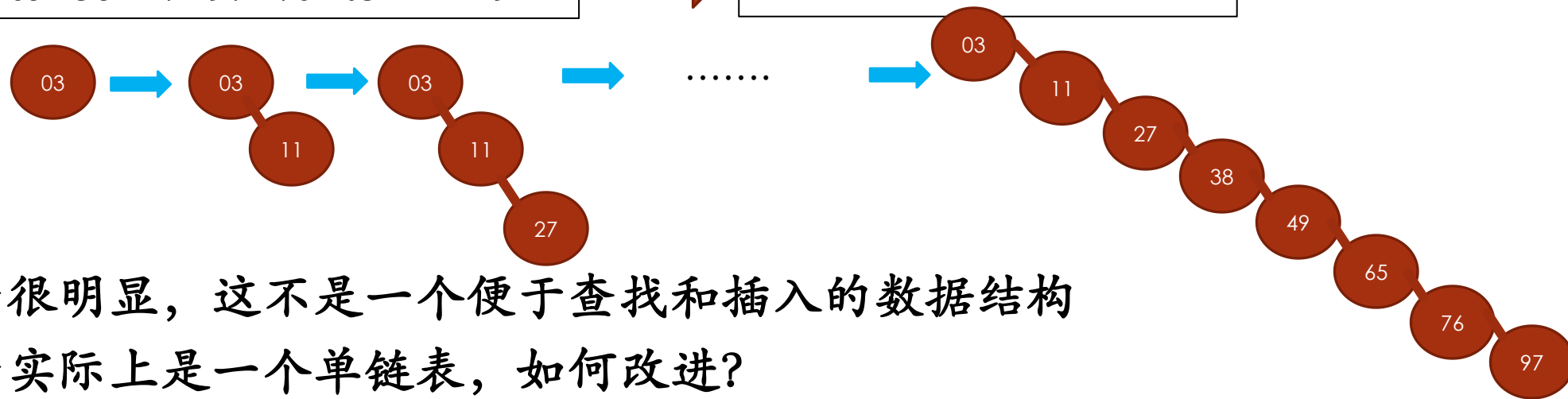
❖ 回忆一下前面例子中的二叉排序树

❖ 可是慢的程序在进行内存分配时，~~是~~一旦申请后释放

❖ 这样每次申请到的内存地址key大概率是一个递增序列

❖ 如果直接插入到树中，就会一直向右子树上插入

65 38 27 97 76 03 11 49 → 03 11 27 38 49 65 76 97



例1-1：程序A代码

慢

```
int main(int argc, char **argv) {
    int i = 0;
    for (i = 0; i < 1000000; i++) {
        malloc(32);
    }
    exit (0);
}
```

例1-3：程序C代码

慢

```
int main(int argc, char **argv) {
    int i = 0;
    void *addrs[1000000];
    for (i = 0; i < 1000000; i++) {
        addrs[i] = malloc(32);
    }
    for (i = 0; i < 1000000; i++) {
        free(addrs[i]);
    }
    exit (0);
}
```

❖ 很明显，这不是一个便于查找和插入的数据结构

❖ 实际上是一个单链表，如何改进？

一个故事：内存泄露检查（取自《算法技术手册 p11-p17》）

如果需要，尽可能用实践检验

❖ Gray后Graham讨论了malloc内存泄漏问题

❖ 采用二叉树存储内存

❖ 每个结点的key是内存地址

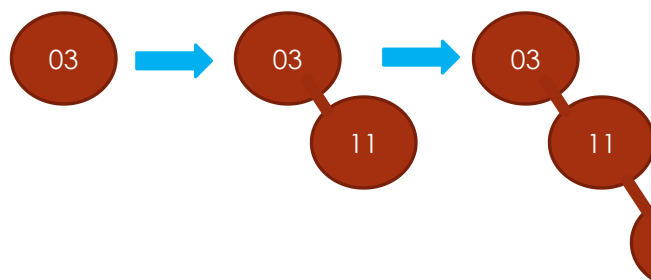
❖ 回忆一下前面例子中的

❖ 可是慢的程序在进行

❖ 这样每次申请到的内存

❖ 如果直接插入到树中

65 38 27 97 76 03 11 49



❖ 很明显，这不是一个便

❖ 实际上是一个单链表，如何改进？

例 1.1 程序A代码

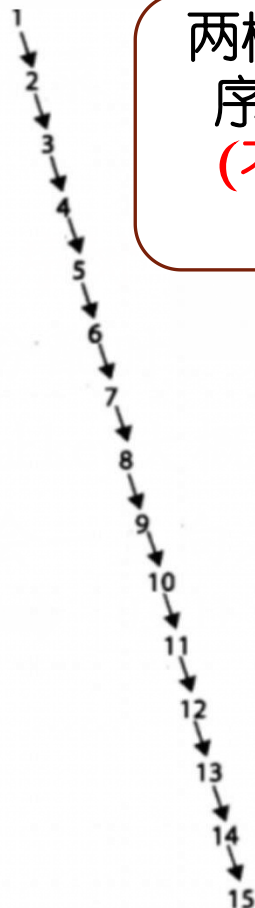
快

例 1.2 程序C代码

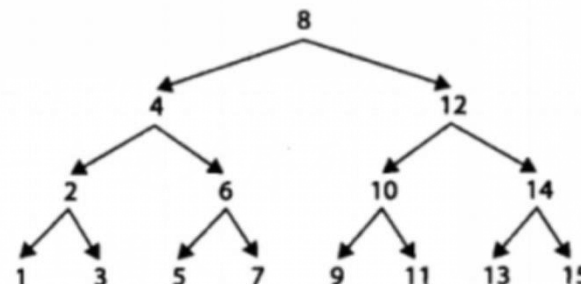
慢

```
(int argc, char **argv) {  
    = 0;  
    addrs[1000000];  
    = 0; i < 1000000; i++) {  
        drs[i] = malloc(32);  
  
    = 0; i < 1000000; i++) {  
        ee(addrs[i]);  
    });
```

两棵数据元素内容相同的二叉排序树，右边的更平衡，更便于（不是便于编程，而是程序执行更快）查找、插入和删除



(a)



(b)

76

97

一个故事：内存泄露检查（取自《算法技术手册 p11-p17》）

如果需要，尽可能用实践检验

❖ Gray后Graham讨论了malloc的实现：

- ❖ 采用二叉树存储内存分配记录
- ❖ 每个结点的key是内存地址

❖ 回忆一下前面例子中的二叉排序树

- ❖ 不断申请内存，集中释放时，地址大致是一个递增序列
- ❖ 如果直接插入到树中，就会一直向右子树上插入

❖ 对插入的二叉排序树进行平衡化，让每个子树的左右子树大致一致

- ❖ 从二叉排序树结构改进为平衡二叉树结构（AVL树）
- ❖ 从AVL树，改进为2-3树（B-tree的一种）
- ❖ 从2-3树，改进为2-3-4树（B-tree的一种）
- ❖ 从2-3-4树，改进为红黑树

❖ 上面是树结构（及对应的增加、删除、查找算法）改进的过程，Gray和Graham意识到数据结构的缺陷后直接改进为红黑树解决了问题。

例1-1：程序A代码

慢

```
int main(int argc, char **argv) {
    int i = 0;
    for (i = 0; i < 1000000; i++) {
        malloc(32);
    }
    exit (0);
}
```

例1-3：程序C代码

慢

```
int main(int argc, char **argv) {
    int i = 0;
    void *addrs[1000000];
    for (i = 0; i < 1000000; i++) {
        addrs[i] = malloc(32);
    }
    for (i = 0; i < 1000000; i++) {
        free(addrs[i]);
    }
    exit (0);
}
```

为什么要学习数据结构

- ❖ 编程基础和基础编程训练

- ❖ 应用计算机解决问题时，对需要保存和处理的数据

- ❖ 分清数据的内在联系

- ❖ 互不关联，一对一，一对多，多对多等

- ❖ 合理组织数据，选择合理的方式表达外部事物

- ❖ 表、树、图

- ❖ 有效处理数据

- ❖ 选择快速、节省资源的**数据结构和算法**解决问题

课程学习指导

课程特点：内容抽象、概念性强、内容灵活、理解后掌握

- 1.提前预习、认真听课、按时完成书面及上机作业
- 2.注意先修课程的知识准备
 - ✓ 离散数学、C语言
- 3.注意循序渐进：
 - ✓ 基本概念、基本思想、基本步骤、算法设计
- 4.注意培养算法设计的能力
 - ✓ 理解所讲算法、对此多做思考：若问题要求不同，应如何选择数据结构，设计有效的算法

课程目标

- 基础：掌握常用数据结构的概念
- 基础：掌握常用数据结构的实现方法
- 技能：通过系统学习在不同的存储结构上的运算方法，能够在不同存储结构上实现不同的运算，并对算法设计的方式和技巧有所体会。

课程目的

- 能够分析研究计算机加工的对象特性，获得其**逻辑结构**，根据需求，选择合适**存储结构及其相应的算法**；
- 学习一些**常用的算法**；
- 复杂程序设计的训练过程，要求编写的程序**结构清楚和正确易读**；
- 初步掌握算法的**时间分析和空间分析技术**

数据结构的研究内容



📖 N.沃思 (Niklaus Wirth)教授提出:

程序=算法+数据结构

📖 电子计算机的主要用途:

☞ 早期:

主要用于数值计算, 数学计算算法的研究

☞ 后来:

处理逐渐扩大到非数值计算领域, 能处理多种复杂的具有一定结构关系的数据

此时, 出现了数据结构的大量基础性的研究

数据的定义

- **数据 (data)** 是客观事物的**符号表示**，是所有能输入到计算机中并被计算机程序处理（输入、存储、运算、输出）的**符号的总称**。（P3）
- 任何需要被计算机处理的数据，首先要做的事情就是符号化：
 - 数值：整数（原码、补码）、实数（定点数、浮点数）
 - 字符：西文字符（ASCII码）、中文字符（Unicode, UTF）
 - 图像：采用像素化的方式，进行采样、量化和编码
 - 声音：采用时间分片的方式，进行PCM式采样、量化、编码
 - 自动控制：采用传感器的方式进行数字化采样，采用继电器的启动停止，步进电机的转动输出
- 数据大的类别分为：**数值数据**和**非数值数据**

数据的例子

- 数据分为：数值数据和非数值数据
- 数值数据典型应用是数值计算：
 - 指有效使用数字计算机求数学问题近似解的方法与过程，以及由相关理论构成的学科。数值计算主要研究如何利用计算机更好的解决各种数学问题，包括连续系统离散化和离散形方程的求解，并考虑误差、收敛性和稳定性等问题。
 - 数值运算的研究领域包括数值逼近、数值微分和数值积分、数值代数、最优化方法、常微分方程数值解法、积分方程数值解法、偏微分方程数值解法、计算几何、计算概率统计等。随着计算机的广泛应用和发展，许多计算领域的问题，如计算物理、计算力学、计算化学、计算经济学等都可归结为数值计算问题。
 - 主要是运用有限逼近的思想来进行误差运算

数据的例子

- **非数值数据**（字符、文字、图形、图像、声音等）
- 例子：图像的位图表示形式
- 在计算机中，图像是由一个个像素点组成，像素点就是颜色点，而颜色最简单的方式就是用RGB或RGBA表示，如图所示

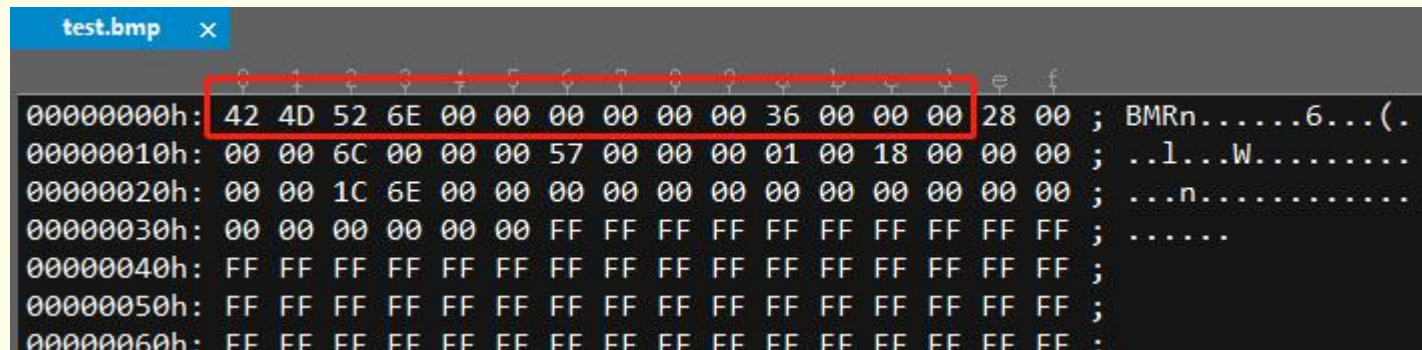
RGB	RGB	RGB	RGB	RGB
RGB	RGB	RGB	RGB	RGB
RGB	RGB	RGB	RGB	RGB
RGB	RGB	RGB	RGB	RGB
RGB	RGB	RGB	RGB	RGB

数据



数据的例子

- 在计算机中文件系统存储一个位图数据时，采用BMP文件格式
- BMP文件的数据按照从文件头开始的先后顺序分为四个部分：
 - 位图文件头(bitmap file header)：提供文件的格式、大小等信息 共14字节



- 位图信息头(bitmap information)：提供图像数据的尺寸、位平面数、压缩方式、颜色索引等信息 共40字节
- 调色板(color palette)：可选，如使用索引来表示图像，调色板就是索引与其对应的颜色的映射表
- 位图数据(bitmap data)：图像数据区

数据的例子

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	
00000000h:	42	4D	52	6E	00	00	00	00	00	00	36	00	00	00	28	00	; BMRn.....6...()
00000010h:	00	00	6C	00	00	00	57	00	00	00	01	00	18	00	00	00	; ..1...W.....
00000020h:	00	00	1C	6E	00	00	00	00	00	00	00	00	00	00	00	00	; ...n.....
00000030h:	00	00	00	00	00	00	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	;
00000040h:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	;
00000050h:	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	;

- 位图信息头(bitmap information)的定义

```
struct bmp_info //图像信息区
```

```
{  
    unsigned int bsize; //本结构体所占用字节数,即40个字节  
    int width;          // 位图的宽度, 以像素为单位  
    int height;         // 位图的高度, 以像素为单位  
    unsigned short planes; // 目标设备的级别, 必须为1  
    unsigned short count; //每个像素的色彩深度, 1(双色)4(16色)8(256色)或24(真彩色  
  
    unsigned int compression; // 位图压缩类型  
    unsigned int sizeimage;    // 位图的大小, 以字节为单位  
    unsigned int xmeter;      // 位图水平分辨率, 每米像素数  
    unsigned int ymeter;      // 位图垂直分辨率, 每米像素数  
    unsigned int cused;        // 位图实际使用的颜色表中的颜色数  
    unsigned int cimportant;    // 位图显示过程中重要的颜色数  
};
```

数据的例子

- 完整定义

```
struct bmp_head {  
    struct bmp_file fileHeader;  
    struct bmp_info fileInfo;  
    bypt * pPixel; // 用offset算出来  
};
```

```
struct bmp_head *bmp;  
bmp = (struct bmp_head *)mcalloc  
    (sizeof(struct bmp_head));  
fread(bmp, sizeof(struct bmp_attr), 1, fp);
```

- 从我们采用的结构定义来看，1个位图文件是1个数据，包含1个数据元素
- 这个数据元素包含：
 - fileHeader, fileInfo, 和位图像素字节块这些数据项组成;
 - fileHeader, fileInfo是组合项
 - 位图像素字节块，是很多个字节的顺序排列，也是组合项
 - 采用了一个指针pPixel指向其开始位置，是因为每个像素的像素色彩深度不同，导致的长度是变化的

数据的例子

- BMP文件存储位图图像这个数据元素的内容时，按从左到右、从下到上的顺序来保存图像的每个像素点的值。按一张图的全体像素点作为一个组合的数据项的角度来看：
- 每个像素点（按色彩深度的不同，字节数不同）是一个数据元素，整张图由（`bmp_info.width * bmp_info.height`）个数据元素按照矩阵的逻辑结构构成，再将矩阵转换为逻辑的线性表（按从左下到右上的顺序）
- 存储方式是用顺序存储的方式
- 左下角的像素的数据在文件中的开始的位置就是：
- `bmp->fileHeader.offset`
- 左上角的像素的数据在文件中的开始的位置是：
- `bmp->fileHeader.offset + (bmp->fileInfo.heightsize - 1) * line_length + 1`
- 注意：这里的`line_length`是根据`bmp_info.width`和`bmp_info.count`，结合四字节对准算出来的，这里就不详细介绍了。

数据结构的定义:

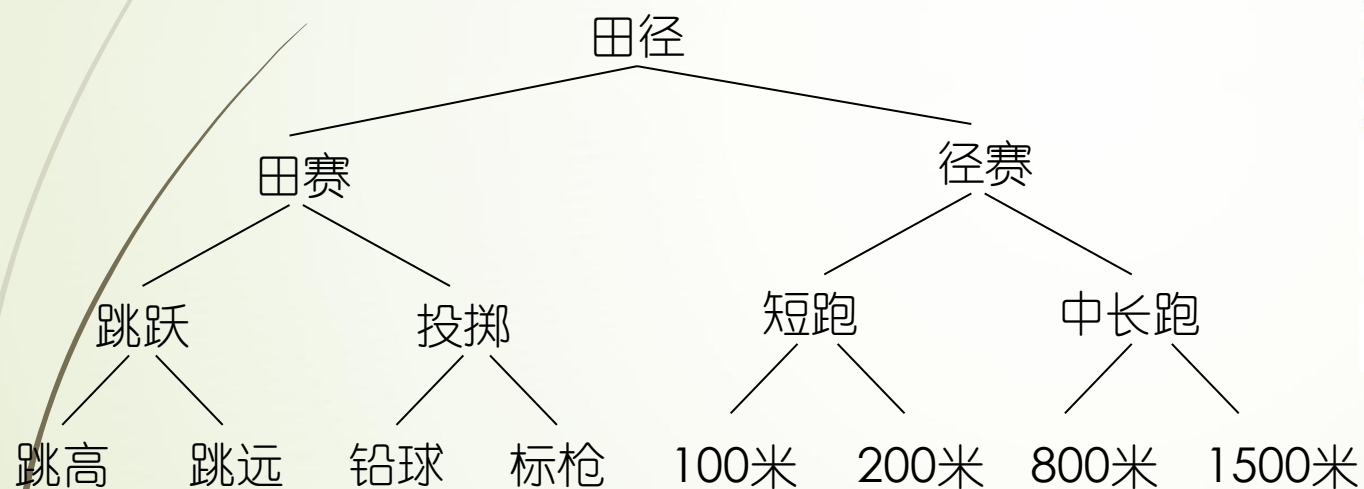
- **数据结构 (data struct)** 是指相互之间存在着一种或多种**特定关系的数据元素**的集合。 (P4)
- **数据元素 (data element)** 是**数据**的基本单位; 在计算机中通常做为一个整体进行考虑和处理; 一个**数据元素**可以由若干个**数据项**组成。 (P3) ,
- **数据项 (data element)** 是具有独立含义的表示单位, 是**数据**的不可分割的最小单位; **数据项**可以是**组合项** (可再由若干数据元素组成)。 (P3)
- **数据 (data)** 是对客观事物的符号表示; 是能被计算机识别、存储、和加工处理的符号的总称。 (P4)

数据 > 数据元素 > 数据项

数据由多个数据元素按照一种或多种关系组成, 单个数据元素往往由多个数据项组成。

学号	姓名	成绩			
		高等数学	C语言	英语	平均成绩
20171801	吴承志	90	95	85	90
20171802	李淑芳	88	76	91	85
20171803	刘 丽	92	78	82	84
20171804	张会友	81	78	72	77
20171805	石宝国	76	82	79	79
20171806	何文颖	86	90	91	89
20171807	赵胜利	76	78	80	78
20171808	崔文靖	82	93	86	87
20171809	刘 丽	80	85	81	82

- 数据：成绩表
- 数据元素：表中的一行
- 数据项：组成表中一行 的每个字段



- 数据：田径项目的组成关系树
- 数据元素：树中的结点
- 数据项：结点可能有的属性



- 数据：人际关系图
- 数据元素：图中的结点
- 数据项：结点可能有的属性，如姓名、年龄等

数据结构的定义:

- **数据结构 (data struct)** 是指相互之间存在着一种或多种**特定关系的数据元素**的集合。 (P4)
- 数据结构包括:
 - 多个数据元素间的**逻辑关系**，数据的**逻辑结构**
 - **集合、线性结构、树形结构、图形结构**
 - 数据在计算机中的**实际存储方式**，数据的**存储结构**
 - **顺序存储、链式存储、索引存储、散列存储**
 - 在这些数据上定义的一**组运算**，数据的**运算**或操作
 - **引用型运算、加工型运算**

逻辑结构---

数据元素间抽象化的相互关系，与数据的存储无关，独立于计算机，它是从具体问题抽象出来的数学模型。

存储结构（物理结构）----

数据元素及其关系在计算机存储器中的存储方式。

方面 层次	数据表示	数据处理
	逻辑结构	基本运算
抽象		
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

数据模型-表

学号	姓名	性别	专 业	年 级
000801	何文颖	女	计算机科学与技术	2000 级
000802	赵胜利	男	数学与应用数学	2000 级
000803	崔文靖	男	信息与计算科学	2000 级
010601	刘 丽	女	计算机科学与技术	2001 级
010602	魏永鸣	男	数学与应用数学	2001 级

例1 学生信息表

学号	姓名	高等数学	C语言	英语	平均成绩
20171801	吴承志	90	95	85	90
20171802	李淑芳	88	76	91	85
20171803	刘 丽	92	78	82	84
20171804	张会友	81	78	72	77
20171805	石宝国	76	82	79	79
20171806	何文颖	86	90	91	89
20171807	赵胜利	76	78	80	78
20171808	崔文靖	82	93	86	87
20171809	刘 丽	80	85	81	82

例2 学生成绩表A

学号	姓名	成绩			
		高等数学	C语言	英语	平均成绩
20171801	吴承志	90	95	85	90
20171802	李淑芳	88	76	91	85
20171803	刘 丽	92	78	82	84
20171804	张会友	81	78	72	77
20171805	石宝国	76	82	79	79
20171806	何文颖	86	90	91	89
20171807	赵胜利	76	78	80	78
20171808	崔文靖	82	93	86	87
20171809	刘 丽	80	85	81	82

例3 学生成绩表B

- 1、计算机中如何存储
- 2、可进行哪些操作，如何操作
- 3、上述存储和操作的实现是否高效，是否简单方便

例子-表---ASCII码表

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

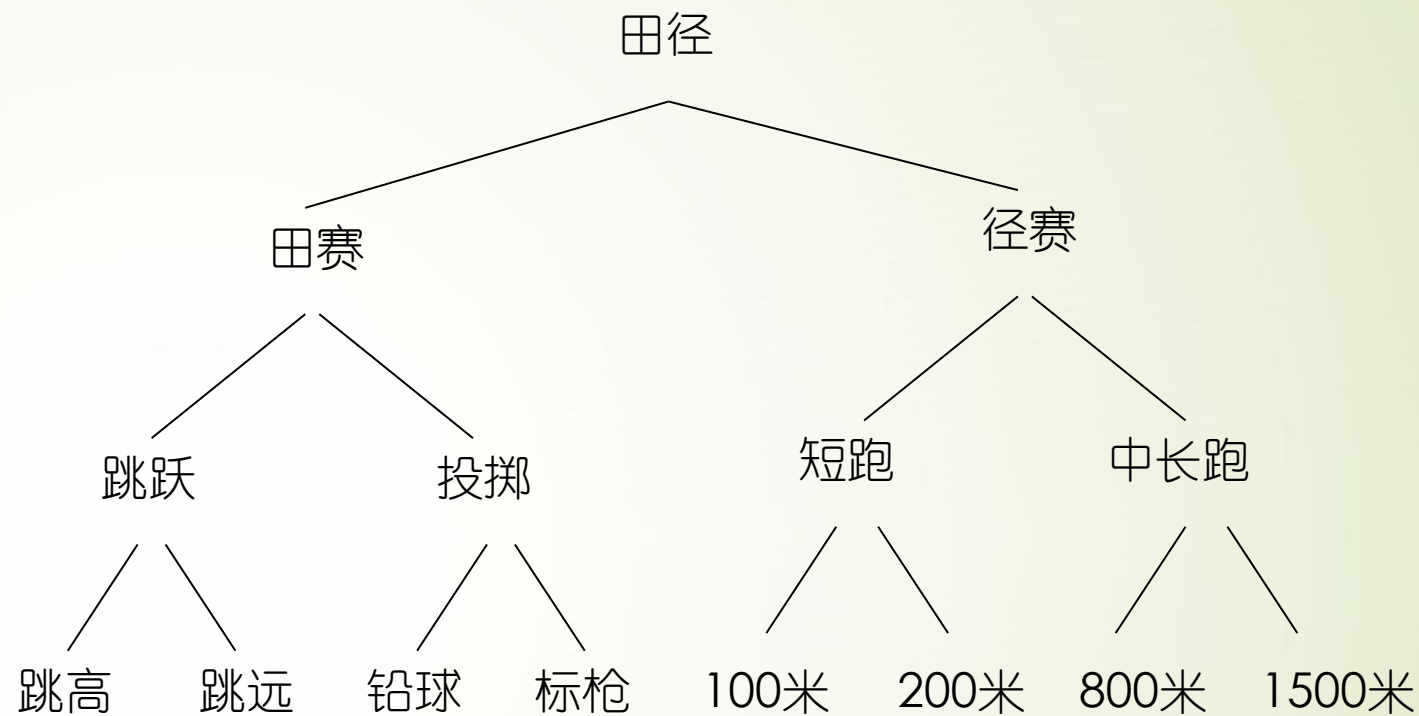
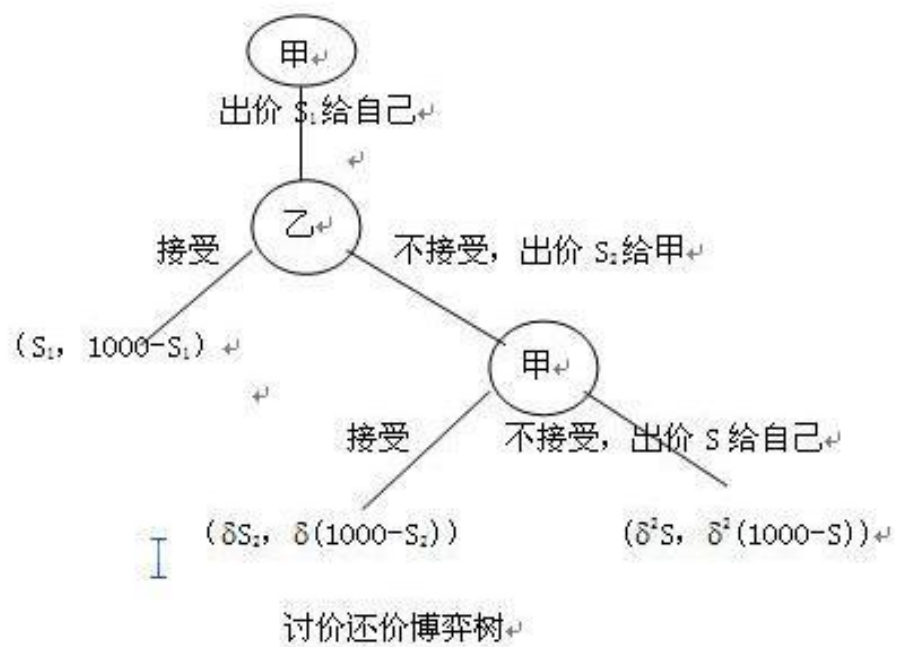
例子-表-windows PC内的路由表

IPv4 路由表

=====
活动路由:

网络目标	网络掩码	网关	接口	跃点数
0.0.0.0	0.0.0.0	10.103.240.1	10.103.241.86	55
10.103.240.0	255.255.240.0	在链路上	10.103.241.86	311
10.103.241.86	255.255.255.255	在链路上	10.103.241.86	311
10.103.255.255	255.255.255.255	在链路上	10.103.241.86	311
127.0.0.0	255.0.0.0	在链路上	127.0.0.1	331
127.0.0.1	255.255.255.255	在链路上	127.0.0.1	331
127.255.255.255	255.255.255.255	在链路上	127.0.0.1	331
224.0.0.0	240.0.0.0	在链路上	127.0.0.1	331
224.0.0.0	240.0.0.0	在链路上	10.103.241.86	311
255.255.255.255	255.255.255.255	在链路上	127.0.0.1	331
255.255.255.255	255.255.255.255	在链路上	10.103.241.86	311

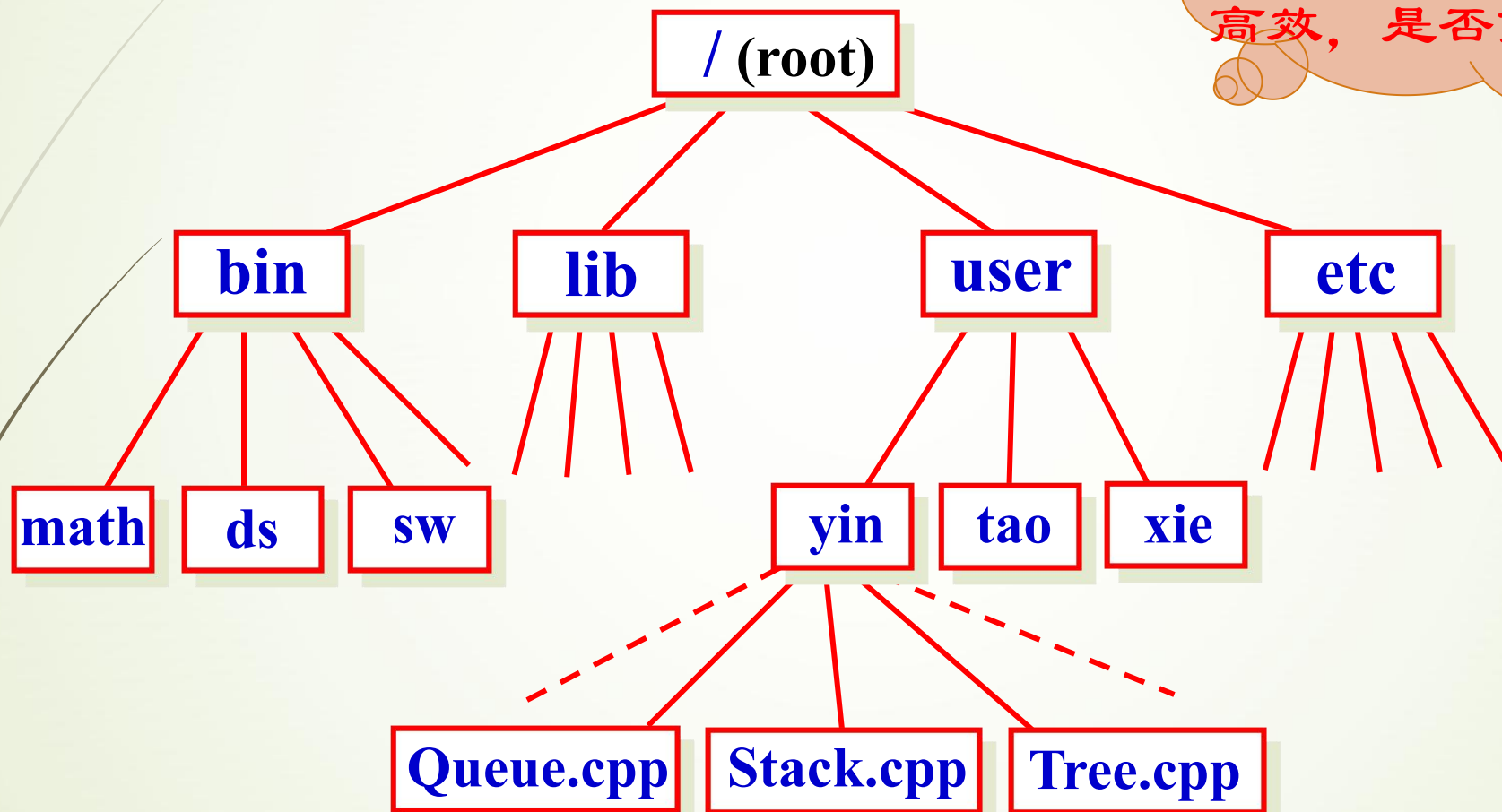
数据模型-树



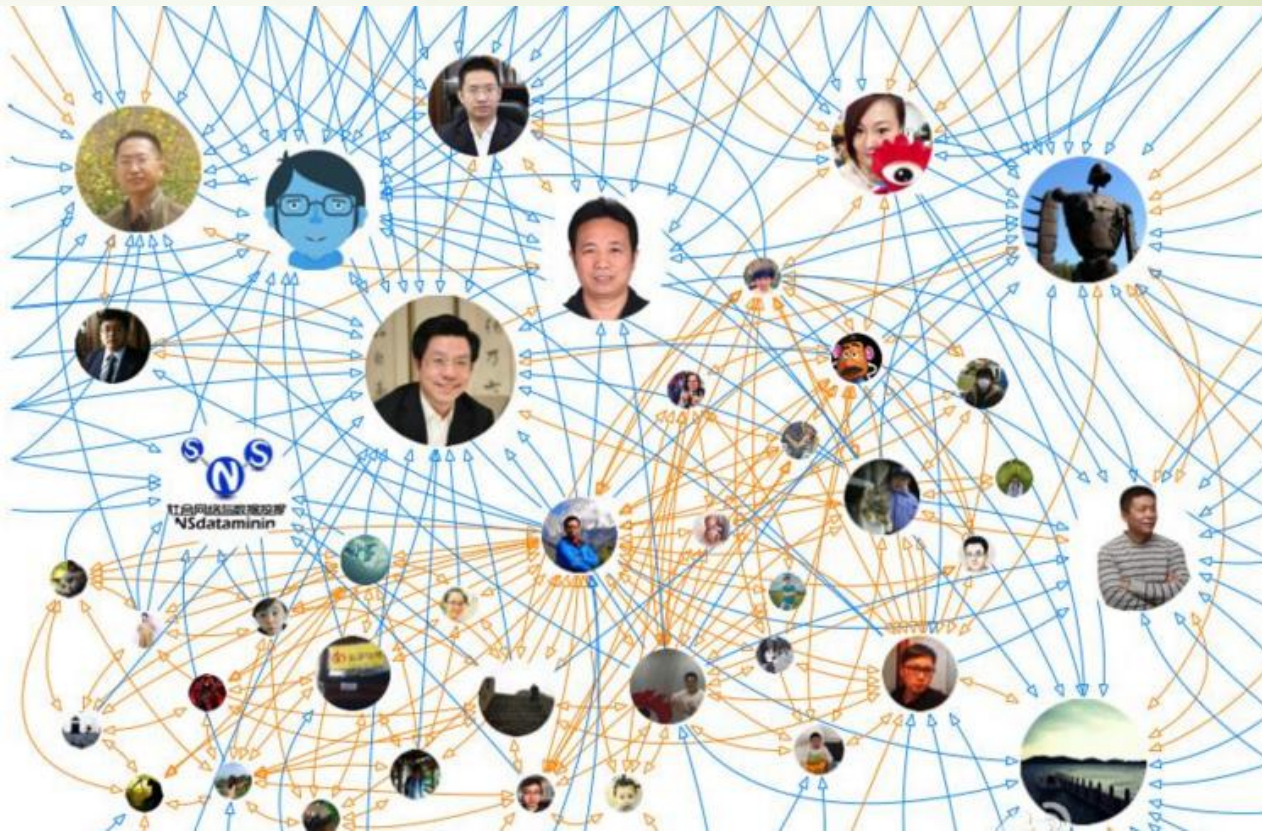
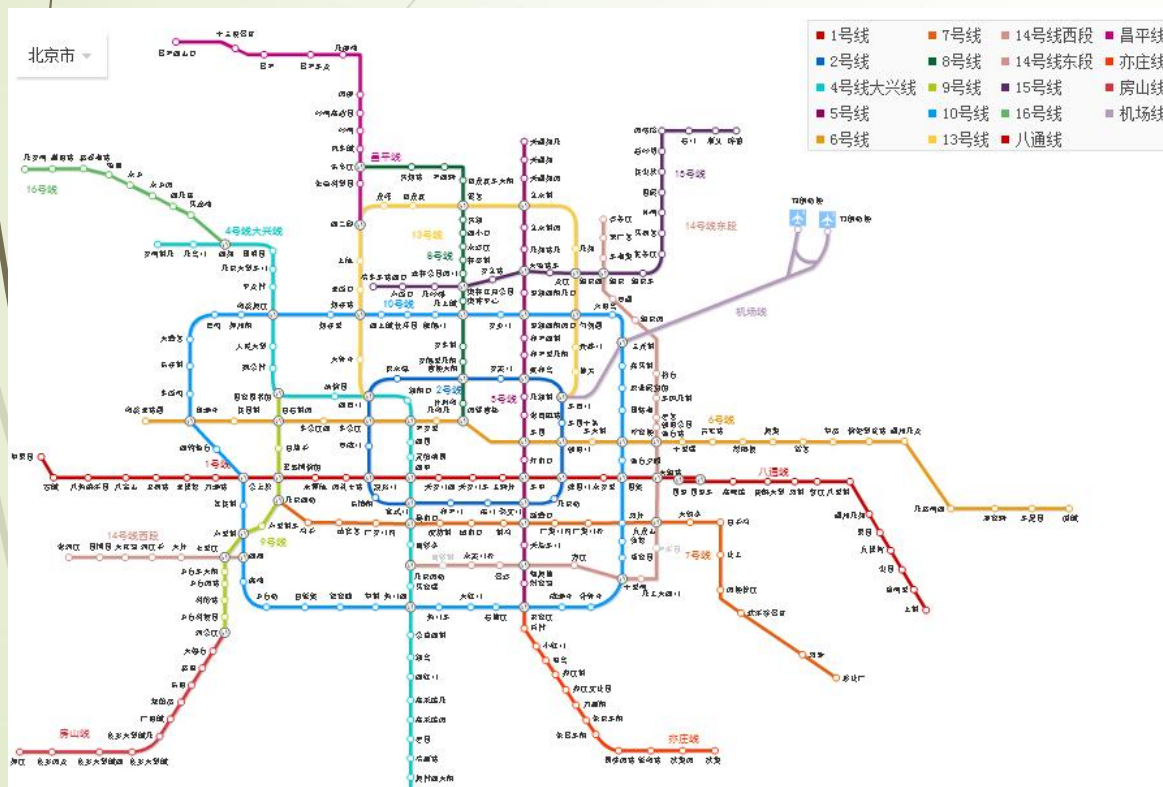
- 1、计算机中如何存储
- 2、可进行哪些操作，如何操作
- 3、上述存储和操作的实现是否高效，是否简单方便

数据模型-树

- 1、计算机中如何存储
- 2、可进行哪些操作，如何操作
- 3、上述存储和操作的实现是否高效，是否简单方便



数据模型-图

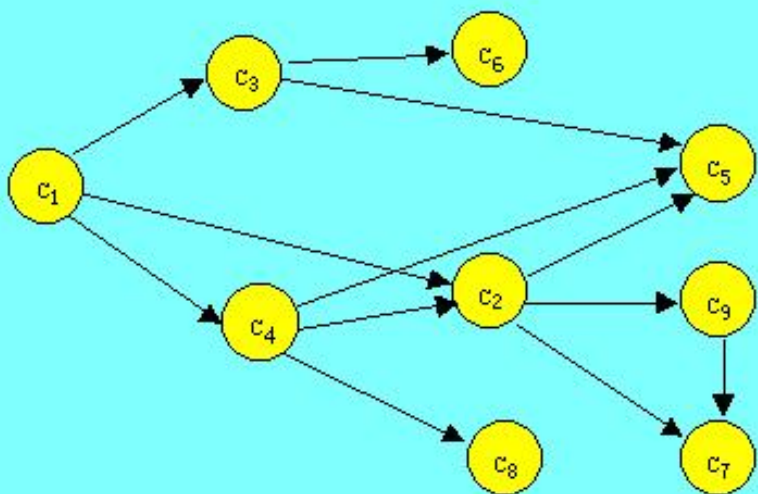


- 1、计算机中如何存储
- 2、可进行哪些操作，如何操作
- 3、上述存储和操作的实现是否高效，是否简单方便

数据模型-图

课程编号	课程名称	先修课程
C ₁	计算机导论	无
C ₂	数据结构	C ₁ , C ₄
C ₃	汇编语言	C ₁
C ₄	C 程序设计语言	C ₁
C ₅	计算机图形学	C ₂ , C ₃ , C ₄
C ₆	接口技术	C ₃
C ₇	数据库原理	C ₂ , C ₉
C ₈	编译原理	C ₄
C ₉	操作系统	C ₂

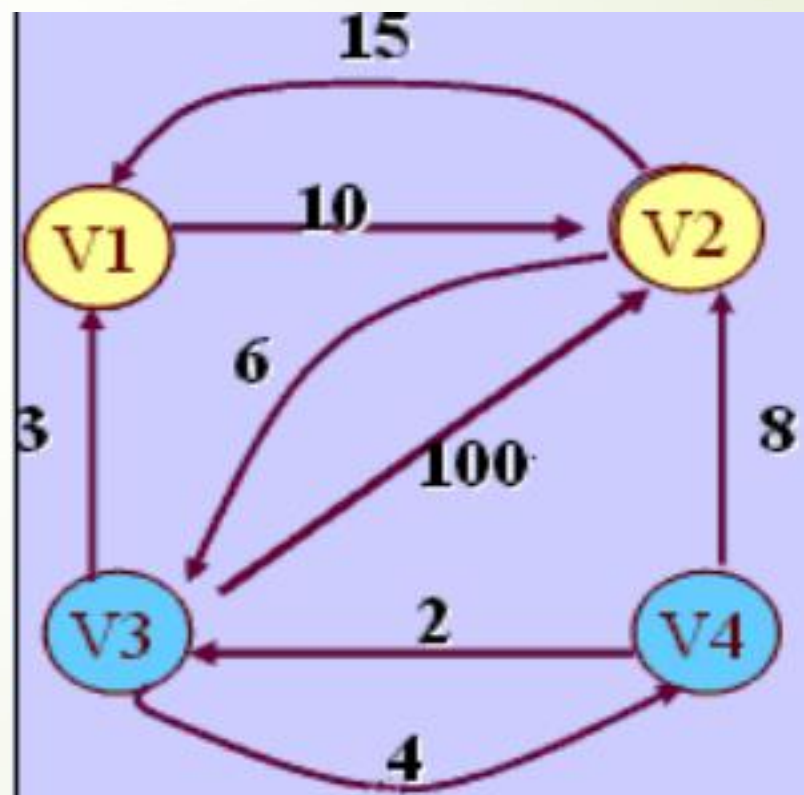
(a) 计算机专业的课程设置



(b) 表示课程之间优先关系的有向图

图 1.3 教学计划编排问题的数据结构

- 1、计算机中如何存储
- 2、可进行哪些操作，如何操作
- 3、上述存储和操作的实现是否高效，是否简单方便



求解非数值计算的问题：

设计出合适的数据结构及相应的算法

即：首先要考虑对相关的各种信息如何表示、组织和存储？

数据结构的研究内容为：

研究非数值计算的程序设计问题中计算机的
操作对象以及它们之间的关系和操作。

方面 层次	数据表示	数据处理
	逻辑结构	基本运算
抽象		
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	



数据结构课程的形成和发展：

形成阶段：

60年代初期，“数据结构”有关的内容散见于操作系统、编译原理和表处理语言等课程。1968年，“数据结构”被列入美国一些大学计算机科学系的教学计划。

发展阶段：

数据结构的概念不断扩充，包括了网络、集合代数论、关系等“离散数学结构”的内容。

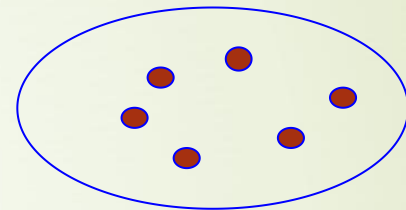
70年代后期，我国高校陆续开设该课程。

逻辑结构

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

划分方法一

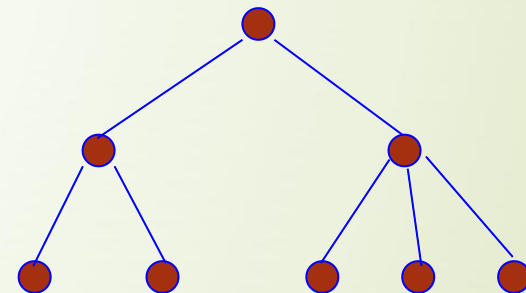
集合——数据元素间除“同属于一个集合”外，无其它关系，如，一堆沙子



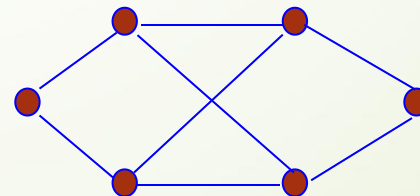
线性结构——一个对一个，有逻辑的先后关系，包括：线性表、栈、队列等；如，一根链条，一个单词中的所有字母



树形结构——一个对多个，树、二叉树、森林如，一棵树，家谱树等



图形结构——多个对多个，图如，交通图，高铁图



逻辑结构

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

划分方法二：按结点间的对应关系来分

即，**线性结构**和其他（**非线性结构**）。

(1) 线性结构---- (1对1)

有且仅有一个开始和一个终端结点，并且所有结点都最多只有一个直接前趋和一个后继。

例如：线性表、栈、队列、串

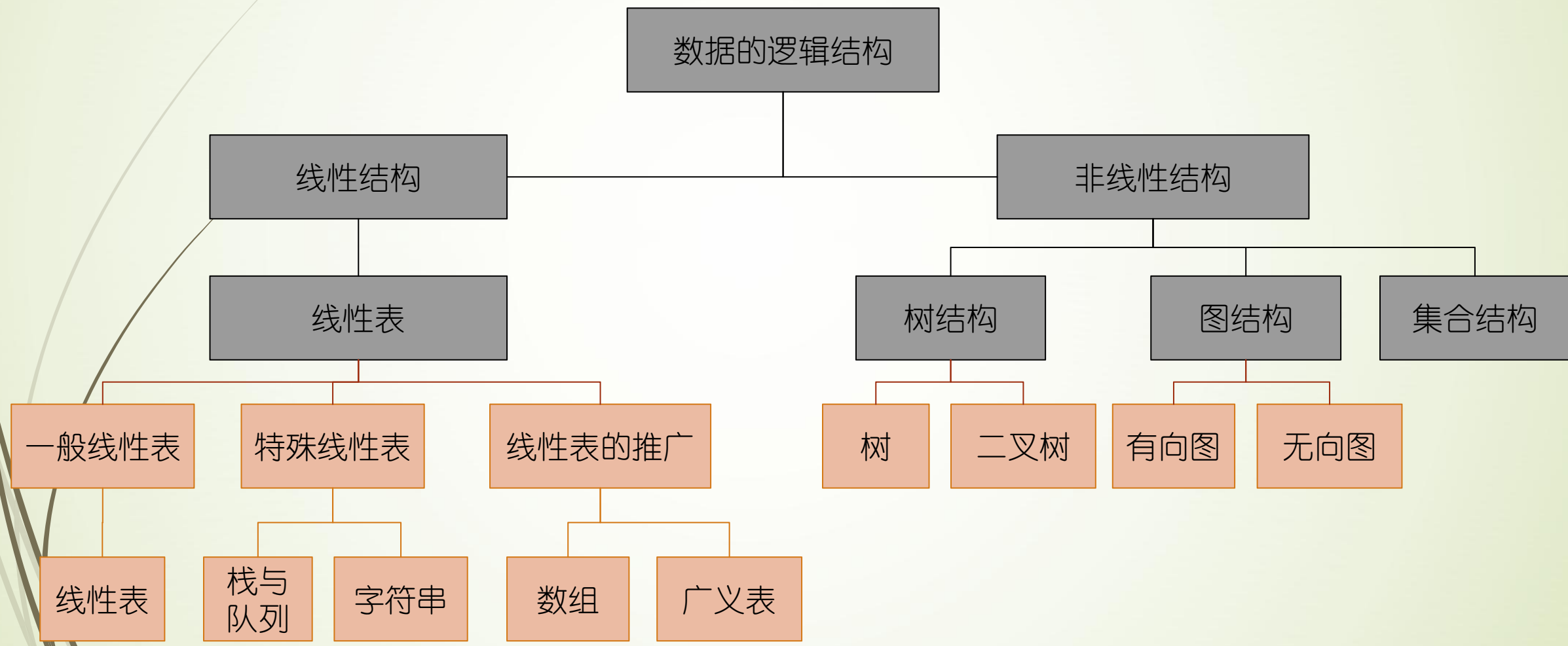
(2) 非线性结构---- (1对多或1对0)

一个结点可能有多个或0个直接前趋和直接后继。

例如：树、图、集合

逻辑结构总结

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

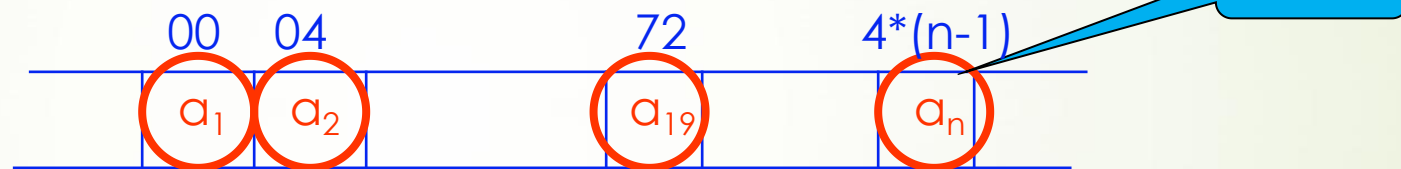


存储结构

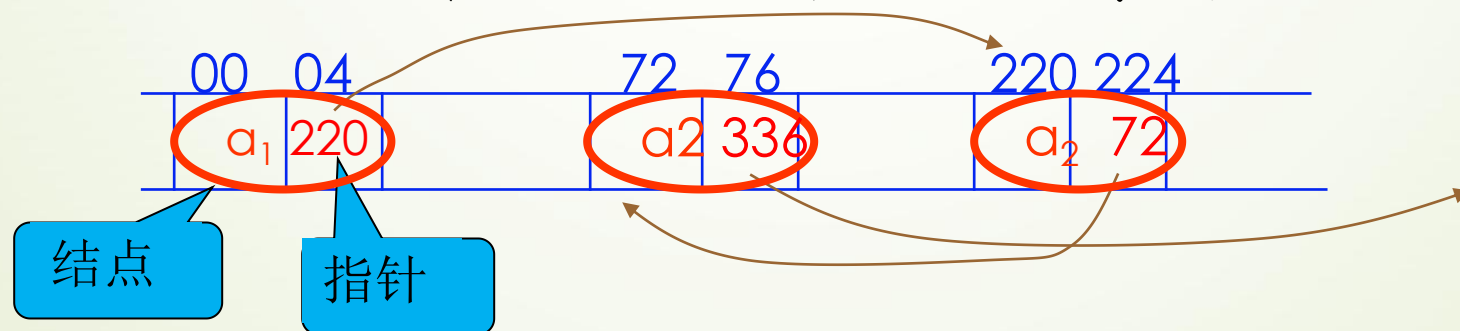
方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

存储结构分为：

顺序存储结构——数据元素依次放在连续的存储单元中表示数据元素间的逻辑关系（如前后关系），相对位置可通过计算得到

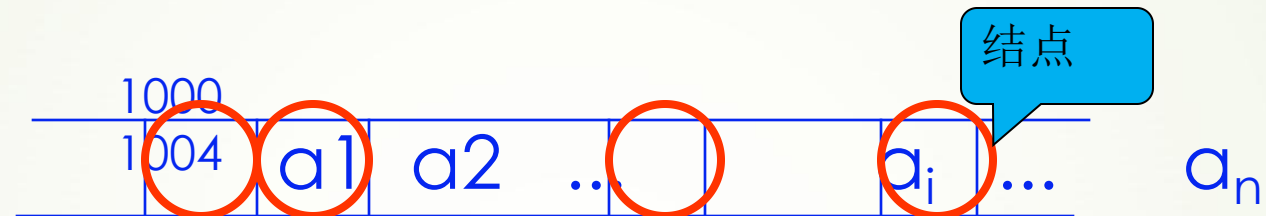


链式存储结构——借助指示元素存储地址的**指针**表示数据元素间的逻辑关系（如前后关系），需通过遍历（依次访问）链表得到指定元素的存储位置

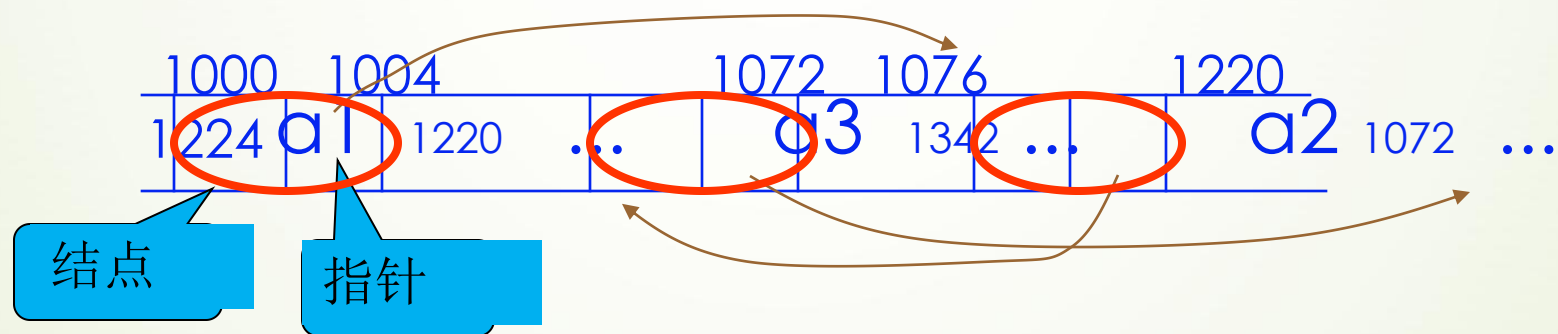


数据存储方式的四种常用结构

(1) **顺序存储**: 数据元素依次放在连续的存储单元中。



(2) **链式存储**: 在存储结点中增加若干指针域，记录后继或者相关结点的地址（指针）。



存储地址 虚拟存储

L_0	元素1
L_0+m	元素2
	
$L_0+(i-1)*m$	元素i
	
$L_0+(n-1)*m$	元素n

$Loc(\text{元素}i)=L_0+(i-1)*m$,
其中元素的大小为m

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

顺序存储主要特点

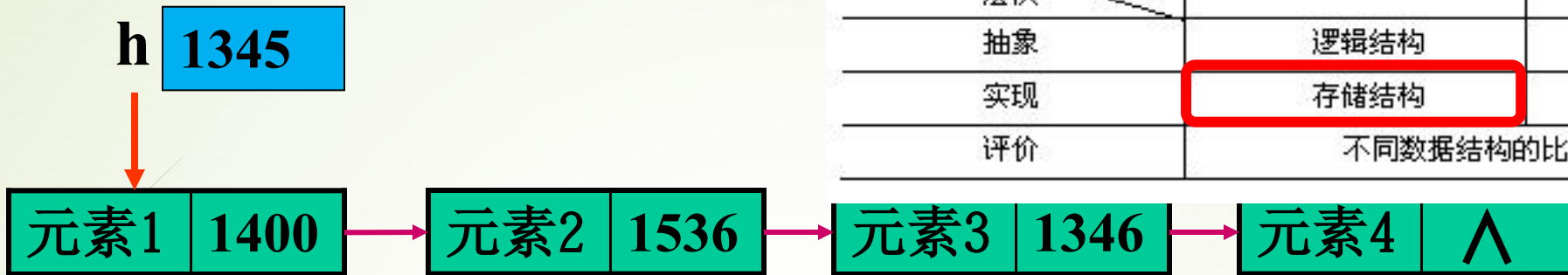
- (1) 结点中只有自身信息域，没有连接信息域。因此存储密度大，利用率高。
- (2) 可以通过计算确定数据结构中第i个元素的位置，可以（直接）随机存取。
- (3) 插入和删除操作会引起大量的元素移动。
- (4) 必须存储在一片地址连续的存储单元中。

顺序表的类型定义

```
typedef struct {  
    int a, b, c;  
    char ch;  
}ElemType;
```

```
#define MAXSIZE 100    //最大长度  
typedef struct {  
    ElemType elem[MAXSIZE];    //连续的地址空间  
    int length;                //线性表的当前长度  
}SqList;
```

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	



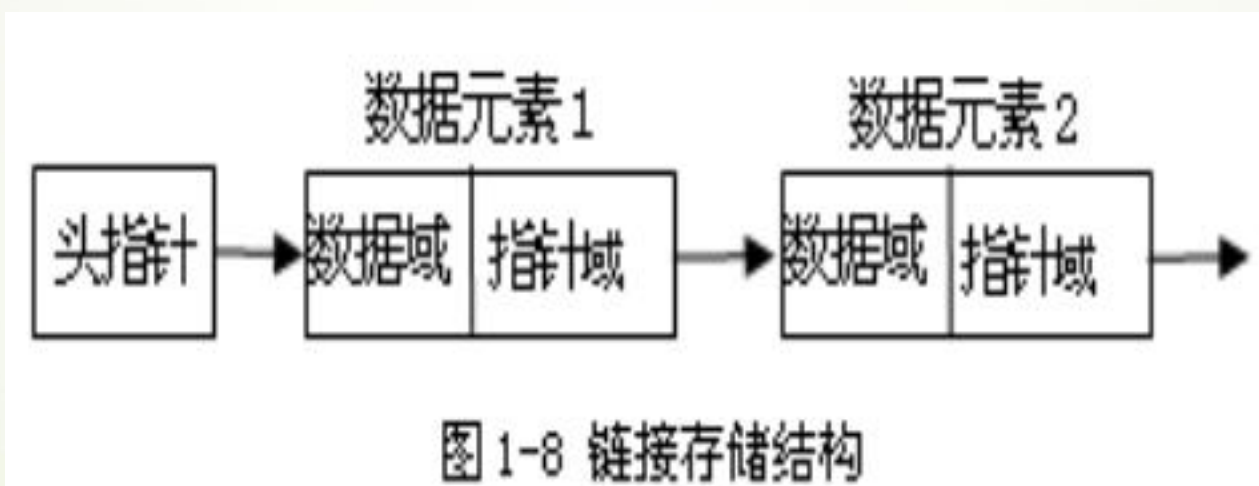
存储地址	存储内容	指针
1345	元素1	1400
1346	元素4	^
.....		
1400	元素2	1536
.....		
1536	元素3	1346

数据元素间的相邻关系（前后关系）不是通过存储位置表示的，而是链接关系表示的。

链式存储主要特点：

- （1）结点除有自身信息域外，还有表示连接信息的指针域。因此存储密度小，利用率低。
- （2）逻辑上相邻的结点在物理上不一定邻接，使用于复杂的逻辑结构。
- （3）插入和删除操作灵活方便；只要修改指针域的值即可。
- （4）可以存储在不连续的存储单元中。

在这种存储结构中，可以把逻辑上相邻的元素存放在物理上不相邻的存储单元中；也可以在线性编址的计算机存储器中表示结点的非线性联系。



如何实现？

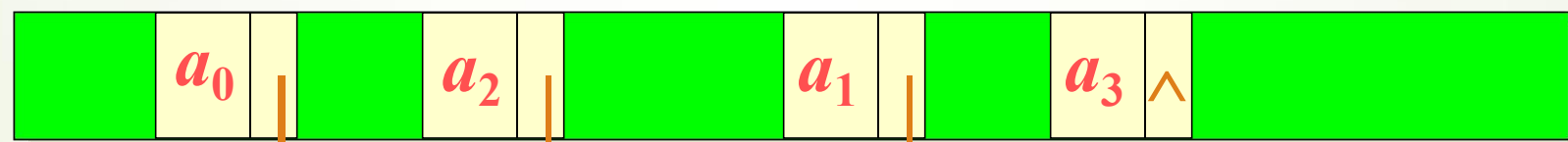
通过**指针**来实现

单链表的存储映像



↑
free

(a) 可利用存储空间



↑
first

↑
free

(b) 经过一段运行后的单链表结构

单链表的存储结构定义

```
typedef struct LNode{  
    ElemType    data;           //数据域  
    struct LNode *next;        //指针域  
} LNode, *LinkList; // *LinkList为Lnode类型的指针
```

LNode *p

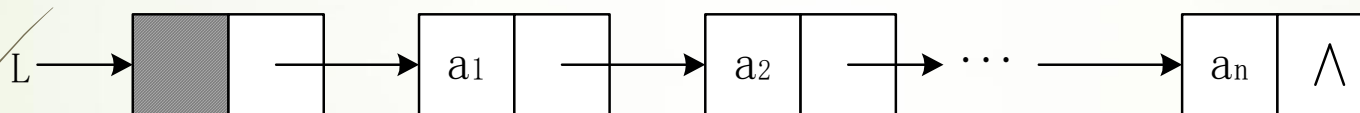


LinkList p

注意区分指针变量和结点变量两个不同的概念

- 指针变量p: 表示结点地址
- 结点变量*p: 表示一个结点

LNNode *p

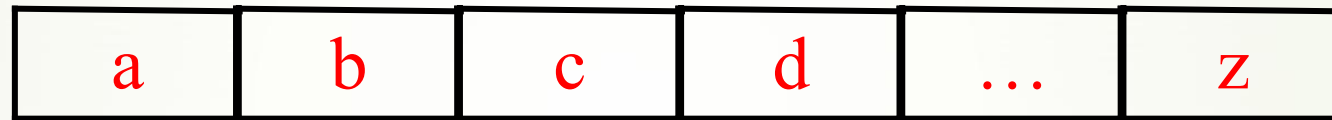


若 $p \rightarrow \text{data} = a_i$, 则 $p \rightarrow \text{next} \rightarrow \text{data} = a_{i+1}$

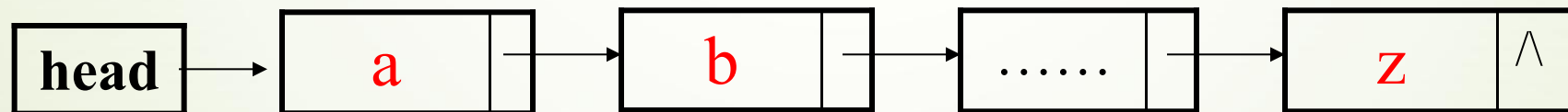
例 画出26个英文字母表的顺序存储和链式存储结构

逻辑结构: (a, b, ..., y, z)

顺序存储结构:

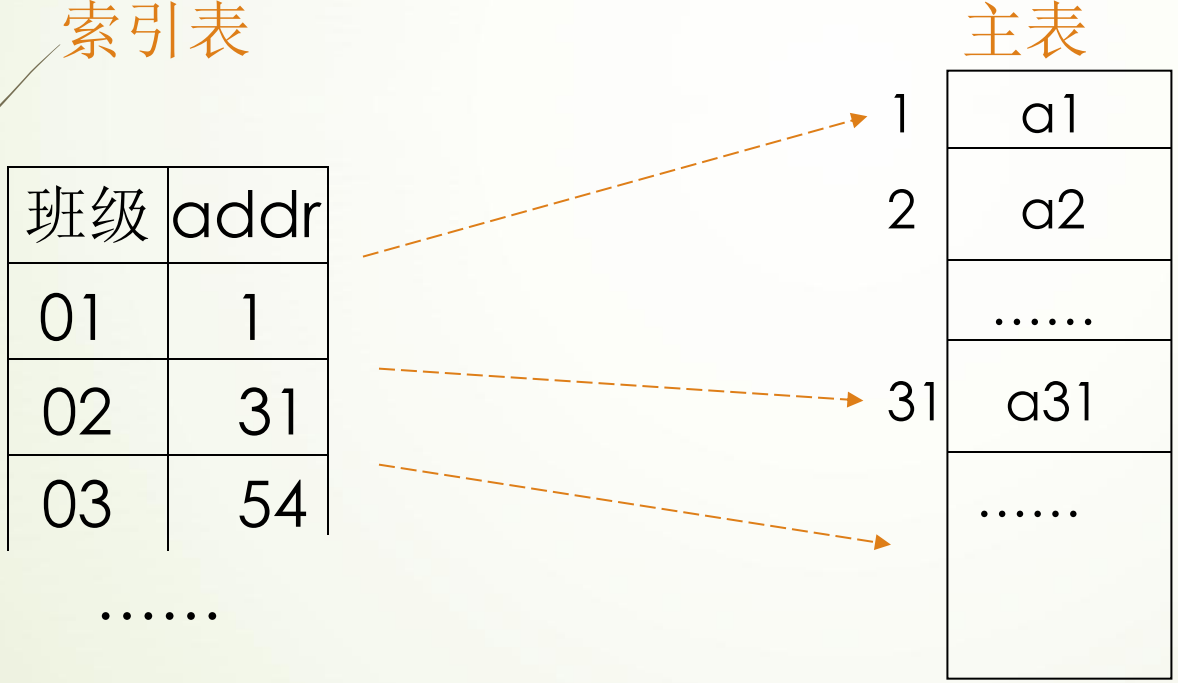


链式存储结构:



数据存储方式的四种常用结构

(3)索引存储： 在存储结点信息的同时，建立索引表，存放关键字（唯一标识一个数据元素的数据项）和地址（指出数据元素所在的位置）。



目录

CONTENTS

第一章 冲泡美味咖啡

让咖啡越来越美味..... 6

第1节 挑选咖啡豆..... 8

阿拉比卡和罗布斯塔..... 8

咖啡豆的等级..... 10

烘焙程度..... 14

研磨咖啡豆..... 16

第2节 冲泡咖啡..... 20

咖啡壶的种类..... 20

冲泡方法与咖啡风味的关系..... 22

滴漏式咖啡壶..... 24

滴漏式咖啡壶..... 30

滴漏式与滴漏式的冲泡指南..... 34

法式咖啡壶..... 40

意式咖啡机..... 42

摩卡壶..... 44

咖啡的功效..... 48

第二章 21种花式咖啡秘方

咖啡店必点咖啡与人气咖啡饮品集... 50

第1节 咖啡店必点咖啡..... 51

欧蕾咖啡..... 51

摩卡咖啡..... 52

卡布奇诺..... 53

维也纳咖啡..... 54

肉桂咖啡..... 55

数据存储方式的四种常用结构

(4)散列存储（哈希存储）： 根据数据元素的关键字值，由散列函数计算出存储地址 $LOC(a_i)=H(key)$

.....	432		713	
-------	-----	-------	-----	-------

王小二

李一凡

数据的运算

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

- 对于一种数据结构,常见的运算指的是逻辑结构上的运算(操作),基本运算包括:
 - 插入(增)
 - 删除(删)
 - 修改(改)
 - 查找(查)
 - 排序(有时在增、删、改的过程中一起完成)
- 逻辑结构和存储结构都相同,但运算不同,则数据结构不同.例如,栈与队列(都是线性表,但栈的运算在逻辑结构的第一个位置进行,队列可以在第一个、最后一个及某个指定位置进行)。

数据结构的定义:

- **数据结构 (data struct)** 是指相互之间存在着一种或多种**特定关系**的**数据元素**的集合。 (P4)
- 数据结构包括:
 - 多个数据元素间的**逻辑关系**，数据的**逻辑结构**
 - **集合、线性结构、树形结构、图形结构**
 - 数据在计算机中的**实际存储方式**，数据的**存储结构**
 - **顺序存储、链式存储、索引存储、散列存储**
 - 在这些数据上定义的一**组运算**，数据的**运算**或操作
 - **引用型运算、加工型运算**

N.沃思 (Niklaus Wirth)教授提出：程序=算法+数据结构

求解非数值计算的问题时，设计出合适的数据结构及相应的算法

即：首先要考虑对相关的各种信息如何表示、组织和存储？

数据结构的研究内容为：

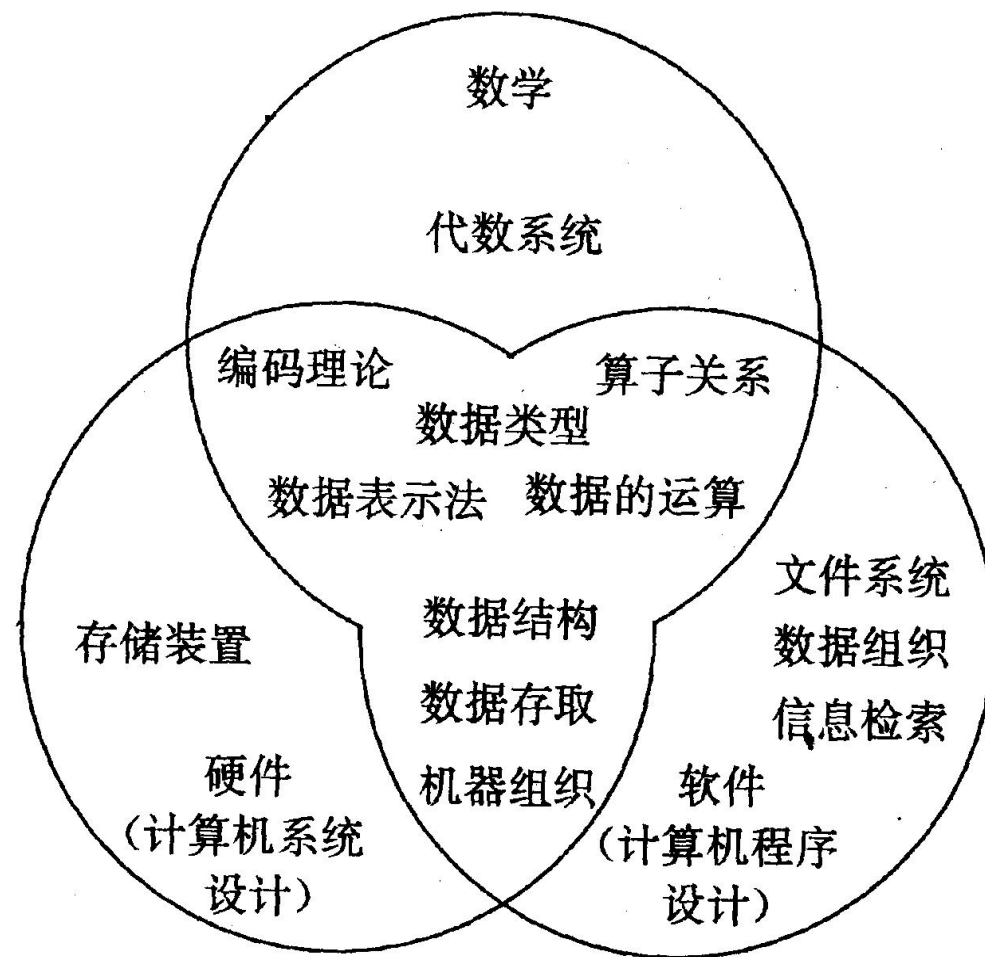
研究非数值计算的程序设计问题中计算机的**操作对象**以及它们之间的**关系和操作**。

方面 层次	数据表示	数据处理
	逻辑结构	基本运算
抽象		
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	



《数据结构》所处的地位：

介于数学、计算机硬件和计算机软件三者之间的一门核心课程



数据结构在计算机学科中的地位

Web信息处理

队列、图、字符、矩阵、哈希表、排序、索引、检索

人工智能

广义表、集合、有向图、搜索树

图形图像

队列、栈、图、矩阵、空间索引树、检索

数据库

线性表、多链表、排序、B+树

操作系统

队列、存储管理表、排序、目录树

编译原理

字符串、栈、哈希表、语法树

算法分析与设计

数据结构与算法

计算复杂性理论

程序设计语言

概率统计

计算概论

集合论与图论

为什么要学习数据结构

- ❖ 编程基础和基础编程训练

- ❖ 应用计算机解决问题时，对需要保存和处理的数据

- ❖ 分清数据的内在联系

- ❖ 互不关联，一对一，一对多，多对多等

- ❖ 合理组织数据，选择合理的方式表达外部事物

- ❖ 表、树、图

- ❖ 有效处理数据

- ❖ 选择快速、节省资源的**数据结构和算法**解决问题

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

逻辑结构

唯一

存储结构

不唯一

运算的实现

依赖于

存储结构

数据的逻辑结构

线性结构

非线性结构

线性表

栈、队列

串、数组

树形结构

图形结构

数据的存储结构

顺序存储

链式存储

数据的基本运算：插入、删除、修改、查找、排序

数据类型

- **定义：**在一种程序设计语言中，变量所具有的数据种类

FORTRAN语言：整型、实型、和复数型

C语言：

基本数据类型：char int float double void

构造数据类型：数组、结构体、共用体、文件

- ✓ 数据类型是一组性质相同的值的集合, 以及定义于这个集合上的一组运算的总称

抽象数据类型

抽象数据类型

(ADTs: Abstract Data Types)

- ◆ 更高层次的数据抽象
- ◆ 由用户定义，用以表示应用问题的**数据模型**
- ◆ 由**基本的数据类型**组成，并包括**一组相关的操作**

数据的逻辑结构+运算的定义-----面向用户
(抽象数据类型) 概念层

数据的存储结构+运算的实现-----面向计算机
实现层

抽象数据类型可以用以下的三元组来表示：

$$\text{ADT} = (\text{D}, \text{S}, \text{P})$$

数据对象 D上的关系集 D上的操作集

ADT抽象数据类型名{

数据对象：<数据对象的定义>

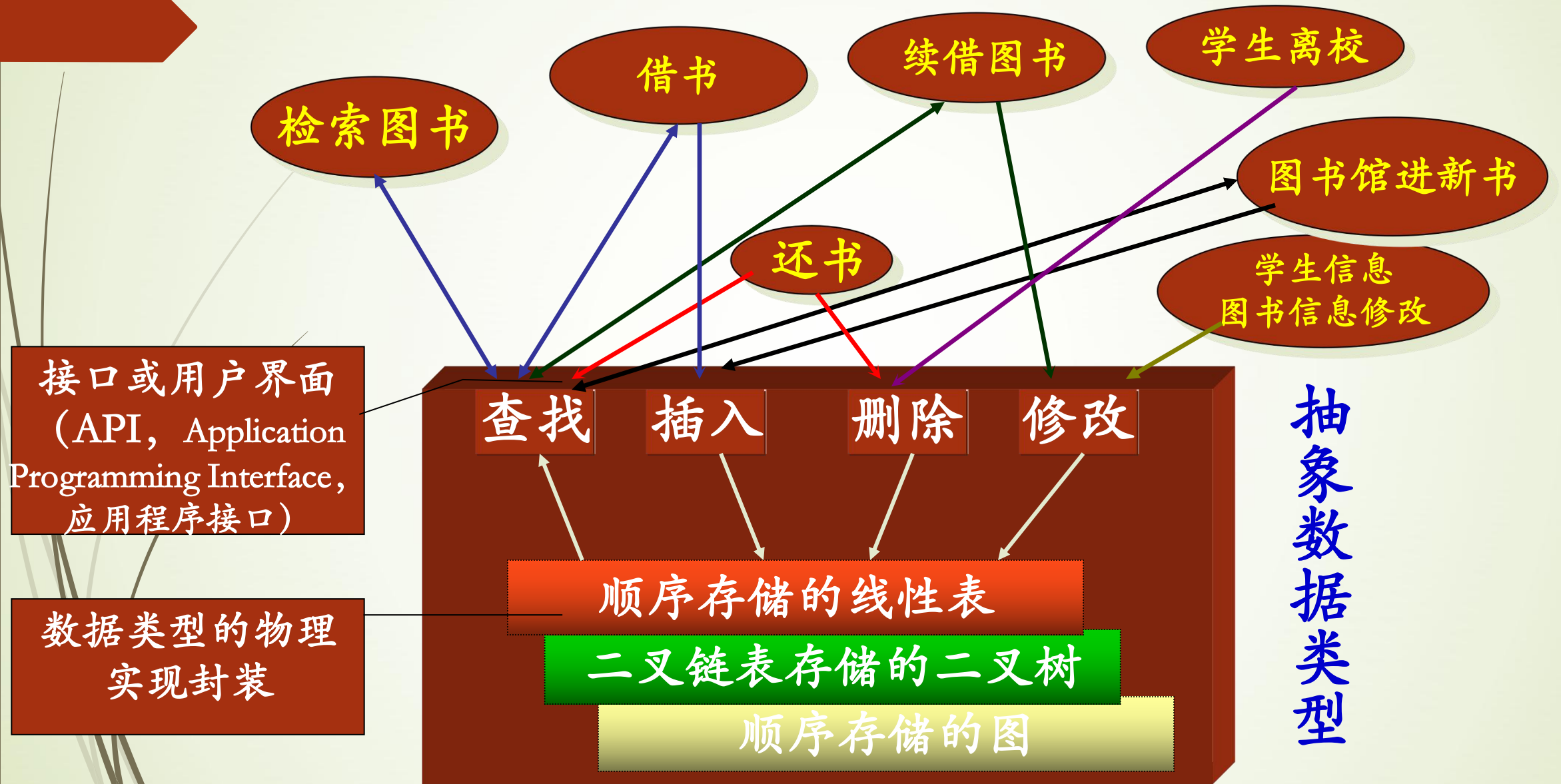
数据关系：<数据关系的定义>

基本操作：<基本操作的定义>

} ADT抽象数据类型名

ADT
常用
定义
格式

信息隐蔽和数据封装，使用与实现相分离



1.3 抽象数据类型的表示与实现



抽象数据类型可以通过**固有的**数据类型（如整型、实型、字符型等）来表示和实现。

它有些类似C语言中的**结构（struct）类型**，但增加了相关的**操作**

教材中用的是**类C语言**（介于伪码和C语言之间）
作为描述工具

但上机时要用具体语言实现，如C或C++等

1.4 算法和算法分析



- 算法定义：一个有穷的指令集，这些指令为解决某一特定任务规定了一个运算序列
- 在数据结构课程中，狭义的算法是指在存储结构上实现数据的逻辑结构所必须的基本运算的代码序列。
- 通常在计算机的问题解决中，我们是针对问题设计一组数据描述的逻辑结构和运算方法，并通过设计算法来实现数据的运算，从而最终解决掉问题。

- 算法的描述方法：

- ◆ 自然语言
- ◆ 流程图
- ◆ 程序设计语言
- ◆ 伪码

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

■ 算法的特性：

- ◆ **有穷性** 一个算法必须总是在执行有穷步之后结束，且每一步都在有穷时间内完成。
- ◆ **确定性** 算法中每一条指令必须有确切的含义。不存在二义性。且算法只有一个入口和一个出口。
- ◆ **可行性** 一个算法是可行的。即算法描述的操作都是可以通过已经实现的基本运算执行有限次来实现的。
- ◆ **输入** 一个算法有零个或多个输入，这些输入取自于某个特定的对象集合。
- ◆ **输出** 一个算法有一个或多个输出，这些输出是同输入有着某些特定关系的量。



算法有好坏之分！

即通过几个方面来对算法进行评价

- ◆ 正确性
- ◆ 可读性
- ◆ 健壮性
- ◆ 高效性（效率与低存储量需求）
 - ◆ 时间代价，效率
 - ◆ 空间代价，存储量使用需求
 - ◆ 这两个是互相影响的，要找到针对被解决的问题的最佳平衡点



求10个数的最大值。

```
main()
```

```
{
```

```
int max=0,x;
```

```
for(int i=1;i<=n;i++)
```

```
{ scanf("%d",&x);
```

```
if(x>max) max=x;}
```

```
printf("%d",max);
```

```
}
```

提示：本程序无语法错误

格式不好，不满足算法的“可读性”要求。



求10个数的最大值。

```
main()
{
    int max=0,x,i,n=10;
    for(i=1;i<=n;i++)
    {
        scanf("%d",&x);
        if(x>max)
        {
            max=x;
        }
    }
    printf("%d",max);
}
```

提示：本程序无语法错误

当输入全为正数时，结果正确；
当输入全为负数时，求得的最大值为0，结果不正确。
既要输入正确的数据，也要输入错误的数据进行健壮性测试。

不满足算法的“正确性”要求。

算法效率

不同算法的比较，求 $1+2+3+\dots+100$ 的程序。

```
main()
{
    int i,s=0;n=100;
    s=(1+n)*n/2;
    printf("%d",s);
}
```

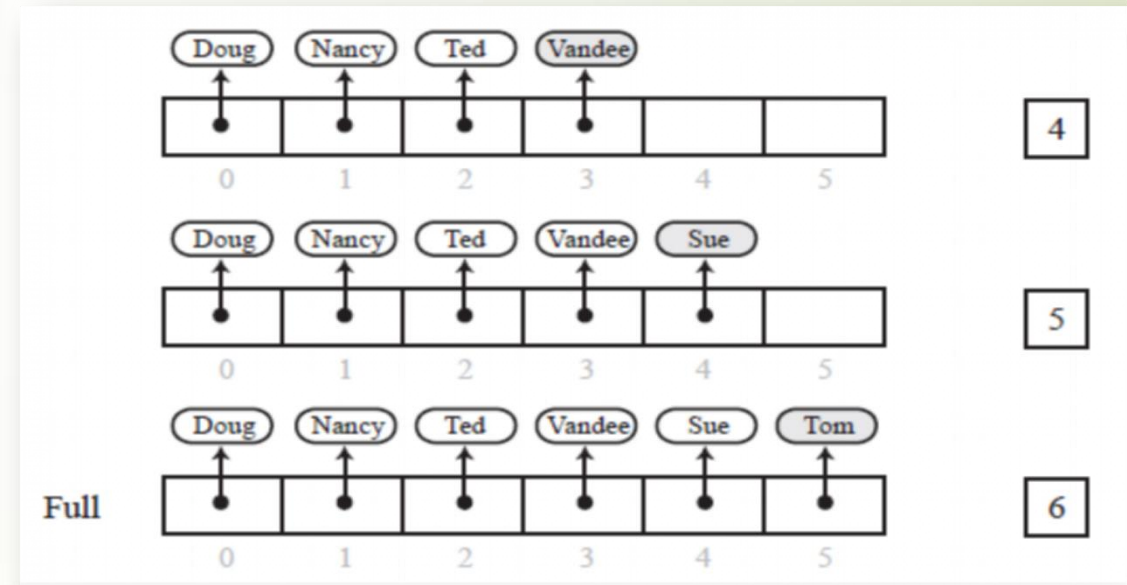
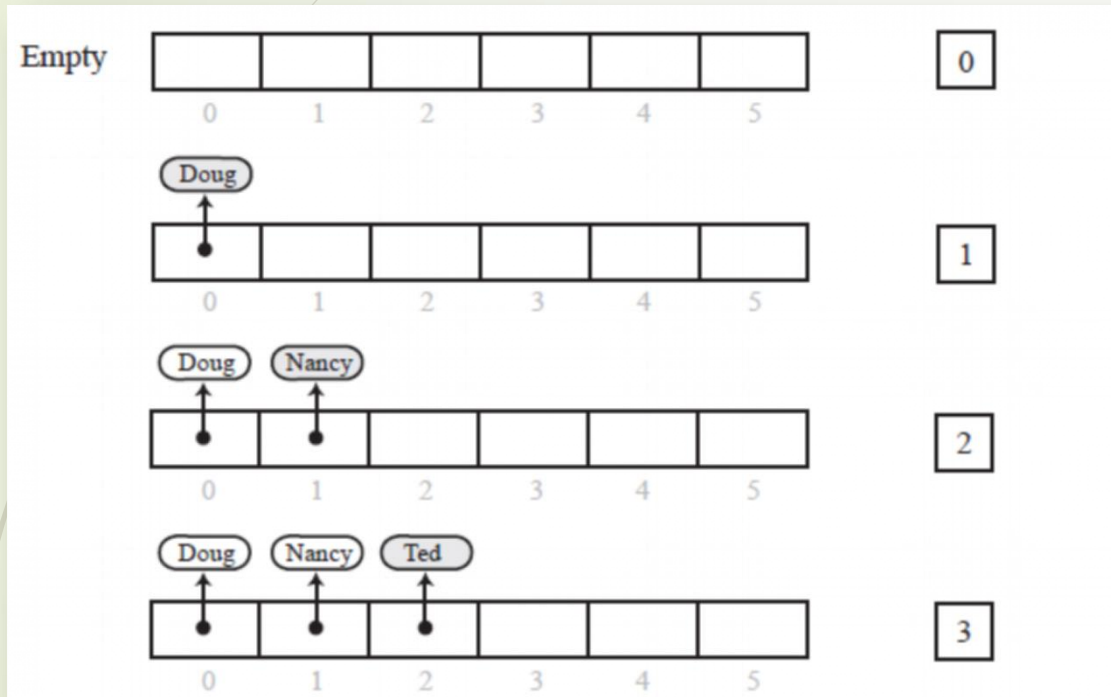
$$\begin{array}{r} s = 1 + 2 + 3 + \dots + 99 + 100 \\ + \quad s = 100 + 99 + 98 + \dots + 2 + 1 \\ \hline 2S = 101 + 101 + 101 + \dots + 101 + 101 \end{array}$$

```
main()
{
    int i,s=0;n=100;
    for(i=1;i<=100;i++)
        s=s+i;
    printf("%d",s);
}
```

这是数学家高斯的解释

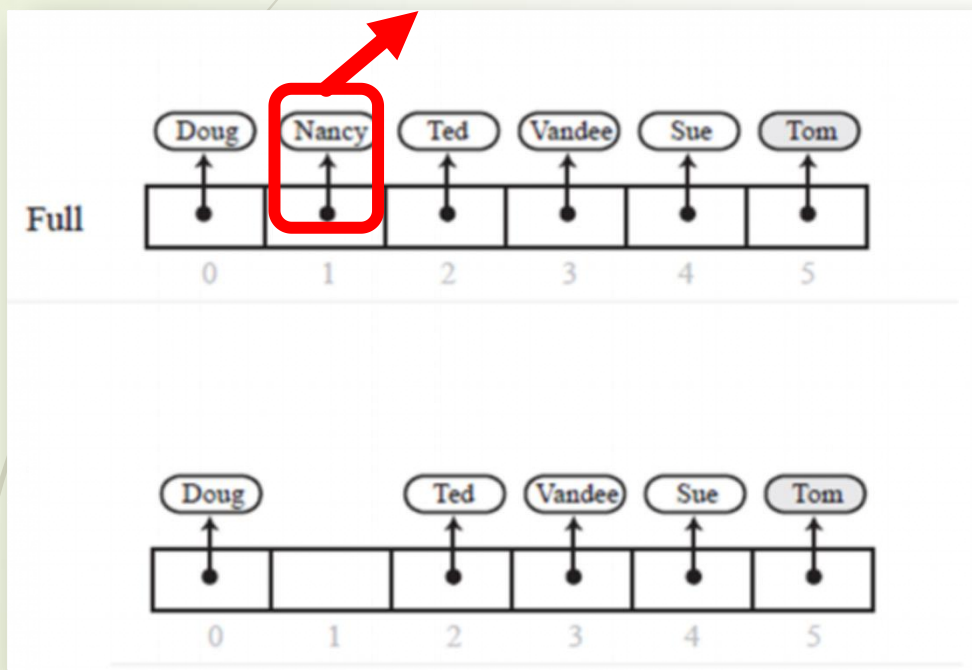
$(1+100) * 100 / 2$ ，就是加到一千，一万，也能很快口算出来。

例子---保持队列连续



一开始队列是空的，经过6个步骤，队列满了

例子---保持队列连续

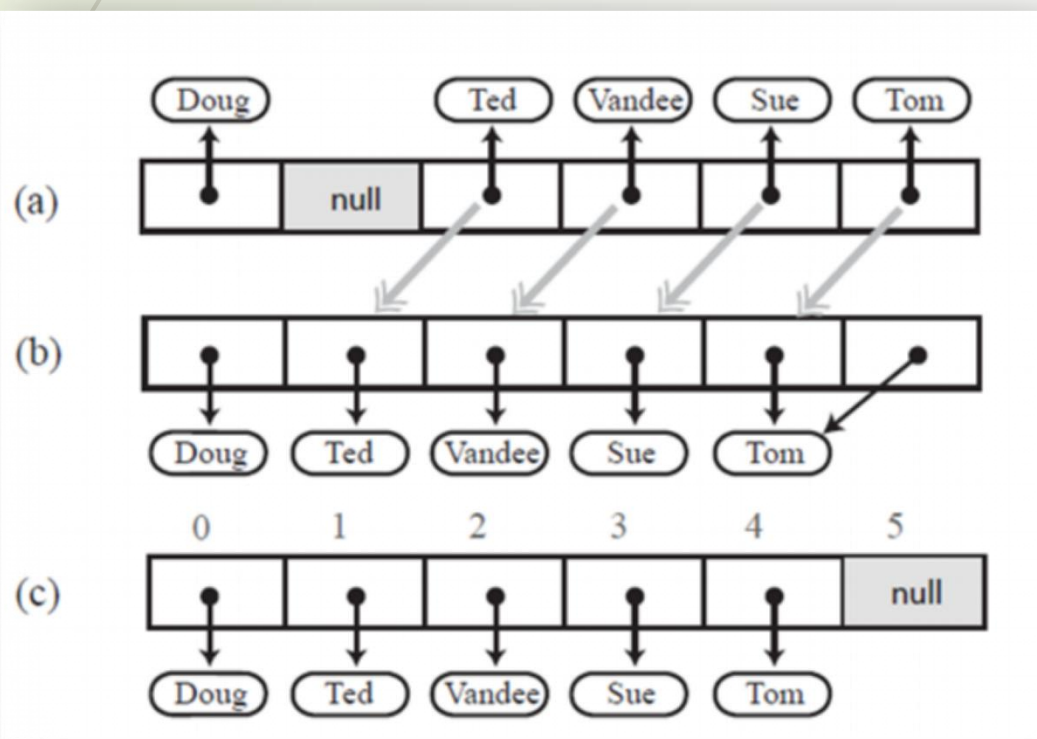


再经过1个步骤，队列中的“Nancy”离开了。

例子---保持队列连续

问题：如何保持队列连续？

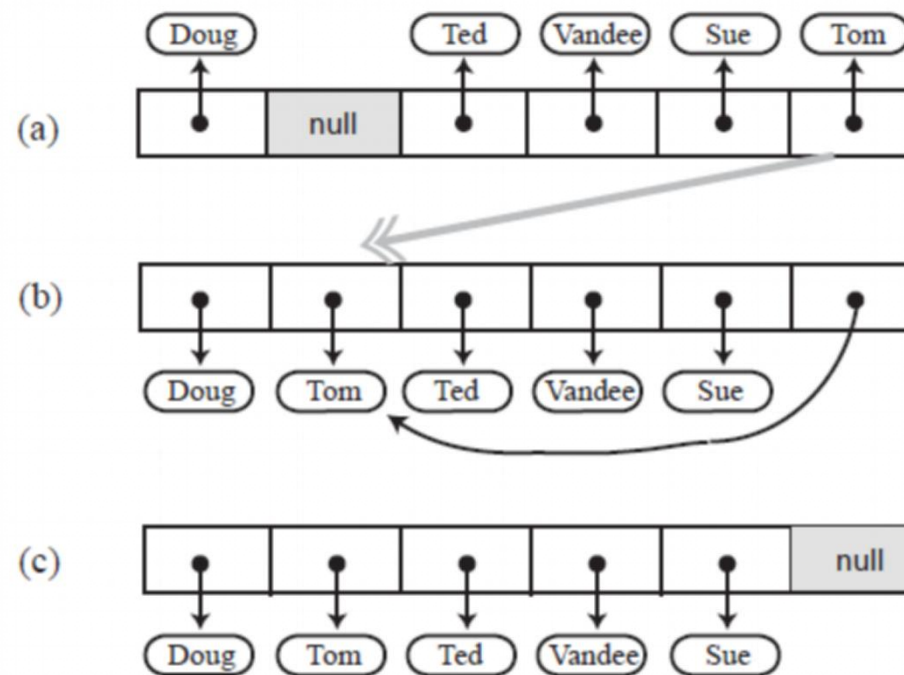
方法1：依次向前移动一个位置。



例子---保持队列连续

问题：如何保持队列连续？

方法2：将最后一个挪过来。



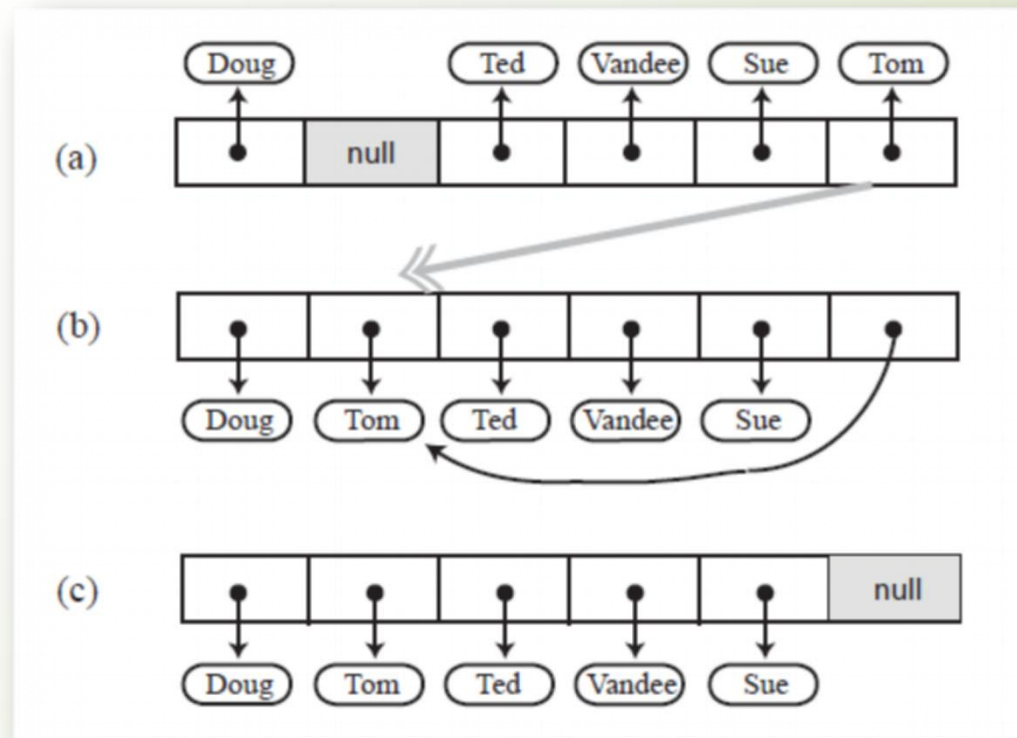
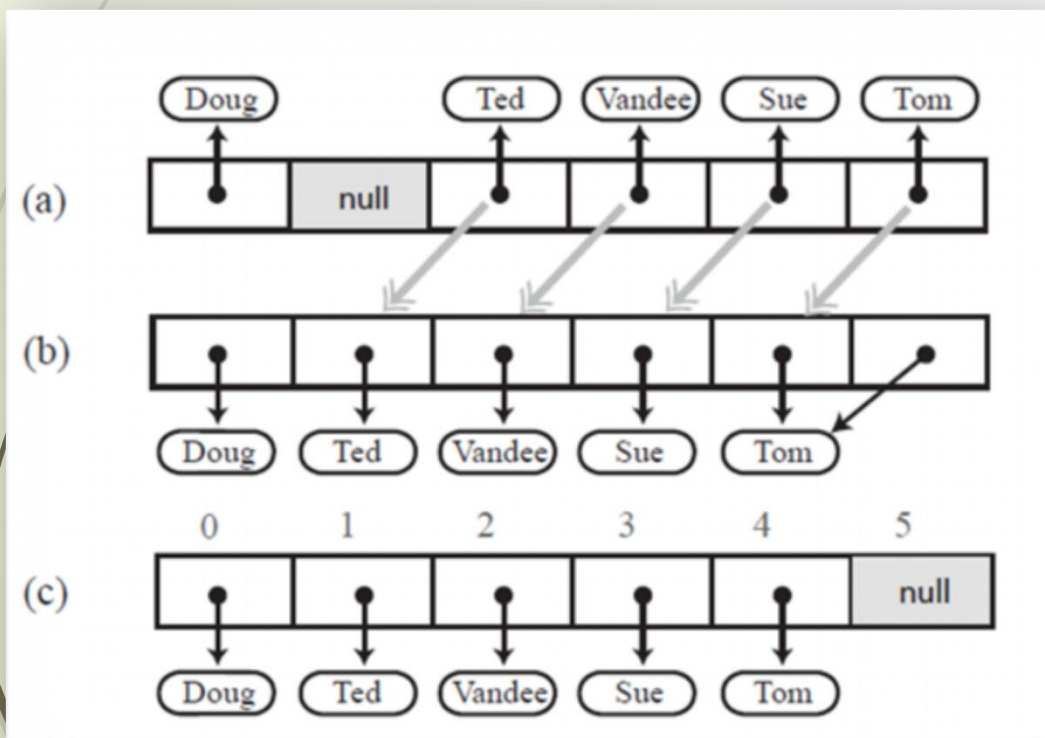
例子---保持队列连续

同样的逻辑结构,
同样的运算,
算法不同!

问题: 如何保持队列连续?

方法1: 依次向前移动一个位置。

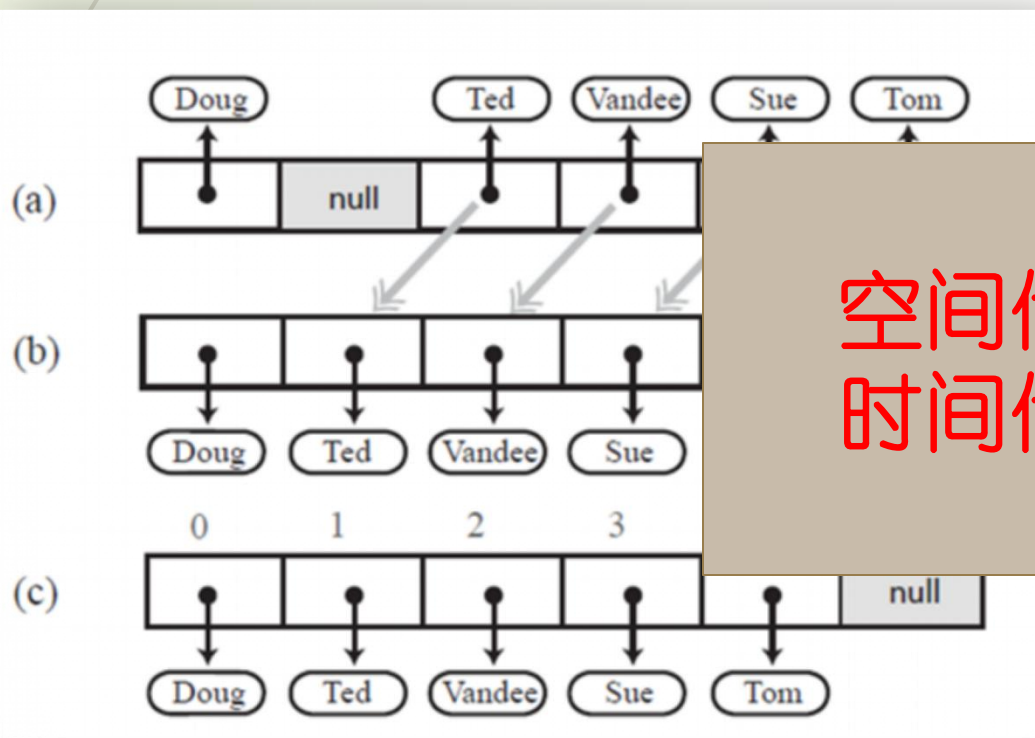
方法2: 将最后一个挪过来。



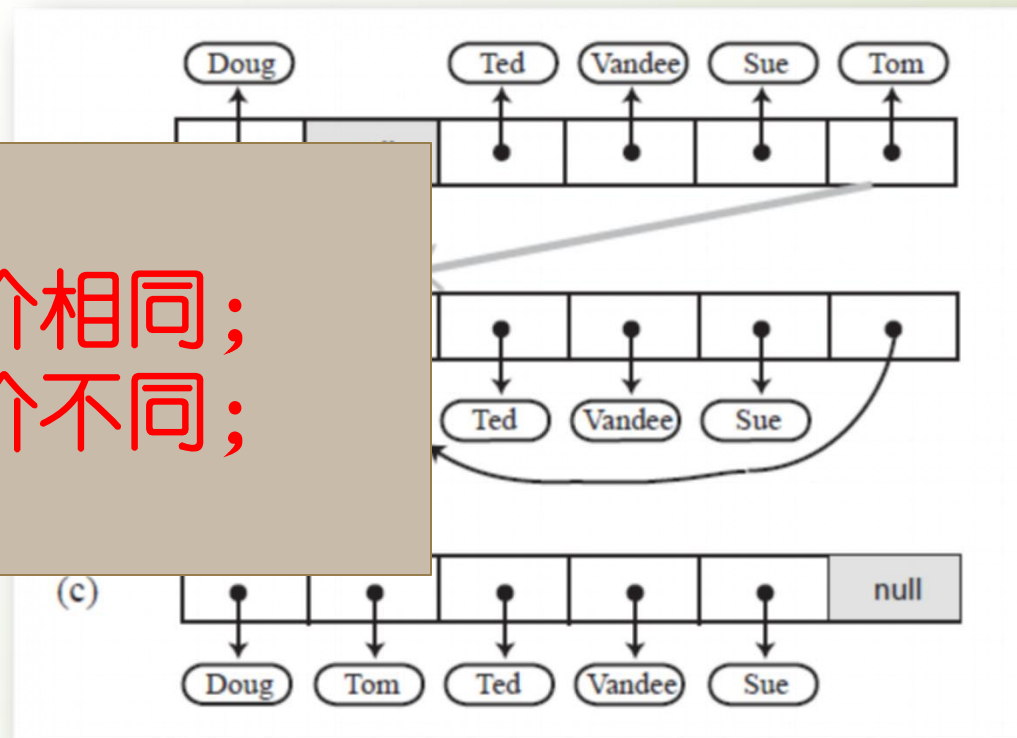
例子---保持队列连续

同样的逻辑结构,
同样的运算,
算法不同!

问题修改为：保持队列连续并且队员间的相互顺序不变？
如何解决？



空间代价相同；
时间代价不同；



1.事后统计：利用计算机内的计时功能，不同算法的程序可以用一组或多组相同的统计数据区分

缺点：

- ①必须先运行依据算法编制的程序
- ②所得时间统计量依赖于硬件、软件等环境因素，掩盖算法本身的优劣

如何克服缺点2：

用多种硬件、多种软件、多种数据进行多次组合测试 ----性能测试

签名集类型

1 个线程

2 个线程

4 个线程

8 个线程

16 个线程

18 个线程

表 2-2 输入规模为 100 万个数据元素时的最好、最坏及平均情况下的排序时间 (单位: 毫秒)

排序算法	平均情况	最坏情况 (逆序)	最好情况 (正序)
冒泡排序	549 432	1 534 035	366 936
选择排序	478 694	587 240	367 658
插入排序	253 115	515 621	0.897
希尔排序/增量 3	61	203	35
堆排序	79	126	74.8
归并排序	70	140	61
快速排序	39	93	30
基数排序/进制 100	117	118	116
基数排序/进制 1000	89	90	88

注: 1. 算法运行环境为 Intel 酷睿 2 双核 E8400, 3.0G, Windows 7*64。

图片来自《算法之道》P20

Frames:					(us)
					041222
					134333
					679667
512	100	169171785.6	692927.634	3383458640	36.364167
1024	99.297	85600180.49	701236.679	1712015076	32.688889
1280	96.484	66796374.12	683994.871	1335937516	30.182944
1518	100	58517140.29	710632.152	1170351092	42.11325

2.事前分析估计:

一个高级语言程序在计算机上运行所消耗的时间取决于:

- ①依据的算法选用何种策略
- ②问题的规模
- ③程序语言
- ④编译程序产生机器代码质量
- ⑤机器执行指令速度

同一个算法用不同的语言、不同的编译程序、在不同的计算机上运行，效率均不同，——使用**绝对时间单位**衡量算法效率不合适

算法效率

不同算法的比较，求 $1+2+3+\dots+100$ 的程序。

```
main()
{
    int i,s=0;n=100;
    s=(1+n)*n/2;
    printf("%d",s);
}
```

$$\begin{array}{r} s = 1 + 2 + 3 + \dots + 99 + 100 \\ + \quad s = 100 + 99 + 98 + \dots + 2 + 1 \\ \hline 2S = 101 + 101 + 101 + \dots + 101 + 101 \end{array}$$

```
main()
{
    int i,s=0;n=100;
    for(i=1;i<=100;i++)
        s=s+i;
    printf("%d",s);
}
```

这是数学家高斯的解释

$(1+100) * 100 / 2$ ，就是加到一千，一万，也能很快口算出来。

算法性能分析与度量

A 算法	B 算法
<u>int</u> s=0,n=1000; // 执行 1 次	<u>int</u> s=0,n=1000; // 执行 1 次
s=(1+n)*n/2; //执行 1 次	for(i=1;i= <u>n</u> ;i++) //执行 n+1 次
<u>printf</u> ("%" <u>d</u> ","s); // 执行 1 次	s= <u>s</u> +i; // 执行 n 次
	<u>printf</u> ("%" <u>d</u> ","s); // 执行 1 次
A 算法共执行 3 次	B 算法执行了 $2n+3$ 次

两个算法的第一条、最后一条语句是一样的，中间部分是我们关注的，把循环当作一个整体，忽略头尾循环判断的次数，这两个算法的差别就是1与n的区别。



算法有好坏之分!

即通过几个方面来对算法进行评价

- ◆ 正确性
- ◆ 可读性
- ◆ 健壮性
- ◆ 高效性
- ◆ 高效性（效率与低存储量需求）
 - ◆ 时间代价，效率
 - ◆ 空间代价，内存储器使用量需求
 - ◆ 这两个是互相影响的，要找到针对被解决的问题的最佳平衡点

算法的效率的度量

- 算法效率：用依据该算法编制的程序在计算机上执行所消耗的时间来度量
- 方法可分为：

事后统计

事前分析估计

1.4 算法

时间复杂度:

- 一个算法执行所耗费的时间是所有语句执行时间之和, 用 **$T(n)$** 表示, 假设每个语句执行时间相同时 (用 t_i 表示), 只需要统计所有语句执行的次数总和即可

$$T(n) = \sum_{i \in \{\text{语句集合}\}} t_i * c_i = \sum_{i \in \{\text{语句集合}\}} c_i$$

时间频度:

- 一个算法中的原操作执行次数称为语句频度或时间频度, 第 i 条语句的时间频度用 c_i 表示;

1.4 算法

算法1:

```
int i, sum =0, n=100;    //执行1次
for(i=1;i<=n;i++)        //执行n+1次
{
    sum=sum+i;            //执行n次
}
```

总共执行？ $2n+2$ 次

1.4 算法

算法2:

```
int i, sum=0, n=100; //执行1次
```

```
sum =(1+n)*n/2; //执行1次
```

总共执行2次

1.4 算法

算法3:

```
int i, j, sum = 0, n = 100; //执行1次
for(i = 1; i <= n; i++)      //执行n+1次
{
    for(j = 1; j <= n; j++)    //执行 (n+1) * n次
    {
        sum = sum + j;        //执行n*n次
    }
}
```

总: $1 + n + 1 + (n + 1) * n + n * n = 2n^2 + 2n + 2$

1.4 算法

对于问题输入规模 n ，四种算法执行的次数为：

$$2n+3$$

$$2n$$

$$3n+1$$

$$3n$$

次数	$2n+3$	$2n$	$3n+1$	$3n$
$n=1$	5	2	4	3
$n=2$	7	4	7	6
$n=3$	9	6	10	9
$n=10$	23	20	31	30
$n=100$	203	200	301	300

$5 > 4$

那个时间复杂度低？

$203 < 301$

1.4 算法

对于问题输入规模 n ,三种算法执行的次数为:

$$2n^2$$

$$3n+1$$

$$2n^2+3n+1$$

那个时间复杂度低?

次数	$2n^2$	$3n+1$	$2n^2+3n+1$	$10n$
$n=1$	2	4	6	10
$n=2$	8	7	15	20
$n=5$	50	16	66	50
$n=10$	200	31	231	100
$n=100$	20000	301	20301	1000

n 越大, 差异越小

时间复杂度的渐进表示法

- 算法中基本语句重复执行的次数是问题规模 n 的某个函数 $f(n)$, 算法的时间量度记作:

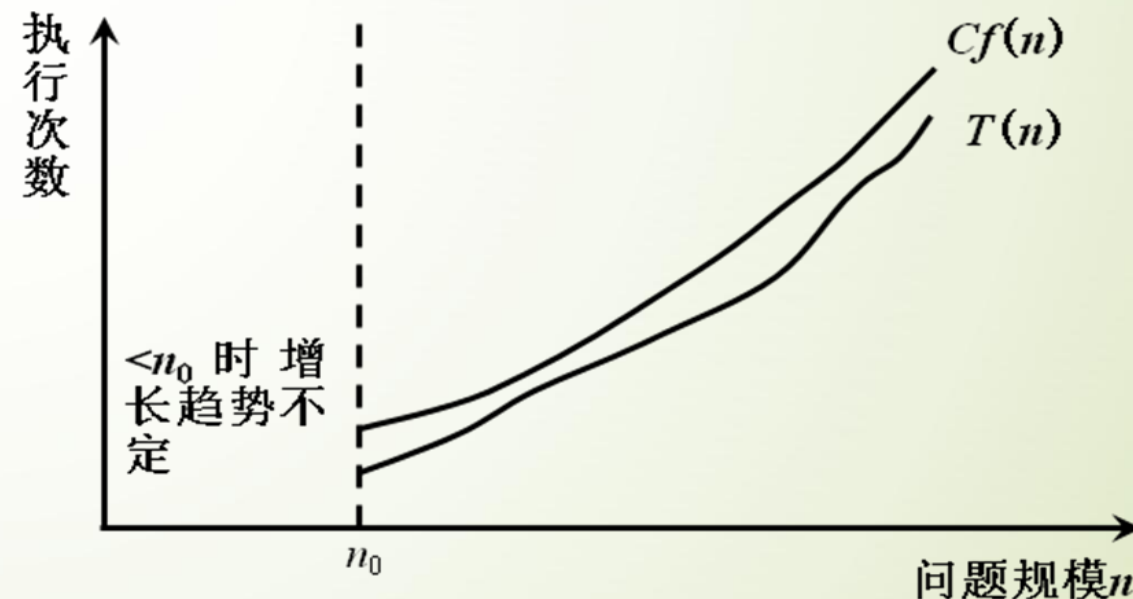
$$T(n) = O(f(n))$$

表示随着 n 的增大, 算法执行的时间的增长率和 $f(n)$ 的增长率相同, 称渐近时间复杂度。

$f(n)$ 通常选取容易衡量的函数: $\log_2 n$, n , $\log_2 n$, n^2 , n^3 , 2^n

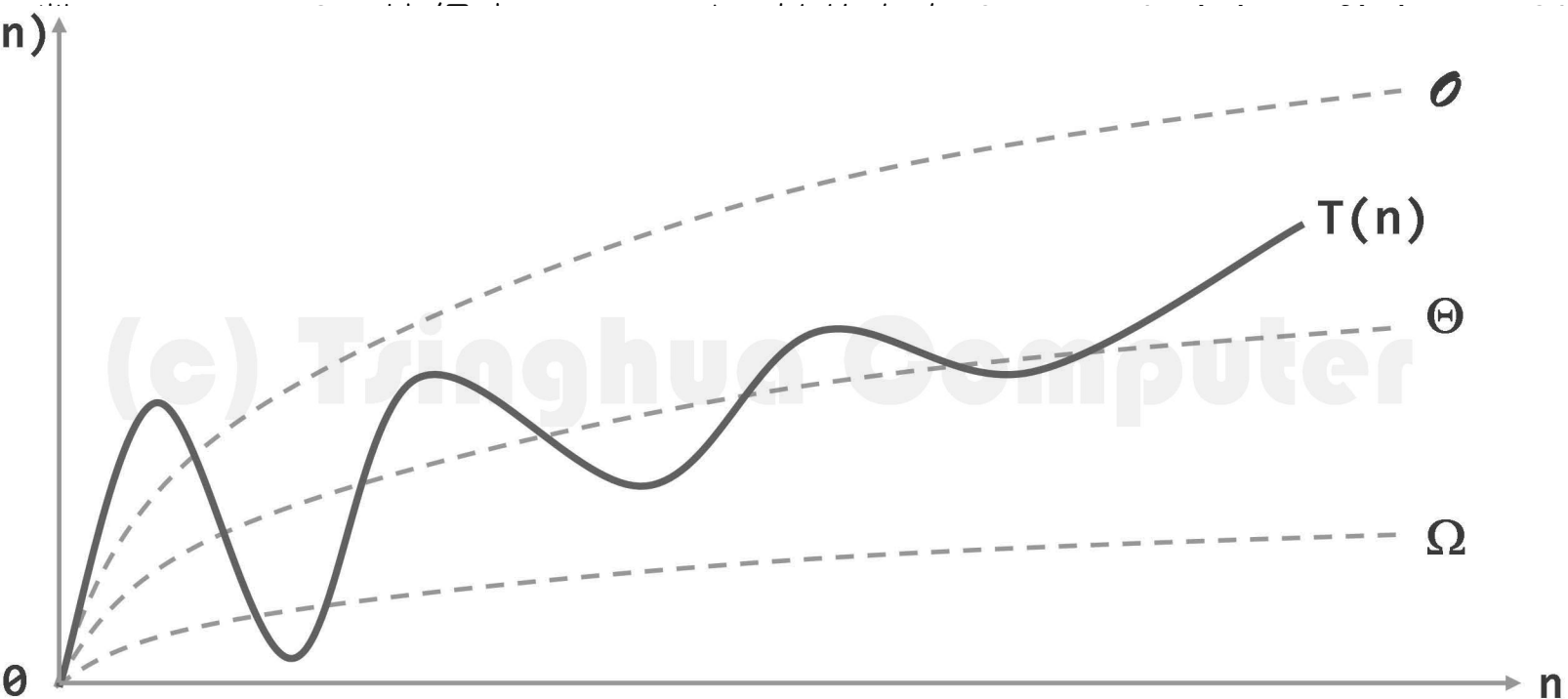
数学符号“ O ”的定义为:

若 $T(n)$ 和 $f(n)$ 是定义在正整数集合上的两个函数, 则 $T(n) = O(f(n))$ 表示存在正的常数 C 和 n_0 , 使得当 $n \geq n_0$ 时都满足 $0 \leq T(n) \leq Cf(n)$ 。



- 1、对于 $f(n)$ ，存在正数 n_0 、 c ，使得当 $n > n_0$ 时，始终存在 $0 \leq f(n) < c \cdot g(n)$ ，则可以用 $f(n) = o(g(n))$ 表示。
- 2、对于 $f(n)$ ，存在正数 n_0 、 c ，使得当 $n \geq n_0$ 时，始终存在 $0 \leq f(n) \leq c \cdot g(n)$ ，则可以用 $f(n) = O(g(n))$ 表示。
- 3、对于 $f(n)$ ，存在正数 n_0 、 c_1 、 c_2 ，使得当 $n \geq n_0$ 时，始终存在 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ ，则可以用 $f(n) = \Theta(g(n))$ 表示。
- 4、对于 $f(n)$ ，存在正数 n_0 、 c ，使得当 $n \geq n_0$ 时，始终存在 $f(n) \geq c \cdot g(n)$ ，则可以用 $f(n) = \Omega(g(n))$ 表示。
- 5、对于 $f(n)$ ，存在正数 n_0 、 c ，使得当 $n \geq n_0$ 时，始终存在 $f(n) \leq c \cdot g(n)$ ，则可以用 $f(n) = \omega(g(n))$ 表示。

采用上述5个符号的随 n 变化的情况



符号	读音		
o	/əʊ/, 小o		
O	/əʊ/, 大O	上界	相当于 $f(n) \leq c \cdot g(n)$ ，即实际时间低于或等于已知函数时间
Θ	/ˈθi:tə/, theta	精确的渐近行为	相当于 $f(n) = c \cdot g(n)$ ，即实际时间等于已知函数时间
Ω	/ʊˈmegə/, 大Omega	下界	相当于 $f(n) \geq c \cdot g(n)$ ，即实际时间高于已知函数时间
ω	/ʊˈmegə/, 小omega	松下界	相当于 $f(n) > c \cdot g(n)$ ，即实际时间高于或等于已知函数时间

1.4 算法

例1 分析以下程序段的时间复杂度

```
for (i=1;i<n;i++) {
```

```
    y=y+1;           //n-1
```

```
    for (j=0; j<=(2*n); j++)
```

```
        x++;          //(n-1)(2n+1) = 2n2 - n - 1
```

```
}
```

该程序段的时间复杂度:

$$T(n) = 2n^2 - n - 1 = O(n^2)$$

推导大O阶的方法

1) 程序运行时间中的常数用1代替。如运行3次，记为 $O(1)$ ，运行100次，也记为 $O(1)$ 。

2) 在大O（）函数中运行次数只取最高阶项，并且最高阶项的常数为1。如 $5n^2$ 记 $O(n^2)$ 。

它表示随问题规模 n 的增大算法的执行时间的增长率和 $f(n)$ 的增长率相同，称作算法的渐进时间复杂度，简称时间复杂度。

一般情况下，随 n 的增大， $T(n)$ 的增长较慢的算法为最优的算法。

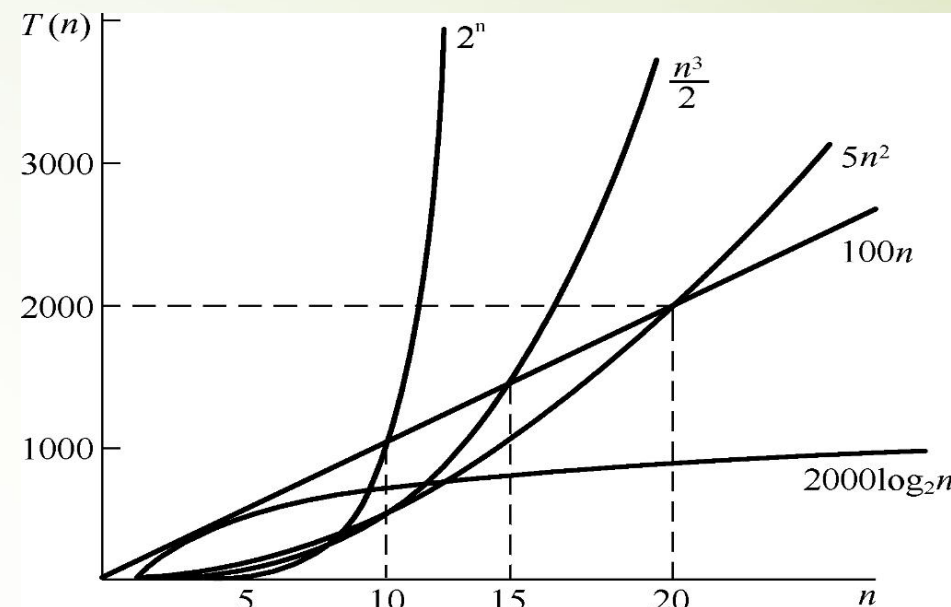
【例】在下列三段程序段中，给出原操作 $x=x+1$ 的时间复杂度分析。

(1) $x=x+1$ ；程序执行1次，其时间复杂度为 $O(1)$
称之为常量阶；

(2) for ($i=1$; $i \leq n$; $i++$) //执行 $n+1$ 次
 $x=x+1$; //执行 n 次
程序共执行 $2n+1$ ，只取最高项，去掉该项的系数，
 $T(n)=O(n)$ ，称之为线性阶；

(3) for ($i=1$; $i \leq n$; $i++$) //执行 $n+1$ 次
 for ($j=i$; $j \leq n$; $j++$) //执行 $n(n+1)$ 次
 $x=x+1$; //执行 n^2 次，
程序共执行 $2n^2+2n+1$ ，只取最高项，去掉该项的系数，
 $T(n)=O(n^2)$ ，称之为平方阶。

当n取得很大时，指数时间算法和多项式时间算法在所需时间上非常悬殊



时间复杂度T(n)按数量级递增顺序为：

时间复杂度低

时间复杂度高

常数阶	对数阶	线性阶	线性对数阶	平方阶	立方阶	...	K次方阶	指数阶
$O(1)$	$O(\log_2 n)$	$O(n)$	$O(n \log_2 n)$	$O(n^2)$	$O(n^3)$		$O(n^k)$	$O(2^n)$

$$O(1) < O(\log_2 n) < O(n) < O(n^2) < O(n^3) < O(2^n)$$

1.4 算法

例2、for(i=1, i<=n;++i)

for(j=1;j<=n;++j)

{

c[i][j]=0;

for(k=1;k<=n;++k)

c[i][j]+=a[i][k]*b[k][j];

}

由于是一个三重循环，每个循环从1到n，则总次数为： $n \times n \times n = n^3$

时间复杂度为 $T(n) = O(n^3)$

1.4 算法

例3、 `{++x;s=0;}`

将`++x`看成是基本操作，则语句频度为 1，即时间复杂度为 $O(1)$

如果将`s=0`也看成是基本操作，则语句频度为 2，其时间复杂度仍为 $O(1)$ ，即常量阶。

1.4 算法

例4、for(i=1;i<=n;++i)

{++x;s+=x;}

语句频度为：**n** 其时间复杂度为： **$O(n)$**

即时间复杂度为线性阶。

1.4 算法

- ➡ 例5、`for(i=1;i<=n;++i)`
- ➡ `for(j=1;j<=n;++j)`
- ➡ `{++x;s+=x;}`
- ➡ 语句频度为： n^2
- ➡ 其时间复杂度为： $O(n^2)$
- ➡ 即时间复杂度为平方阶。

分析算法时间复杂度的基本方法

- 找出语句频度最大的那条语句作为基本语句
- 计算基本语句的频度得到问题规模 n 的某个函数 $f(n)$
- 取其数量级用符号“ O ”表示

```
x = 0; y = 0;
```

```
for ( int k = 0; k < n; k ++ )
```

```
    x ++;
```

$T(n) = O(n^2)$

```
for ( int i = 0; i < n; i ++ )
```

```
    for ( int j = 0; j < n; j ++ )
```

$f(n)=n^2$

```
        y ++;
```

时间复杂度是由**嵌套最深层**语句的频度决定的

```
void exam ( float x[ ][ ], int m, int n ) {  
    float sum [ ];  
    for ( int i = 0; i < m; i++ ) {  
        sum[i] = 0.0;  
        for ( int j = 0; j < n; j++ )  
            sum[i] += x[i][j];  
    }  
    for ( i = 0; i < m; i++ )  
        cout << i << " : " << sum [i] << endl;  
}
```

$$T(n) = O(m*n)$$

$$f(n)=m*n$$

分析算法时间复杂度的基本方法

```
x = 0; y = 0;
```

```
for ( int k = 0; k < fun(n); k ++ )  
    x ++;
```

```
for ( int i = 0; i < n; i ++ )  
    for ( int j = 0; j < n; j ++ )  
        y ++;
```

第二段代码的时间复杂度是多少?
 $O(n^2)$ 吗?
要考虑 $\text{fun}(n)$ 的时间复杂度,

第三段代码是
 $O(n^2)$

- 找出语句频度最大的那条语句作为基本语句
- 在这个例子里面，对第二段代码要考虑 $\text{fun}(n)$ 的时间复杂度，即第二段代码的时间复杂度是： $O(n) * O(T(\text{fun}(n)))$
- 在不知道 $O(T(\text{fun}(n)))$ 的时间复杂度的情况下，不能贸然用第三段代码的时间复杂度来做为整体的时间复杂度。

有的情况下，算法中基本操作重复执行的次数还随问题的**输入数据集**不同而不同

例4：顺序查找，在数组a[i]中查找值等于e的元素，返回其所在位置。

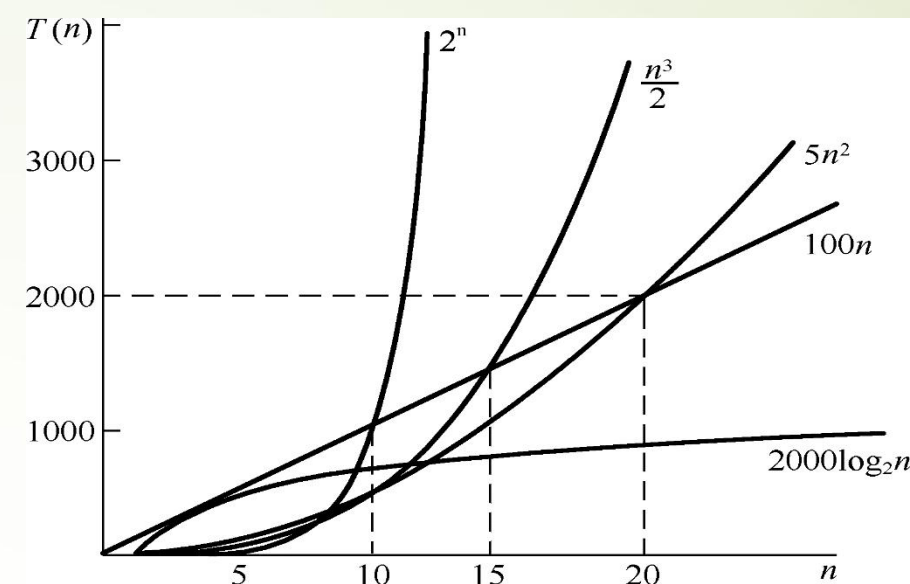
```
for (i=0;i< n;i++)  
    if (a[i]==e) return i+1;  
return 0;
```

- 最好情况：1次（排在第一个）
- 最坏情况：n（排在最后一个或不存在）
- 平均时间复杂度为： $O(n)$

注意和下面代码对比一下：

```
for ( int k = 0; k < n; k ++ )  
    x ++;  
是 $O(n)$ ，没有最好、最坏之分
```

当n取得很大时，指数时间算法和多项式时间算法在所需时间上非常悬殊



时间复杂度 $T(n)$ 按数量级递增顺序为：

复杂度低

复杂度高

常数阶	对数阶	线性阶	线性对数阶	平方阶	立方阶	...	K次方阶	指数阶
$O(1)$	$O(\log_2 n)$	$O(n)$	$O(n \log_2 n)$	$O(n^2)$	$O(n^3)$		$O(n^k)$	$O(2^n)$

$$O(1) < O(\log_2 n) < O(n) < O(n^2) < O(n^3) < O(2^n)$$

- 算法的存储空间需求
- 空间复杂度:算法所需存储空间的度量, 记作:
- $S(n)=O(f(n))$
- 其中n为问题的规模(或大小)

算法的存储空间

- 输入数据所占空间
- 程序本身所占空间
- 辅助变量所占空间

可分为：

- 固定部分：程序代码，常量，简单变量，定长的结构变量；
- 可变部分：与问题规模有关的存储空间
- 当问题规模较大时，可变部分可能会远大于固定部分，所以一般讨论算法的渐近空间复杂度，针对的是可变部分，分析方法和时间复杂度类似。

(3) 算法的空间复杂度

一个程序的空间复杂度 (**Space complexity**)

是指程序运行从开始到结束所需的存储量。

关于算法的存储空间需求，类似于算法的时间复杂度，为算法所需存储空间的量度，记做：

$$S(n) = O(f(n))$$

其中 n 为问题的规模。一般情况下，一个程序在机器上执行时，除了需要寄存本身所用的指令，常数，变量和输入数据以外，还需要一些对数据进行操作的**辅助存储空间**。其中对于**输入数据**所占的具体存储量只取决于问题本身，与算法无关，只需要分析该算法在实现时所需要的辅助空间单元个数就可以了。

若算法执行时所需要的辅助空间相对于输入数据量而言是**常数**，则称算法为**原地工作**，**辅助空间为 $O(1)$** 。

渐进空间复杂度

➡ 空间复杂度: 算法所需存储空间的度量, 记作:

$$S(n) = O(f(n))$$

其中 n 为问题的规模(或大小)

■ 算法要占据的空间

- 算法本身要占据的空间, 输入/输出, 指令, 常数, 变量等
- 算法要使用的辅助空间

例5：将一维数组a中的n个数逆序存放到原数组中。

$S(n) = O(1)$
原地工作

【算法1】
for(i=0;i<n/2;i++)
{
 t=a[i];
 a[i]=a[n-i-1];
 a[n-i-1]=t;
}

【算法2】
for(i=0;i<n;i++)
 b[i]=a[n-i-1];
for(i=0;i<n;i++)
 a[i]=b[i];

$S(n) = O(n)$



算法的执行时间的耗费和所序的存储空间的耗费两者是矛盾的，难以兼得。

即算法执行时间上的节省一定是以增加空间存储为代价的，反之亦然。就一般情况而言，常常以算法执行**时间**做为算法优劣的**主要衡量指标**。

- 1、数据、数据元素、数据项、数据结构等基本概念
- 2、对数据结构的两个层次的理解
 - 逻辑结构
 - 存储结构
- 3、抽象数据类型的表示方法
- 4、算法、算法的时间复杂度及其分析的简易方法

基本概念和术语



- 1、**数据** (data)—所有能输入到计算机中去的**描述客观事物的符号**
 - ◆ 数值性数据
 - ◆ 非数值性数据 (多媒体信息处理)
- 2、**数据元素** (data element) —数据的**基本单位**，也称**结点** (node) 或**记录** (**record**)，**c语言中用struct来定义一个结构化的数据类型**
- 3、**数据项** (data item) —有独立含义的数据**最小单位**，也称**域**(**field**)，**c语音中struct中的一个成员**。

三者之间的关系：数据 > 数据元素 > 数据项

例：

学生表 > 个人记录 > 学号、姓名……

文件目录树 > 文件结点记录 > 文件名称、属性、大小、块数量、起始文件块

方面 层次	数据表示	数据处理
抽象	逻辑结构	基本运算
实现	存储结构	算法
评价	不同数据结构的比较及算法分析	

逻辑结构

唯一

存储结构

不唯一

运算的实现

依赖于

存储结构

数据的逻辑结构

线性结构

非线性结构

线性表

栈、队列

串、数组

树形结构

图形结构

数据的存储结构

顺序存储

链式存储

数据的基本运算：插入、删除、修改、查找、排序

4、数据对象(Data Object): 相同特性数据元素的集合, 是数据的一个子集

- ◆ 整数数据对象

$$\mathbf{N} = \{ 0, \pm 1, \pm 2, \cdots \}$$

- ◆ 学生数据对象

- ➡ 学生记录的集合

按大O阶推导方法:

A算法 $f(n)=3$, 常数项3次用1代替, 记 **$O(1)$** 。

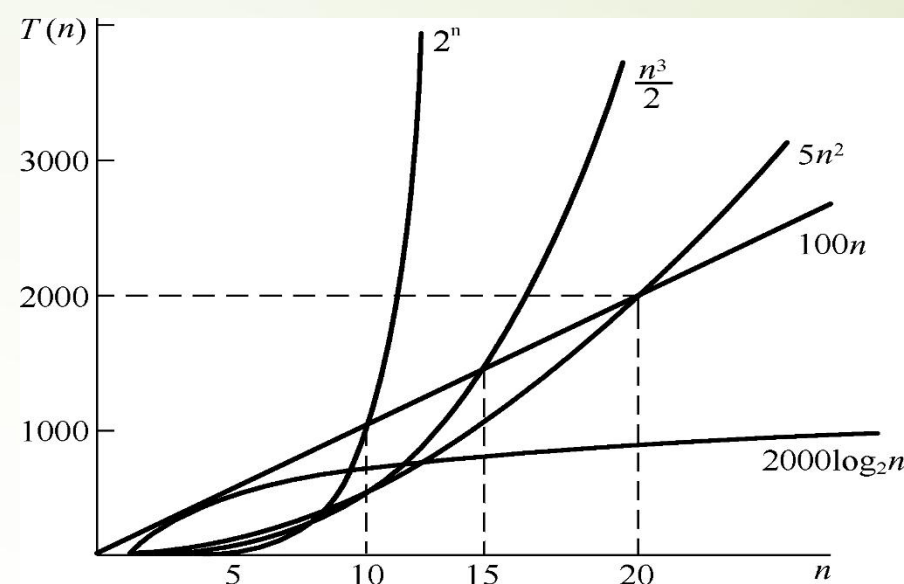
B算法 $f(n)=2n+3$, 常数项3次用1代替, $2n$ 的系数2也改为1, 记 **$O(n)$**

C算法 $f(n)=C(3n+5)$, 记为 **$O(n)$** 。

D算法 $f(n)=(2n^2)$, 记为 **$O(n^2)$** 。

E算法 $f(n)=(2n^2+3n+5)$, 记为 **$O(n^2)$** 。按最高项, 其余忽略不计。

当n取得很大时，指数时间算法和多项式时间算法在所需时间上非常悬殊



时间复杂度 $T(n)$ 按数量级递增顺序为：

复杂度低

复杂度高

常数阶	对数阶	线性阶	线性对数阶	平方阶	立方阶	...	K次方阶	指数阶
$O(1)$	$O(\log_2 n)$	$O(n)$	$O(n \log_2 n)$	$O(n^2)$	$O(n^3)$		$O(n^k)$	$O(2^n)$

$$O(1) < O(\log_2 n) < O(n) < O(n^2) < O(n^3) < O(2^n)$$